Martin-Luther-Universität Halle-Wittenberg

Naturwissenschaftliche Fakultät III

Institut für Informatik

Generierung von SQL-Übungsaufgaben

mit Musterlösung durch ein LLM

Masterarbeit

im Studiengang Informatik (M.Sc.)

Vorgelegt von: Paul Göttert

Matrikelnummer: 217231046

Erstgutachter: Prof. Dr. Stefan Brass

Zweitgutachter: Dr. Alexander Hinneburg

Halle (Saale), 17. September 2026

Inhaltsverzeichnis

1. Einführung **5**

1.1. Motivation 5

1.2. Methodik 7

2. Grundlagen und verwandte Arbeiten **9**

2.1. Sprachmodelle, Prompt Engineering und Modellanpassung 9

2.2. Text-to-SQL und SQL-to-Question 10

2.3. Automatische Analyse und Validierung von SQL-Anfragen 11

2.4. Verwandte Ansätze zur Generierung von SQL-Aufgaben 12

2.4.1. Automatische Generierung von Programmier- und SQL-Aufgaben . . 13

2.4.2. Mehrfachgenerierung und Reparatur von SQL-Anfragen 14

2.4.3. Synthese von Frage-SQL-Paaren 15

2.4.4. Knowledge-Graph-basierte Aufgabengenerierung 17

3. Generierung von SQL-Übungsaufgaben **18**

3.1. Grundidee der Pipeline 19

3.2. Phase 1: Erzeugung geprüfter Musterlösungs-Anfragen 21

3.2.1. Schema-Aufbereitung 21

3.2.2. Initiale SQL-Generierung 22

3.2.3. Analyse und Bewertung 24

3.2.4. Auswahl des besten SQL-Anfrage-Kandidaten 27

3.2.5. Optionale Modellanpassung 27

3.2.6. Verbesserung der SQL-Anfragen 28

3.2.7. Erneute Analyse der verbesserten SQL-Anfragen 29

3.2.8. Auswahl geprüfter Musterlösungs-Anfragen 30

3.3. Phase 2: Erzeugung vollständiger SQL-Übungsaufgaben 31

3.3.1. Erzeugung natürlichsprachlicher Aufgabentexte 31

3.3.2. Semantische Auswahl des Aufgabentextes 31

3.3.3. Erzeugung einer rekonstruierten SQL-Anfrage 33

3.3.4. Vergleich der Ausführungsergebnisse 33

3.3.5. Zusätzlicher Qualitätsfilter 34

3.3.6. Finaler Aufgabenpool 35

4. Implementierung **35**

4.1. Aufbereitung des Spider-Datensatzes . . . 37
4.2. Implementierung der Promptvarianten für die initiale SQL-Generierung . . . 38
4.3. Initiale SQL-Generierung . . . 39
4.4. Automatische Analyse der generierten SQL-Anfragen . . . 40
4.4.1. Parsing und semantische Validierung . . . 41
4.4.2. Ausführung der SQL-Anfragen . . . 42
4.4.3. Erkennung von Fremdschlüsselverwendungen in SQL-Anfragen . . . 43
4.4.4. Linting der SQL-Anfragen . . . 45
4.4.5. Prüfung der erwarteten SQL-Schwierigkeitskategorie . . . 47
4.5. Berechnung des Scores . . . 48
4.6. Auswahl des besten Kandidaten aus mehreren Promptvarianten . . . 49
4.7. Erzeugung der Trainingsdaten für SFT und DPO . . . 50
4.8. Fine-Tuning mit Axolotl . . . 51
4.9. Verbesserung generierter SQL-Anfragen durch ein zweites Modell . . . 52
4.10. Erneute Analyse der verbesserten SQL-Anfragen . . . 53
4.11. Erzeugung natürlichsprachlicher Aufgabentexte . . . 53
4.12. Semantische Auswahl des Aufgabentextes . . . 54
4.13. Rekonstruktionsprüfung des Aufgabentextes . . . 55
4.14. Implementierung des zusätzlichen Qualitätsfilters . . . 56
4.15. Struktur und Speicherung des finalen Aufgabenpools . . . 58

5. Evaluierung **59**

5.1. Gesamtergebnis und Evaluationsgrundlage . . . 59
5.2. Evaluation der Musterlösungs-Anfragen . . . 61
5.2.1. Promptentwicklung für die initiale SQL-Generierung . . . 62
5.2.2. Vergleich verschiedener Modelle in der initialen Generierung . . . 63
5.2.3. Initiale SQL-Generierung und Promptauswahl mit ArcticSQL . . . 64
5.2.4. Qualität nach SQL-Schwierigkeitskategorie . . . 67
5.2.5. Fehleranalyse vor und nach der Auswahl des besten SQL-Anfrage-Kandidaten . . . 68
5.2.6. Evaluation der Reparaturmodelle . . . 71
5.2.7. Ergebnisse der SQL-Qualitätssicherung . . . 74
5.3. Evaluation der vollständigen SQL-Übungsaufgaben . . . 75
5.3.1. Erzeugung und Auswahl der Aufgabentexte . . . 75

5.3.2. Rekonstruktionsprüfung und Qualitätsfilter 77
5.4. Diskussion 79

6. Fazit und Ausblick **82**
6.1. Beantwortung der Forschungsfrage 82
6.2. Ausblick 84

Anhang 86

A. Modellaufrufe und Inferenzparameter **86**

B. Promptvarianten der initialen SQL-Generierung **86**

C. Eingabestruktur der Fremdschlüsselprüfung **93**

D. Syntaxbaumdarstellung der Fremdschlüsselprüfung **95**

E. Pseudocode der Fremdschlüsselprüfung **96**

F. Beispiele zur Fremdschlüsselprüfung **97**
F.1. Auswertung spezieller SQL-Konstrukte 97
F.2. Beispiel eines Analyseergebnisses 101
F.3. Beispiel der Testvalidierung 102

G. Beispiele für SFT- und DPO-Trainingsdaten **103**

H. Technische Einstellungen der Fine-Tuning-Läufe **105**

I. Promptbeispiel der Aufgabentexterzeugung **107**

J. Beispiel zur Auswahl des besten Aufgabentextes **109**

K. Beispiel zur Rekonstruktionsprüfung **110**

L. Beispiel für eine finale Übungsaufgabe **111**

Literaturverzeichnis **112**

1. Einführung

SQL-Übungsaufgaben verbinden mehrere voneinander abhängige Bestandteile: ein Datenbankschema, einen konkreten Datenbankzustand, einen präzisen Aufgabentext und eine dazu passende Musterlösungs-Anfrage. Bereits Abweichungen zwischen einzelnen Bestandteilen können dazu führen, dass eine Aufgabe nicht eindeutig lösbar oder ihre Musterlösungs-Anfrage fachlich ungeeignet ist. Gleichzeitig werden für die Lehre umfangreiche Aufgabenpools benötigt, die unterschiedliche SQL-Konstrukte und Anforderungen abdecken. Diese Arbeit untersucht, inwieweit sich solche Aufgaben mithilfe von Sprachmodellen automatisiert erzeugen und durch nachgelagerte Prüfverfahren absichern lassen.

1.1. Motivation

In der Datenbanklehre werden umfangreiche Aufgabenpools benötigt, um unterschiedliche SQL-Konstrukte und SQL-Schwierigkeitskategorien abzudecken. Eine größere Auswahl ermöglicht wiederholte Übungsangebote, unterschiedliche Klausurvarianten und die Anpassung der Aufgaben an den jeweiligen Lernstand. Da jede Aufgabe aus einem Schema, einem Datenbankzustand, einem eindeutigen Aufgabentext und einer passenden Musterlösungs-Anfrage besteht, ist ihre manuelle Erstellung und Prüfung mit erheblichem Aufwand verbunden.

Sprachmodelle können Lehrende bei der Formulierung einzelner Aufgaben unterstützen. Eine direkte interaktive Verwendung, beispielsweise durch einen einzelnen ChatGPT-Dialog, erfordert jedoch weiterhin, dass jede Eingabe manuell erstellt und jedes Ergebnis einzeln geprüft und gegebenenfalls korrigiert wird. Sie erzeugt daher noch keinen umfangreichen Aufgabenpool nach einheitlichen Anforderungen. Gegenstand dieser Arbeit ist stattdessen ein automatisierter Ablauf, der eine große Zahl von Aufgaben systematisch erzeugt, nach denselben Kriterien bewertet und ungeeignete Ergebnisse vor der weiteren Verwendung herausfiltert.

Die automatische Erzeugung solcher Aufgaben ist mit mehreren Herausforderungen verbunden. Eine generierte SQL-Anfrage muss zum vorgegebenen Schema passen, ausführbar sein und die jeweils geforderten SQL-Konstrukte enthalten. Darüber hinaus

können syntaktisch gültige Anfragen semantisch auffällige oder für eine Übungsaufgabe ungeeignete Ergebnisse liefern. Bei der anschließenden Formulierung des Aufgabentextes muss zudem sichergestellt werden, dass der beschriebene Informationsbedarf tatsächlich mit der Musterlösungs-Anfrage übereinstimmt. Die plausible Formulierung einzelner Bestandteile genügt daher nicht. Entscheidend ist ihre gemeinsame Konsistenz.

Für Programmieraufgaben wurde bereits gezeigt, dass große Sprachmodelle Aufgaben, Musterlösungen und Erklärungen erzeugen können [1]. Diese Ergebnisse deuten darauf hin, dass Sprachmodelle Lehrende bei der Erstellung von Übungsmaterial unterstützen können. Erste Untersuchungen zur automatischen Erzeugung von SQL-Übungsaufgaben mit Sprachmodellen bewerten diesen Ansatz ebenfalls als grundsätzlich vielversprechend [2]. Zugleich zeigen diese Arbeiten, dass die fachliche Qualität generierter Aufgaben nicht vorausgesetzt werden kann, sondern systematisch überprüft werden muss.

Die Erzeugung mehrerer Kandidaten pro Eingabe dient dabei zwei Zielen. Zum einen kann ein breiter Aufgabenpool unterschiedliche Datenbankschemata, SQL-Konstrukte und SQL-Schwierigkeitskategorien abdecken. Zum anderen stehen mehrere alternative Modellausgaben für die Auswahl zur Verfügung. Dies ist relevant, weil die Qualität einer einzelnen Generierung von der Promptgestaltung und der stochastischen Ausgabe des Sprachmodells abhängt.

Aus diesem Spannungsfeld ergibt sich die Notwendigkeit eines Verfahrens, das die generativen Fähigkeiten von Sprachmodellen mit systematischen Analyse- und Validierungsschritten verbindet. Ziel dieser Arbeit ist daher die Entwicklung und Evaluation einer mehrstufigen Pipeline zur automatischen Erzeugung und Prüfung von SQL-Übungsaufgaben.

Qualität wird in dieser Arbeit anhand festgelegter, automatisiert überprüfbarer Kriterien operationalisiert. Für eine Musterlösungs-Anfrage umfassen diese Kriterien insbesondere die syntaktische Verarbeitbarkeit, das Bestehen der schema- und ausführungsbezogenen Prüfungen, das Fehlen erkannter semantisch auffälliger Konstruktionen sowie die Einhaltung der vorgegebenen SQL-Schwierigkeitskategorie. Für vollständige Übungsaufgaben wird zusätzlich geprüft, ob sich aus dem Aufgabentext eine Anfrage

rekonstruieren lässt, deren Ausführungsergebnis mit dem Ergebnis der Musterlösungs-Anfrage übereinstimmt, und ob der zusätzliche Qualitätsfilter einen Ausschlussgrund erkennt. Gemessen wird Qualität somit anhand des Anteils der Generierungsdurchläufe, deren Ergebnisse sämtliche für die jeweilige Pipelinephase vorgesehenen Prüfungen bestehen. Diese Operationalisierung ermöglicht eine quantitative Evaluation, garantiert jedoch keine vollständige fachliche Fehlerfreiheit.

Die zentrale Forschungsfrage lautet, welcher Anteil der Generierungsdurchläufe einer sprachmodellgestützten und automatisch geprüften Pipeline für vorgegebene Datenbankschemata und unterschiedliche SQL-Schwierigkeitskategorien zu vollständigen SQL-Übungsaufgaben führt, die sämtliche operationalisierten Qualitätskriterien erfüllen. Ergänzend wird untersucht, welche Unterschiede bei verschiedenen Formen der Schemaübergabe, Promptgestaltungen und Sprachmodellen auftreten und wie sich die nachgelagerte Verbesserung fehlerhafter Anfragen in den beobachteten Zwischenergebnissen der Pipeline widerspiegelt.

Der Beitrag dieser Arbeit liegt in der Konzeption, Implementierung und Evaluation eines durchgängigen Verfahrens, das die Erzeugung von Musterlösungs-Anfragen und natürlichsprachlichen Aufgabentexten mit mehreren automatisierten Qualitätsprüfungen verbindet. Im Unterschied zur interaktiven Erzeugung einzelner Aufgaben zielt das Verfahren auf einen umfangreichen, nach einheitlichen Kriterien vorgeprüften Aufgabenpool. Hierzu werden alternative Modellausgaben erzeugt und bewertet, erkannte Fehler für eine mögliche Verbesserung genutzt und die Zuordnung zwischen Aufgabentext und Musterlösungs-Anfrage gesondert validiert.

1.2. Methodik

Zur Beantwortung der Forschungsfrage wird ein mehrstufiges Verfahren entwickelt, das im Folgenden als Pipeline bezeichnet wird. Anstatt Aufgabentext und Musterlösungs-Anfrage gemeinsam in einem einzelnen Modellaufruf zu erzeugen, entstehen beide Bestandteile nacheinander. Zunächst wird die SQL-Anfrage generiert, da sie weitgehend automatisiert geprüft werden kann. Erst auf Grundlage einer geprüften Musterlösungs-Anfrage wird der zugehörige Aufgabentext formuliert. Diese Trennung erleichtert es,

Fehler einem Verarbeitungsschritt zuzuordnen und ungeeignete Zwischenergebnisse frühzeitig auszusondern.

Die Pipeline gliedert sich entsprechend in zwei Phasen. Als Datengrundlage dienen die Schemata und Datenbankzustände des Spider-Datensatzes [3]. In der ersten Phase erzeugt ein Sprachmodell für dieselbe Eingabe mehrere SQL-Anfrage-Kandidaten mithilfe unterschiedlicher Promptvarianten. Die Mehrfachgenerierung berücksichtigt, dass eine einzelne Modellausgabe keine verlässliche Grundlage für die Auswahl einer Musterlösungs-Anfrage bildet. Die Kandidaten werden anhand mehrerer automatischer Prüfungen bewertet, sodass die Auswahl an explizite Kriterien gebunden ist und kein zusätzliches Modellurteil erfordert. Erkannte Fehler können anschließend für eine gezielte Überarbeitung genutzt werden. In die zweite Phase gelangen nur Musterlösungs-Anfragen, welche die vorgesehenen Prüfungen vollständig bestehen.

In der zweiten Phase werden zu jeder geprüften Musterlösungs-Anfrage mehrere Aufgabentexte erzeugt und aus diesen eine geeignete Formulierung ausgewählt. Die Verwendung mehrerer Kandidaten verringert auch hier die Abhängigkeit von einer einzelnen Modellausgabe. Zur Überprüfung der Zuordnung wird aus dem ausgewählten Aufgabentext erneut eine SQL-Anfrage erzeugt. Deren Ausführungsergebnis wird mit dem Ergebnis der Musterlösungs-Anfrage verglichen. Dieser Vergleich wird anstelle einer textuellen Gleichheitsprüfung verwendet, da unterschiedlich formulierte SQL-Anfragen denselben Informationsbedarf ausdrücken können. Ein zusätzlicher Qualitätsfilter entfernt erkannte problematische Aufgaben oder markiert sie für eine fachliche Kontrolle.

Die Evaluation untersucht, wie sich die Zwischenergebnisse bei den betrachteten Gestaltungsentscheidungen unterscheiden und wie sich die Datenmenge entlang der beiden Pipelinephasen verändert. Die automatische Verarbeitung ersetzt keine fachliche Beurteilung. Vor dem produktiven Einsatz prüfen Lehrende die ausgewählten Aufgaben abschließend und entscheiden über ihre Freigabe. Diese Einbindung einer menschlichen Fachentscheidung entspricht einem Human-in-the-Loop-Ansatz.

Kapitel 2 erläutert die fachlichen und technischen Grundlagen und ordnet die Arbeit in bestehende Ansätze ein. Kapitel 3 beschreibt die entwickelte Pipeline auf konzeptioneller Ebene. Kapitel 4 dokumentiert ihre konkrete Implementierung. Die Ergebnisse

der einzelnen Verarbeitungsschritte werden in Kapitel 5 evaluiert. Kapitel 6 fasst die wesentlichen Erkenntnisse zusammen und zeigt mögliche Weiterentwicklungen auf.

2. Grundlagen und verwandte Arbeiten

Dieses Kapitel führt die für die entwickelte Pipeline erforderlichen Grundlagen ein und ordnet die Arbeit in den bestehenden Forschungsstand ein. Zunächst werden Sprachmodelle, Prompting-Strategien und Verfahren zur Modellanpassung erläutert. Anschließend werden die Übersetzung zwischen natürlicher Sprache und SQL sowie Möglichkeiten zur automatischen Prüfung erzeugter Anfragen betrachtet. Abschließend werden verwandte Ansätze zur automatischen Erzeugung von Programmier- und SQL-Aufgaben eingeordnet.

2.1. Sprachmodelle, Prompt Engineering und Modellanpassung

Große Sprachmodelle erzeugen Text, indem sie auf Grundlage des bisherigen Eingabekontexts Wahrscheinlichkeiten für nachfolgende Tokens bestimmen. Vaswani et al. stellen in *Attention Is All You Need* die Transformer-Architektur vor, die Abhängigkeiten innerhalb einer Sequenz mithilfe von Aufmerksamkeitsmechanismen verarbeitet [4]. Als Vortraining wird die erste Trainingsphase bezeichnet, in der ein Modell aus umfangreichen Text- und Codebeständen allgemeine sprachliche und strukturelle Muster lernt. Dadurch können große Sprachmodelle neben natürlicher Sprache auch strukturierte Ausgaben wie SQL-Anfragen erzeugen.

Die Ausgabe eines Sprachmodells lässt sich durch die Gestaltung seiner Eingabe, des Prompts, steuern. Prompt Engineering bezeichnet die gezielte Formulierung dieses Prompts, um Aufgabe, Kontext und erwartetes Ausgabeformat eindeutig vorzugeben. Für die SQL-Generierung betrifft dies insbesondere die Darstellung des Datenbankschemas, die Beschreibung der gewünschten Anfrageform und die Vorgabe eines verwertbaren Ausgabeformats. Wei et al. untersuchen in *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models* die Anleitung eines Modells zu expliziten Herleitungsschritten, die insbesondere mehrstufige Schlussfolgerungsaufgaben unterstützen

können [5]. Brown et al. zeigen in *Language Models Are Few-Shot Learners*, wie wenige Beispiele der zu bearbeitenden Aufgabe in den Prompt aufgenommen werden können [6]. Diese Beispiele sind ausschließlich Teil des Eingabekontexts. Eine Anpassung der Modellparameter findet dabei nicht statt. Mehrere Promptvarianten können daher für dieselbe Eingabe unterschiedliche Kandidaten erzeugen, die anschließend anhand einheitlicher Kriterien verglichen werden.

Embedding-Modelle erzeugen demgegenüber keinen neuen Text, sondern überführen Wörter, Sätze oder Textabschnitte in numerische Vektoren fester Länge. Semantisch ähnliche Texte sollen im Vektorraum nahe beieinander liegen, sodass ihre Ähnlichkeit etwa über die Kosinusähnlichkeit bestimmt werden kann. Reimers und Gurevych stellen in *Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks* ein Verfahren zur Erzeugung effizient vergleichbarer Satzrepräsentationen vor [7]. In dieser Arbeit bilden solche Repräsentationen die Grundlage für die semantische Auswahl aus mehreren alternativen Aufgabentexten.

Von der Steuerung durch Prompts ist die Anpassung der Modellparameter zu unterscheiden. Beim Supervised Fine-Tuning (SFT) wird ein vortrainiertes Modell mit aufgabenspezifischen Eingabe-Ausgabe-Paaren weitertrainiert. Speicherärmere Verfahren wie die von Hu et al. in *LoRA: Low-Rank Adaptation of Large Language Models* vorgestellte Low-Rank Adaptation (LoRA) verändern dafür nicht sämtliche Modellgewichte, sondern trainieren zusätzliche Matrizen mit niedrigem Rang [8]. Dettmers et al. verbinden LoRA in *QLoRA: Efficient Finetuning of Quantized LLMs* mit einer quantisierten Repräsentation des Basismodells [9]. Rafailov et al. stellen in *Direct Preference Optimization: Your Language Model Is Secretly a Reward Model* ein Verfahren vor, das Paare aus einer bevorzugten und einer abgelehnten Antwort als Trainingssignal nutzt [10]. Diese Verfahren werden in der Pipeline optional zur Anpassung eines Modells für die Reparatur fehlerhafter SQL-Anfragen eingesetzt.

2.2. Text-to-SQL und SQL-to-Question

Text-to-SQL bezeichnet die Überführung eines natürlichsprachlich formulierten Informationsbedarfs in eine SQL-Anfrage unter Berücksichtigung des zugehörigen Datenbankschemas. Einen Überblick über dieses Forschungsgebiet geben Deng et al. in *Recent*

Advances in Text-to-SQL: A Survey of What We Have and What We Expect [11]. Die entgegengesetzte Richtung wird als SQL-to-Question beziehungsweise SQL-to-Text bezeichnet. Dabei wird der Inhalt einer vorgegebenen SQL-Anfrage in natürlicher Sprache wiedergegeben. Xu et al. untersuchen diese Aufgabe in *SQL-to-Text Generation with Graph-to-Sequence Model* [12].

Beide Richtungen bilden unterschiedliche Abbildungen zwischen natürlicher Sprache, Datenbankschema und SQL. Während Text-to-SQL auf die Erzeugung einer ausführbaren Anfrage zielt, ist das Ergebnis von SQL-to-Question eine sprachliche Beschreibung des durch die Anfrage ausgedrückten Informationsbedarfs.

2.3. Automatische Analyse und Validierung von SQL-Anfragen

Automatische Prüfverfahren für SQL-Anfragen lassen sich nach der jeweils untersuchten Eigenschaft unterscheiden. Die syntaktische Analyse prüft, ob eine Anfrage gemäß der SQL-Grammatik verarbeitet werden kann. Eine schemaabhängige Validierung kontrolliert, ob referenzierte Tabellen und Spalten vorhanden sind und verwendete Ausdrücke zu den zugehörigen Datentypen passen. Durch die Ausführung auf einem konkreten Datenbankzustand lässt sich feststellen, ob die Anfrage ausführbar ist und welches Ergebnis sie liefert. Die semantische Analyse sucht darüber hinaus nach auffälligen Konstruktionen, die trotz syntaktischer Gültigkeit und Ausführbarkeit auf einen inhaltlichen Fehler hindeuten können. Diese Prüfungsebenen ergänzen einander. Das Bestehen einer einzelnen Prüfung belegt daher noch nicht die fachliche Korrektheit einer Anfrage.

Für die semantische Analyse einzelner SQL-Anfragen beschreiben Brass und Goldberg in *Semantic Errors in SQL Queries: A Quite Complete List* Fehlerklassen, die von Datenbanksystemen üblicherweise nicht gemeldet werden [13]. Dazu zählen beispielsweise widersprüchliche Bedingungen, problematische `NULL`-Vergleiche und unnötige Konstruktionen. SQLlint setzt diese Fehlerklassen in Regeln um und meldet entsprechende Auffälligkeiten in einer Anfrage [14]. Eine Meldung ist dabei als Hinweis auf einen möglichen semantischen Fehler zu verstehen und nicht als Beweis dafür, dass die Anfrage inhaltlich falsch ist. Einen verwandten Ansatz stellen Dintyala et al. in

SQLCheck: Automated Detection and Diagnosis of SQL Anti-Patterns vor [15]. SQLCheck erkennt und diagnostiziert SQL-Anti-Patterns durch statische Anfrageanalyse und Informationen aus dem Datenbanksystem.

Davon zu unterscheiden ist die ausführungsbasierte Validierung zweier SQL-Anfragen. Hierbei werden beide Anfragen auf demselben Datenbankzustand ausgeführt und ihre Ergebnisse miteinander verglichen. Übereinstimmende Ergebnisse können auf semantische Äquivalenz hindeuten, beweisen diese jedoch nicht: Inhaltlich verschiedene Anfragen können auf einem einzelnen Datenbankzustand zufällig dasselbe Ergebnis liefern. Zhong et al. begegnen dieser Einschränkung in *Semantic Evaluation for Text-to-SQL with Distilled Test Suites* mit destillierten Testsuiten, in denen Anfragen auf mehreren gezielt ausgewählten Datenbankzuständen ausgeführt werden [16]. Kim et al. ergänzen die ausführungsbasierte Bewertung in *FLEX: Expert-level False-Less EXecution Metric for Text-to-SQL Benchmark* um eine modellgestützte Beurteilung, um falsch positive und falsch negative Bewertungen etablierter Metriken zu reduzieren [17].

Die Kombination der genannten Verfahren bildet die konzeptionelle Grundlage für die mehrstufige Bewertung der erzeugten SQL-Anfragen. Sie erhöht die Aussagekraft der Prüfung, kann eine vollständige fachliche Korrektheit jedoch nicht garantieren.

2.4. Verwandte Ansätze zur Generierung von SQL-Aufgaben

Die für diese Arbeit relevanten Ansätze lassen sich nach ihrer jeweiligen Zielsetzung in vier Gruppen einteilen. Die erste Gruppe untersucht die automatische Erzeugung unmittelbar einsetzbarer Programmier- oder SQL-Aufgaben. Eine zweite Gruppe befasst sich mit der robusteren Erzeugung von SQL-Anfragen durch mehrere Kandidaten und nachgelagerte Korrekturen. Weitere Arbeiten betrachten gezielt die Synthese natürlichsprachlicher Fragen aus SQL-Anfragen oder verwenden Knowledge Graphs als strukturierte Grundlage der Aufgabengenerierung. Die Ansätze werden im Folgenden hinsichtlich ihres Vorgehens, ihrer Evaluation und ihres Bezugs zu der in dieser Arbeit entwickelten Pipeline eingeordnet.

2.4.1. Automatische Generierung von Programmier- und SQL-Aufgaben

Sarsa et al. untersuchen in *Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models* die automatische Erzeugung von Programmieraufgaben mit Codex [1]. Das Sprachmodell generiert auf Grundlage inhaltlicher Vorgaben sowohl Aufgabenbeschreibungen als auch Musterlösungen, Tests und Erklärungen. Die erzeugten Materialien wurden anschließend durch Menschen beurteilt. Dabei wurden 75,0 % der Aufgaben als sinnvoll und 81,8 % als neuartig bewertet. Von den 240 erzeugten Aufgaben enthielten 84,6 % eine Musterlösung. 89,7 % dieser Musterlösungen waren ausführbar. Von den 165 Aufgaben, für die sowohl eine Musterlösung als auch Tests vorlagen, bestanden jedoch nur 30,9 % sämtliche Tests. Die Untersuchung zeigt damit, dass Sprachmodelle zusammenhängende Lehrmaterialien erzeugen können, ihre Ausgaben aber nicht ohne eine zusätzliche Qualitätskontrolle übernommen werden sollten.

Aerts et al. übertragen diese Fragestellung in *A Feasibility Study on Automated SQL Exercise Generation with ChatGPT-3.5* auf SQL-Übungsaufgaben [2]. In ihrer Machbarkeitsstudie erzeugt ChatGPT-3.5 zunächst Datenbankschemata und Beispieldaten und anschließend dazugehörige Aufgaben und Musterlösungs-Anfragen. Unterschiedliche Prompts steuern unter anderem die einzusetzenden SQL-Konstrukte und den angestrebten Anspruch der Aufgaben. Bewertet werden die Ergebnisse anhand ihrer Sinnhaftigkeit, Neuartigkeit, thematischen Übereinstimmung mit dem Prompt, unmittelbaren Verwendbarkeit und Ausführbarkeit. Die Studie weist grundsätzlich brauchbare Ergebnisse nach, zeigt jedoch zugleich Grenzen der Steuerbarkeit: So führte eine auf `HAVING` ausgerichtete Formulierung nur in einem der fünf untersuchten Durchläufe zuverlässig zu einer entsprechenden Aufgabe, und als fortgeschritten angeforderte Aufgaben enthielten häufig keine hinreichend anspruchsvollen Konstrukte. Präzisere Prompts verbesserten die thematische Übereinstimmung, erhöhten jedoch teilweise die inhaltliche Wiederholung.

Beide Arbeiten belegen das Potenzial sprachmodellbasierter Aufgabengenerierung, machen aber auch deutlich, dass Ausführbarkeit allein kein hinreichendes Qualitätskriterium darstellt. Die vorliegende Arbeit verwendet deshalb bestehende Datenbankschemata und ergänzt die Generierung um eine mehrstufige Analyse, Auswahl, Verbesserung

und Validierung der entstehenden Aufgabenbestandteile.

2.4.2. Mehrfachgenerierung und Reparatur von SQL-Anfragen

Pourreza et al. stellen in *CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL* ein Text-to-SQL-Verfahren vor, das mehrere unterschiedliche Generierungswege kombiniert [18]. Ein Divide-and-Conquer-Ansatz zerlegt komplexe Aufgaben in Teilprobleme, ein weiterer Ansatz leitet die Anfrage anhand eines Ausführungsplans schrittweise her, und ein dritter Weg erzeugt eingabespezifische synthetische Beispiele für den Few-Shot-Kontext. Aus den so entstehenden SQL-Anfrage-Kandidaten wird durch paarweise Vergleiche ein Ergebnis ausgewählt. Hierfür verwenden die Autoren ein eigens trainiertes binäres Auswahlmodell. Auf dem BIRD-Testdatensatz erreicht das Gesamtsystem eine Ausführungsgenauigkeit von 73,0 %. Die Ergebnisse zeigen, dass voneinander abweichende Generierungswege komplementäre Lösungen hervorbringen können und eine nachgelagerte Auswahl daher gegenüber einer einzelnen Generierung vorteilhaft sein kann.

Die in dieser Arbeit verwendeten Promptvarianten folgen ebenfalls dem Grundgedanken, für dieselbe Eingabe mehrere SQL-Anfrage-Kandidaten zu erzeugen. Die Auswahl erfolgt jedoch nicht durch ein zusätzlich trainiertes Auswahlmodell, sondern anhand der Ergebnisse der automatischen SQL-Analyse. Dadurch werden die bereits für die Qualitätsprüfung erhobenen Informationen zugleich für die Kandidatenauswahl genutzt.

Pourreza und Rafiei adressieren in *DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction* insbesondere die Verarbeitung komplexer Text-to-SQL-Eingaben [19]. Das Verfahren gliedert die Generierung in die vier Module Schema-Linking, Klassifikation und Zerlegung der Aufgabe, SQL-Generierung sowie Selbstkorrektur. Abhängig von der ermittelten Komplexität wird eine Eingabe direkt bearbeitet oder in einfachere Teilprobleme zerlegt. Die erzeugte Anfrage wird danach in einem gesonderten Schritt auf typische Fehler überprüft und korrigiert. In den Experimenten verbessert diese modulare Verarbeitung die Ergebnisse verschiedener Sprachmodelle gegenüber einer einfachen Few-Shot-Generierung um etwa zehn Prozentpunkte. Auf dem Spider-Testdatensatz wird eine Ausführungsgenauigkeit von 85,3 % erreicht. Für

die vorliegende Arbeit ist insbesondere die Trennung von Erzeugung und nachgelagerter Verbesserung relevant. Anders als bei DIN-SQL basiert das Korrekturfeedback hier jedoch auf den Ergebnissen mehrerer externer Prüfungen der generierten Anfrage.

2.4.3. Synthese von Frage-SQL-Paaren

Die Erzeugung natürlichsprachlicher Beschreibungen aus SQL-Anfragen wurde bereits vor dem breiten Einsatz großer Sprachmodelle untersucht. Xu et al. stellen in *SQL-to-Text Generation with Graph-to-Sequence Model* ein Verfahren vor, das eine SQL-Anfrage in einen gerichteten Graphen überführt. Dessen Knoten und Beziehungen bilden die Struktur der Anfrage ab [12]. Ein Graph-to-Sequence-Modell kodiert diese Repräsentation und erzeugt daraus eine natürlichsprachliche Beschreibung. In den Experimenten auf WikiSQL und einem aus Stack Overflow gewonnenen Datensatz übertrifft das Verfahren sequenz- und baumbasierte Vergleichsmodelle. Der Ansatz verdeutlicht, dass eine explizite Repräsentation der SQL-Struktur die sprachliche Wiedergabe einer Anfrage unterstützen kann.

Guo et al. betrachten in *Question Generation from SQL Queries Improves Neural Semantic Parsing* die SQL-to-Question-Generierung als Verfahren zur Datenerweiterung für semantische Parser [20]. Aus vorhandenen SQL-Anfragen werden zusätzliche natürlichsprachliche Fragen erzeugt, die anschließend für das Training eines Text-to-SQL-Modells verwendet werden. Auf WikiSQL verbessert dieses Vorgehen die Ergebnisse sowohl bei reduziertem als auch bei vollständigem Umfang der manuell annotierten Trainingsdaten. Wu et al. erweitern diesen Gedanken in *Data Augmentation with Hierarchical SQL-to-Question Generation for Cross-domain Text-to-SQL Parsing* um eine hierarchische Generierung, welche die verschachtelte Struktur komplexerer SQL-Anfragen berücksichtigt [21]. Die erzeugten Frage-SQL-Paare verbessern Text-to-SQL-Modelle auf den Datensätzen WikiSQL, Spider und DuSQL. Diese Arbeiten verfolgen primär die Erweiterung von Trainingsdaten, zeigen aber zugleich, dass die Ableitung einer Frage aus einer vorgegebenen SQL-Anfrage eine eigenständige Generierungsaufgabe darstellt.

Li et al. stellen in *OmniSQL: Synthesizing High-quality Text-to-SQL Data at Scale* eine umfangreiche Pipeline zur Synthese von Text-to-SQL-Trainingsdaten vor [22]. Ausge-

hend von Webtabellen werden zunächst relationale Datenbanken erzeugt und erweitert. Danach generiert ein Sprachmodell SQL-Anfragen in vier vorgegebenen Komplexitätsstufen. Bei der Generierung erhält es neben dem Schema auch ausgewählte Datenbankwerte und Informationen zu verwendbaren SQL-Funktionen. Nicht ausführbare Anfragen werden entfernt und strukturell gleiche Varianten anhand ihrer SQL-Schablone reduziert. Anschließend werden aus den verbliebenen SQL-Anfragen natürlichsprachliche Fragen in neun unterschiedlichen Sprachstilen sowie schrittweise Herleitungen der Lösungen erzeugt.

Für jede SQL-Anfrage erzeugt OmniSQL [22] mehrere Aufgabentext-Kandidaten. Ein Sentence-Transformer bildet diese Kandidaten als Vektoren ab. Für jeden Kandidaten wird die mittlere Kosinusähnlichkeit zu den übrigen Formulierungen berechnet. Ausgewählt wird die Frage mit der höchsten mittleren Ähnlichkeit, da sie dem semantischen Zentrum der Kandidatenmenge am nächsten liegt. Aus dem Gesamtverfahren entstand SynSQL-2.5M mit 2 544 390 Frage-SQL-Paaren einschließlich Datenbank und Lösungserklärung aus 16 583 synthetischen Datenbanken. Zur Qualitätsprüfung bewertete GPT-4o jeweils 1 000 Beispiele aus SynSQL-2.5M und BIRD. Zusätzlich beurteilten drei fachkundige Personen eine Stichprobe von 1 000 SynSQL-2.5M-Beispielen. Die mit SynSQL-2.5M trainierten OmniSQL-Modelle verbesserten die zugrunde liegenden Basismodelle über mehrere Text-to-SQL-Datensätze hinweg.

Für die vorliegende Arbeit sind zwei Bestandteile dieses Ansatzes von besonderer Bedeutung. Die synthetische Erzeugung relationaler Datenbanken stellt eine mögliche Alternative zu bestehenden Datenbeständen wie Spider [3] dar. Unmittelbar übernommen wird dagegen der konzeptionelle Ansatz, zu einer SQL-Anfrage mehrere Aufgabentext-Kandidaten zu generieren und den semantisch konsistentesten Kandidaten auszuwählen. Die Zielsetzung unterscheidet sich dennoch: OmniSQL erzeugt umfangreiche Trainingsdaten für Text-to-SQL-Modelle, während die vorliegende Pipeline vollständige SQL-Übungsaufgaben erzeugt und die Übereinstimmung von Aufgabentext und Musterlösungs-Anfrage zusätzlich anhand ihrer Ausführungsergebnisse prüft.

Duan et al. verwenden in *DSQG-Syn: Synthesizing High-quality Data for Text-to-SQL Parsing by Domain Specific Question Generation* die umgekehrte Generierungsreihen-

folge [23]. Das Verfahren erzeugt anhand vorgegebener Fragetypen zunächst domänenspezifische natürlichsprachliche Fragen und leitet daraus passende SQL-Anfragen ab. Eine anschließende Filterung entfernt fehlerhafte oder inkonsistente Paare. Die so erzeugten Daten verbessern beim Fine-Tuning mehrerer quelloffener Sprachmodelle deren Text-to-SQL-Leistung gegenüber den untersuchten Vergleichsverfahren. Der Ansatz ermöglicht eine gezieltere Kontrolle von Domäne und Frageintention, verlagert die Absicherung der Zuordnung jedoch auf die nachgelagerte SQL-Generierung. Die vorliegende Arbeit beginnt demgegenüber mit einer geprüften Musterlösungs-Anfrage und erzeugt erst danach den zugehörigen Aufgabentext.

2.4.4. Knowledge-Graph-basierte Aufgabengenerierung

Christ et al. schlagen in *Generating SQL-Query-Items Using Knowledge Graphs* eine strukturierte Alternative zur ausschließlich sprachmodellbasierten Erzeugung vor [24]. Zunächst werden Tabellen, Attribute, Datentypen sowie Primär- und Fremdschlüssel eines Datenbankschemas in einen Knowledge Graph überführt. Lehrende ergänzen semantische Bezeichnungen für Entitäten und Beziehungen. Auf dieser Grundlage werden SQL-Anfragen anhand konfigurierbarer Vorgaben zu ausgewählten Spalten, Verknüpfungen, Bedingungen, Aggregationen, Gruppierung und Sortierung zusammengesetzt. Aus dem Syntaxbaum der Anfrage und den semantischen Annotationen entstehen anschließend zwei alternative natürlichsprachliche Aufgabenformulierungen: eine schablonenbasierte Variante und eine hybride Variante, bei der ein Sprachmodell einzelne Formulierungen ergänzt.

Die Evaluation von Christ et al. [24] umfasst 667 für eine angepasste IMDB-Datenbank erzeugte Aufgaben, die von jeweils drei Personen hinsichtlich Plausibilität, typischer Verwendung, Komplexität und sprachlicher Formulierung beurteilt wurden. Die Beurteilenden stimmen bei Plausibilität und typischer Verwendung weitgehend überein. Die eingeschätzte Komplexität und die Präferenz zwischen den beiden Textvarianten weisen hingegen eine größere Streuung auf. Da nur ein Datenbankschema untersucht wurde, bleibt die Übertragbarkeit auf andere Domänen eingeschränkt. Der Knowledge Graph ermöglicht eine fein steuerbare und nachvollziehbare Konstruktion von Aufgaben, erfordert aber eine vorgelagerte semantische Annotation des Schemas. Die vorliegende

Arbeit setzt stattdessen auf vorhandene relationale Schemata und Datenbankzustände und lässt Sprachmodelle SQL-Anfrage-Kandidaten erzeugen, deren Qualität anschließend automatisiert geprüft wird.

Die verwandten Arbeiten machen insgesamt zwei wiederkehrende Anforderungen sichtbar: Generierte Aufgabenbestandteile müssen gezielt variiert werden können und dürfen nicht ohne zusätzliche Prüfung übernommen werden. Die vorliegende Arbeit verbindet daher die Mehrfachgenerierung und Reparatur von SQL-Anfragen mit der SQL-to-Question-Generierung und einer nachgelagerten Validierung des vollständigen Paares aus Aufgabentext und Musterlösungs-Anfrage.

3. Generierung von SQL-Übungsaufgaben

In diesem Kapitel wird die konzeptionelle Vorgehensweise zur automatischen Generierung vollständiger SQL-Übungsaufgaben beschrieben. Aus einem gegebenen Datenbankschema und Datenbankzustand wird zunächst eine geprüfte SQL-Anfrage erzeugt und anschließend um einen passenden natürlichsprachlichen Aufgabentext ergänzt. Die Verarbeitung gliedert sich in die zwei in Abbildung 1 dargestellten Phasen.

Die Bewertung eines SQL-Anfrage-Kandidaten wird als Score ausgedrückt. Die maximal erreichbare Bewertung wird im Folgenden als *Full Score* bezeichnet. Sie wird genau dann erreicht, wenn der Kandidat sämtliche vorgesehenen Prüfschritte besteht. Die Zusammensetzung und Gewichtung des Scores wird im weiteren Verlauf dieses Kapitels erläutert.

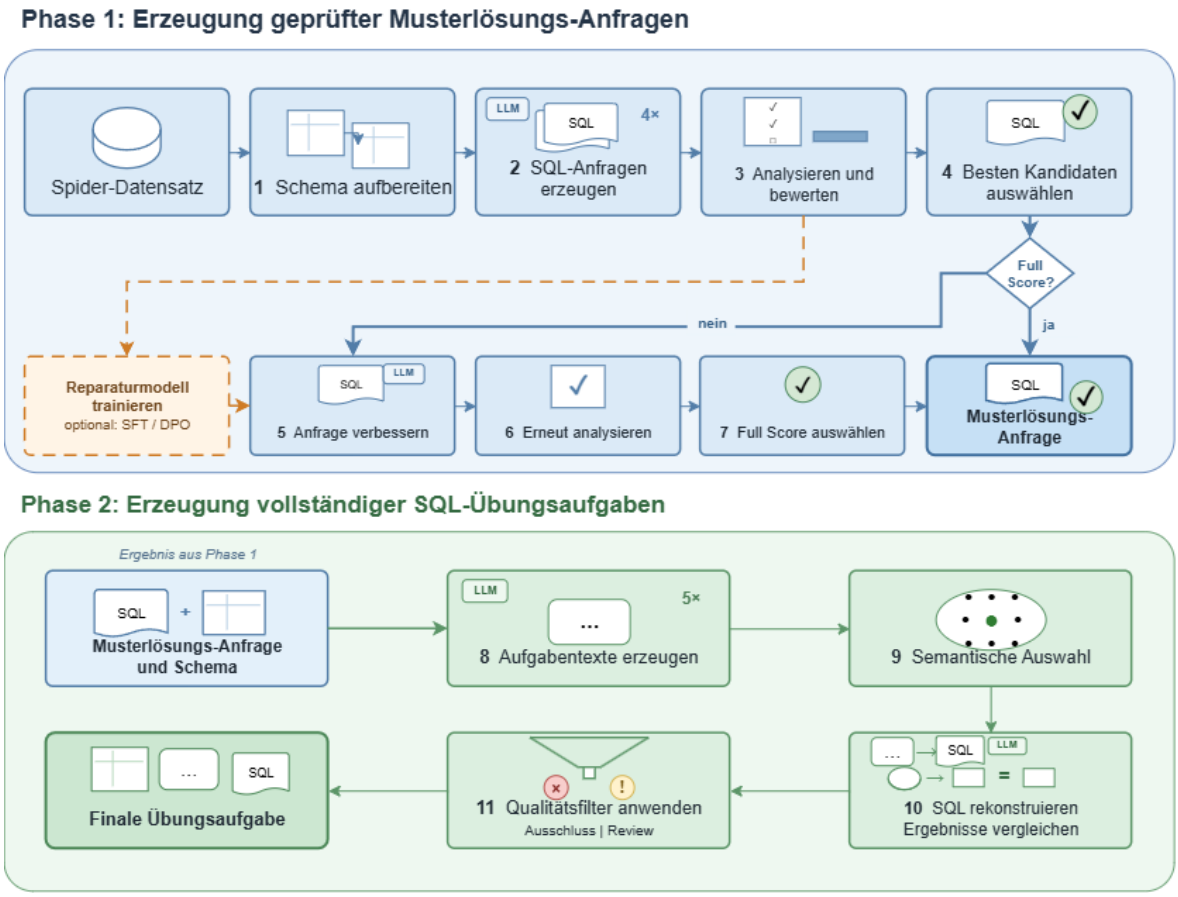


Abbildung 1: Übersicht der zweiphasigen Pipeline zur Erzeugung vollständiger SQL-Übungsaufgaben. Orange hervorgehoben ist das optionale Training eines Reparaturmodells. Die nummerierten Verarbeitungsschritte werden in den folgenden Abschnitten erläutert.

3.1. Grundidee der Pipeline

Für die konzeptionelle Beschreibung wird die Eingabe eines Pipeline-Durchlaufs mit x bezeichnet. Sie besteht aus dem konkreten Datenbankzustand D, dem zugehörigen Datenbankschema S und einer kategorialen Vorgabe z, die Art und Komplexität der zu erzeugenden SQL-Anfrage festlegt. Die Menge aller vorgesehenen Vorgaben wird mit Z bezeichnet. Somit gilt:

$$x = (D, S, z) \quad \text{mit} \quad z \in Z.$$

Das Ziel der Pipeline ist eine vollständige SQL-Übungsaufgabe a. Dabei bezeichnet n einen natürlichsprachlichen Aufgabentext und q eine in SQL formulierte Anfrage,

die als Musterlösungs-Anfrage dient. Sowohl die SQL-Anfrage als auch ihre inhaltliche Übereinstimmung mit dem Aufgabentext werden innerhalb der Pipeline geprüft. Zusammen mit dem Datenbankschema S ergibt sich eine vollständige Aufgabe als

$$a = (S, n, q).$$

Für einen einzelnen Durchlauf entstehen zunächst vier SQL-Anfrage-Kandidaten. Nach ihrer Analyse und Auswahl liegt mit q_{valid} eine geprüfte Musterlösungs-Anfrage vor, sofern ein Kandidat nach der initialen Generierung oder dem Verbesserungsschritt den Full Score erreicht. Zu dieser Anfrage werden mehrere Aufgabentexte erzeugt, aus denen n_{selected} ausgewählt wird. Die Zuordnung zwischen Schema, Musterlösungs-Anfrage und Aufgabentext bleibt während der weiteren Validierung erhalten.

Der Übergang von der Eingabe zur vollständigen Aufgabe lässt sich damit vereinfacht wie folgt darstellen:

$$x = (D, S, z) \xrightarrow{F_{\text{SQL}}} q_{\text{valid}} \xrightarrow{F_{\text{N}}} (q_{\text{valid}}, n_{\text{selected}}) \xrightarrow{V_{\text{final}}} a_{\text{final}} = (S, n_{\text{selected}}, q_{\text{valid}}).$$

Die Symbole F_{SQL} und F_{N} kennzeichnen die erzeugenden Verarbeitungsschritte, während V_{final} für die abschließende Validierung des entstandenen Paares steht.

Dabei umfasst F_{SQL} die Generierung, Analyse, Auswahl und gegebenenfalls Verbesserung der SQL-Anfragen. F_{N} bezeichnet die Erzeugung und semantische Auswahl der Aufgabentexte. V_{final} fasst die Rekonstruktion einer SQL-Anfrage aus dem Aufgabentext, den Vergleich der Ausführungsergebnisse und den zusätzlichen Qualitätsfilter zusammen.

Über alle erfolgreich abgeschlossenen Durchläufe hinweg bilden die Musterlösungs-Anfragen die Menge Q_{valid}, die ausgewählten Aufgabentexte die Menge N_{selected} und die finalen Aufgaben die Menge A_{final}.

Die erste Phase umfasst die Schritte 1 bis 7 in Abbildung 1. Sie führt von der Schemaaufbereitung über die Generierung und scorebasierte Auswahl mehrerer Kandidaten bis zu einer geprüften Musterlösungs-Anfrage. Kandidaten ohne Full Score können vor der erneuten Analyse verbessert werden. Das hierfür eingesetzte Reparaturmodell lässt sich optional durch SFT oder DPO anpassen.

Die Schritte 8 bis 11 bilden die zweite Phase. Ausgehend von der Musterlösungs-Anfrage werden mehrere Aufgabentexte erzeugt und semantisch ausgewählt. Die anschließende Rekonstruktion einer SQL-Anfrage aus dem ausgewählten Aufgabentext folgt der Text-to-SQL-Richtung, während die Erzeugung des Aufgabentextes zuvor der SQL-to-Question-Richtung entspricht. Der Vergleich der Ausführungsergebnisse und ein zusätzlicher Qualitätsfilter bestimmen, welche Paare in den finalen Aufgabenpool eingehen oder für eine fachliche Kontrolle markiert werden.

3.2. Phase 1: Erzeugung geprüfter Musterlösungs-Anfragen

Die folgenden Abschnitte konkretisieren die in Abbildung 1 dargestellten Schritte zur Erzeugung, Bewertung und Auswahl der Musterlösungs-Anfragen.

3.2.1. Schema-Aufbereitung

Als Datengrundlage werden die 140 Datenbankschemata und Datenbankzustände des Trainingssplits von Spider 1 verwendet [3]. Die im Datensatz enthaltenen natürlichsprachlichen Fragen und zugehörigen Referenzanfragen werden nicht übernommen, da die Pipeline neue SQL-Anfragen und Aufgabentexte erzeugt. Die Schemata stellen den strukturellen Kontext bereit. Die konkreten Datenbankzustände ermöglichen die spätere Ausführung und Validierung der generierten Anfragen.

Spider 1 wurde gewählt, weil die Datenbanken aus unterschiedlichen Domänen in einer einheitlich verarbeitbaren relationalen Struktur vorliegen und lokale Datenbankzustände verfügbar sind. Spider 2.0 und BIRD stellen mögliche alternative Datengrundlagen dar [25, 26]. Sie bieten realitätsnähere Aufgaben und zusätzliche Kontextinformationen, würden jedoch eine Erweiterung der Pipeline um die Verarbeitung weiterer Dokumente, Domäneninformationen und SQL-Dialekte erfordern.

Für die initiale SQL-Generierung werden die ausgewählten Schemata, die zugehörigen Datenbankzustände und daraus entnommene Beispielwerte in eine einheitliche Darstellung überführt. Diese aufbereitete Darstellung bildet die Eingabe des folgenden Generierungsschritts.

3.2.2. Initiale SQL-Generierung

Die initiale SQL-Generierung wird durch die gewünschte SQL-Schwierigkeitskategorie gesteuert. Diese legt fest, welche SQL-Konstrukte die zu erzeugende Anfrage enthalten soll.

SQL-Schwierigkeitskategorien

Die Einteilung der SQL-Schwierigkeitskategorien wurde auf Grundlage der Vorlesungsfolien von Prof. Dr. Stefan Brass zur Veranstaltung Datenbank-Programmierung vorgenommen [27]. Sie dient dazu, die generierten SQL-Anfragen nicht nur insgesamt, sondern auch getrennt nach zentralen SQL-Konstrukten auswerten zu können. Dass eine solche Differenzierung sinnvoll ist, zeigen auch Analysen studentischer SQL-Lösungen. Poulsen et al. zeigen, dass Studierende insbesondere bei Anfragen mit `GROUP BY` und korrelierten Unteranfragen Schwierigkeiten haben [28]. Die gewählten SQL-Schwierigkeitskategorien decken daher unterschiedliche Anfrageformen ab, die für die spätere Bewertung der Pipeline getrennt betrachtet werden können.

Die Bezeichnungen *diff1* bis *diff8* kennzeichnen unterschiedliche strukturelle Kategorien und stellen keine lineare Rangfolge zunehmender Schwierigkeit dar. Die jeweilige SQL-Schwierigkeitskategorie bezieht sich auf die geforderte Anfrageform und die darin enthaltenen SQL-Konstrukte.

Die acht SQL-Schwierigkeitskategorien sind wie folgt definiert:

- **diff1:** einfache SQL-Anfrage über eine Tabelle ohne Join und ohne Aggregation, typischerweise mit `WHERE`.
- **diff2:** SQL-Anfrage mit `JOIN`, aber ohne Aggregation.
- **diff3:** nicht-monotone Anfrage mit `NOT EXISTS` oder `NOT IN`, aber ohne Aggregation.
- **diff4:** Anfrage mit Aggregatfunktion ohne `GROUP BY`.
- **diff5:** Anfrage mit Aggregatfunktion und `GROUP BY`.
- **diff6:** Anfrage mit Aggregatfunktion, `GROUP BY` und `HAVING`.

- **diff7:** Anfrage mit `UNION` oder `LEFT JOIN`.
- **diff8:** Anfrage mit `LEFT JOIN` und Aggregation, insbesondere mit Gruppierung und Zählung auch fehlender Zuordnungen.

Für die Generierung erhält das Sprachmodell das aufbereitete Datenbankschema, die Beispielwerte aus dem Datenbankzustand und die Beschreibung der gewählten SQL-Schwierigkeitskategorie. Für jede Kombination aus Datenbank und SQL-Schwierigkeitskategorie wird ein Durchlauf mit vier Promptvarianten gebildet. Jede Variante fordert das Modell zur Erzeugung genau eines SQL-Anfrage-Kandidaten auf, sodass pro Durchlauf vier Kandidaten entstehen. Die Varianten verwenden dieselbe Datenbasis und unterscheiden sich ausschließlich in der Art der Anleitung:

- Der **direkte Schema-Prompt** fordert die SQL-Anfrage unmittelbar aus dem Schema und der SQL-Schwierigkeitskategorie an.
- Der **Divide-and-Conquer-Prompt** zerlegt die Aufgabe intern in die Analyse relevanter Schemaelemente, die Planung der benötigten SQL-Konstrukte und die abschließende Anfrageerzeugung.
- Der **mehrstufige Planungs-Prompt** führt das Modell schrittweise über die Auswahl relevanter Tabellen und Verknüpfungen, die Festlegung von Filtern und Aggregationen sowie den Aufbau der SQL-Klauseln zur finalen Anfrage.
- Der **Few-Shot-Prompt** ergänzt die Eingabe um drei Beispiel-Anfragen derselben SQL-Schwierigkeitskategorie, die als strukturelle Orientierung für die neue Anfrage dienen.

Das Ergebnis dieses Schrittes ist noch keine unmittelbar verwendbare Musterlösungs-Anfrage, sondern eine Gruppe aus vier alternativen SQL-Anfrage-Kandidaten pro Durchlauf. Diese Kandidaten werden im nächsten Schritt einheitlich analysiert und miteinander verglichen.

Die Verwendung mehrerer Generierungsvarianten lässt sich konzeptionell mit dem Mehrkandidatenansatz von CHASE-SQL [18] in Beziehung setzen. CHASE-SQL zeigt, dass die Erzeugung und anschließende Auswahl mehrerer SQL-Anfrage-Kandidaten robuster sein kann als die unmittelbare Übernahme einer einzelnen Modellausgabe. In

dieser Arbeit wird dieses Prinzip für die Generierung von Musterlösungs-Anfragen zu vorgegebenen Datenbankschemata und SQL-Schwierigkeitskategorien eingesetzt.

Die Auswahl des geeignetsten Kandidaten erfolgt jedoch auf einer anderen Grundlage. Während CHASE-SQL [18] für die Kandidatenauswahl einen zusätzlich trainierten Selektor einsetzt, werden in dieser Arbeit die Ergebnisse der nachfolgenden SQL-Analyse verwendet. Da für jeden Kandidaten dieselben Prüfschritte durchgeführt und in einer Bewertung zusammengeführt werden, können die vier Varianten anhand ihrer Analyseergebnisse miteinander verglichen werden. Für die weitere Verarbeitung wird der am besten bewertete SQL-Anfrage-Kandidat eines Laufs ausgewählt.

Auch DIN-SQL zeigt, dass die Zerlegung komplexer Text-to-SQL-Aufgaben in mehrere Teilprobleme und nachgelagerte Korrekturschritte die Ergebnisqualität verbessern kann [19]. Für diese Arbeit ist daran vor allem die Grundidee anschlussfähig, SQL-Generierung nicht als einmalige Modellausgabe zu behandeln, sondern durch weitere Analyse- und Korrekturschritte abzusichern.

3.2.3. Analyse und Bewertung

Aus der initialen Generierung liegen für jede Eingabe vier alternative SQL-Anfrage-Kandidaten vor. Bevor einer von ihnen ausgewählt werden kann, müssen sie nach einheitlichen Kriterien untersucht werden. Die Analyse verbindet hierzu statische Prüfungen der Anfrage mit Prüfungen gegen das Schema und den konkreten Datenbankzustand. Auf diese Weise wird nicht nur festgestellt, ob eine Anfrage formal verarbeitet werden kann, sondern auch, ob sie ausführbar ist, ein beobachtbares Ergebnis liefert, die Beziehungen des Schemas plausibel verwendet und der geforderten SQL-Schwierigkeitskategorie entspricht.

Den ersten Prüfschritt bildet das *Parsing*. Dabei wird die Anfrage anhand der Grammatik des verwendeten SQL-Dialekts in eine strukturierte Darstellung überführt. Schlägt dieser Schritt fehl, ist die Anfrage bereits syntaktisch ungültig und kann nicht zuverlässig weiter untersucht werden. Ein erfolgreiches Parsing bestätigt jedoch nur den syntaktisch korrekten Aufbau. Es sagt noch nicht aus, ob die referenzierten Tabellen und Spalten im jeweiligen Schema vorhanden sind.

Daran schließt sich die *Validierung gegen das Datenbankschema* an. Sie prüft unter anderem, ob Tabellen- und Spaltenreferenzen auflösbar sind, ob Aliase eindeutig verwendet werden und ob die eingesetzten Operationen mit den ermittelten Datentypen vereinbar sind. Parsing und Validierung bilden damit die formale Grundlage der weiteren Analyse: Der erste Schritt untersucht die grammatische Struktur, während der zweite diese Struktur in den Kontext des konkreten Schemas einordnet.

Eine formal gültige Anfrage wird anschließend auf dem zur Eingabe gehörenden Datenbankzustand ausgeführt. Diese Prüfung erfasst Fehler, die erst zur Laufzeit oder bei der Verarbeitung durch das verwendete Datenbanksystem sichtbar werden. Eine erfolgreiche Ausführung ist dabei noch kein Beleg für die fachliche Richtigkeit der Anfrage. Sie zeigt zunächst nur, dass die Anfrage im vorgesehenen SQL-Dialekt auf der betreffenden Datenbank verarbeitet werden kann.

Zusätzlich wird das Ausführungsergebnis erfasst. Neben der Anzahl und den Bezeichnungen der Ergebnisspalten werden eine begrenzte Vorschau der Ergebniszeilen und die gesamte Zeilenanzahl gespeichert. Dadurch kann geprüft werden, ob die Anfrage auf dem gegebenen Datenbankzustand überhaupt ein nicht leeres Ergebnis erzeugt. Eine ausführbare Anfrage mit leerem Ergebnis ist für die weitere Generierung einer Übungsaufgabe nur unzureichend überprüfbar, da weder die inhaltliche Wirkung der Anfrage noch die Form des erwarteten Resultats anhand konkreter Ausgabewerte beurteilt werden kann.

Die nächste Prüfung betrachtet die Verwendung der im Schema definierten Fremdschlüsselbeziehungen. Dazu werden die in den Join- und Vergleichsbedingungen eingesetzten Spaltenbeziehungen ermittelt und den bekannten Fremdschlüsseln des Schemas gegenübergestellt. Auf diese Weise soll festgestellt werden, ob das Sprachmodell die für die Anfrage benötigten Tabellen nicht nur ausgewählt, sondern auch über die vorgesehenen Beziehungen miteinander verbunden hat. Verknüpfungen zwischen Spalten, für die im Schema keine entsprechende Beziehung hinterlegt ist, werden als auffällig bewertet. Die Prüfung muss dabei auch Tabellenaliase, verschachtelte Anfragen und unterschiedliche Join-Schreibweisen auf ihre zugrunde liegenden Tabellen und Spalten zurückführen.

Nachdem Syntax, Schemabezug und Ausführbarkeit geprüft wurden, folgt eine weiter-

gehende semantische Untersuchung. Eine Anfrage kann alle vorherigen Prüfungen bestehen und dennoch Konstruktionen enthalten, die mit hoher Wahrscheinlichkeit nicht der beabsichtigten Bedeutung entsprechen. Dazu zählen beispielsweise widersprüchliche Bedingungen oder ein Vergleich mit `NULL` über gewöhnliche Vergleichsoperatoren. Solche Fehlerklassen werden von Brass und Goldberg als semantische SQL-Fehler beschrieben [13]. In dieser Arbeit wird für ihre Erkennung SQLlint eingesetzt [14]. Das Werkzeug untersucht die Anfrage anhand definierter Regeln auf semantisch auffällige SQL-Muster und liefert für jeden Fund einen Fehlercode mit einer zugehörigen Beschreibung. Ein vergleichbarer Ansatz wäre die Erkennung und Bewertung von SQL-Anti-Patterns, wie sie beispielsweise SQLCheck durch die Verbindung von Anfrage- und Datenanalyse umsetzt [15].

Abschließend wird die strukturelle Form der Anfrage mit der vorgegebenen SQL-Schwierigkeitskategorie verglichen. Hierzu wird ermittelt, welche der zuvor definierten Konstrukte in der Anfrage tatsächlich vorkommen. Der Kandidat besteht diese Prüfung nur, wenn die erkannte Struktur der für die Eingabe geforderten SQL-Schwierigkeitskategorie entspricht. Damit wird verhindert, dass eine formal und semantisch unauffällige Anfrage, deren strukturelle Form nicht der vorgegebenen Kategorie entspricht, als geeigneter Kandidat für diese Kategorie bewertet wird.

Die Bewertung eines SQL-Anfrage-Kandidaten wird als numerischer Score ausgedrückt. Dazu ist jedem Prüfschritt eine feste Punktzahl zugeordnet, deren Höhe seine Bedeutung für die Gesamtbewertung widerspiegelt. Ein bestandener Test erhöht den Score um die zugehörige Punktzahl, während ein nicht bestandener Test keine Punkte beiträgt. Der Gesamtscore eines Kandidaten ergibt sich aus der Summe der Punkte aller bestandenen Prüfschritte. Die maximale Punktzahl kann folglich nur erreicht werden, wenn sämtliche Prüfungen erfolgreich abgeschlossen wurden. Durch die Gewichtung wirkt sich ein grundlegender Fehler, etwa eine nicht ausführbare Anfrage, stärker auf den Score aus als ein leeres Ergebnis auf dem betrachteten Datenbankzustand. Die maximal erreichbare Punktzahl entspricht damit dem zuvor eingeführten Full Score.

Zusätzlich zum Score werden die Resultate der einzelnen Prüfungen und die zugehörigen Fehlerhinweise gespeichert. Der Score stellt damit eine einheitliche Kennzahl für den Vergleich und die anschließende Auswahl der vier Kandidaten bereit. Die detail-

lierten Analyseinformationen dienen darüber hinaus als Rückmeldung für die spätere Verbesserung fehlerhafter Anfragen.

3.2.4. Auswahl des besten SQL-Anfrage-Kandidaten

Ein Durchlauf bezeichnet die Generierung für eine bestimmte Datenbank und eine bestimmte SQL-Schwierigkeitskategorie. Innerhalb dieses Durchlaufs werden vier Promptvarianten verwendet. Jede Promptvariante erzeugt einen eigenen SQL-Anfrage-Kandidaten. Nach der Analyse liegen somit vier bewertete Kandidaten vor, die unter denselben Ausgangsbedingungen erzeugt wurden. Die Auswahl erfolgt innerhalb dieses Durchlaufs: Übernommen wird der Kandidat, der die beste Bewertung erreicht.

Grundlage der Auswahl ist der zuvor berechnete gewichtete Score. Für jeden Durchlauf wird zunächst der SQL-Anfrage-Kandidat mit dem höchsten Score bestimmt. Entscheidend ist somit nicht allein die Anzahl der bestandenen Prüfschritte, sondern auch deren jeweilige Gewichtung. Auf diese Weise wirken sich grundlegende Anforderungen, insbesondere die syntaktische Verarbeitbarkeit, die Übereinstimmung mit dem Schema und die Ausführbarkeit, stärker auf die Auswahl aus als nachgelagerte Qualitätsmerkmale.

Erreichen mehrere Kandidaten denselben Höchstscore, wird der in der festgelegten Eingabereihenfolge zuerst auftretende Kandidat übernommen. Damit bleibt die Auswahl auch bei einem Gleichstand deterministisch.

Für den ausgewählten SQL-Anfrage-Kandidaten bleibt festgehalten, durch welche Promptvariante er erzeugt wurde. Diese Herkunftsinformation ermöglicht es, später zu untersuchen, welche Promptstrategie besonders häufig den höchsten Score erzielt. Die nicht ausgewählten Kandidaten werden im regulären Generierungsablauf nicht weiterverwendet. Ihre SQL-Anfragen und Analyseergebnisse bleiben jedoch erhalten, da sie für die optionale Erzeugung von Trainingsdaten genutzt werden können.

3.2.5. Optionale Modellanpassung

Für die optionale Modellanpassung stehen die analysierten SQL-Anfrage-Kandidaten einschließlich ihrer Scores, bestandenen Prüfschritte und Fehlerhinweise zur Verfügung.

Dadurch können sowohl erfolgreiche als auch fehlerhafte Kandidaten für die Erzeugung der Trainingsdaten genutzt werden. Ziel dieses Schrittes ist es, aus den Analyseergebnissen Trainingsdaten abzuleiten und damit ein Modell zu trainieren, das typische Fehler der initialen Generierung besser vermeidet.

Die Trainingsdaten enthalten nicht nur die SQL-Anfragen selbst, sondern auch die dazugehörigen Bewertungen und Fehlerhinweise. Dadurch kann zwischen besseren und schlechteren Kandidaten unterschieden werden. Fehlerhafte Anfragen werden also nicht unmittelbar verworfen, sondern als Lernmaterial genutzt. Gerade diese Fälle sind wichtig, weil sie zeigen, welche Fehler bei der automatischen SQL-Generierung häufig auftreten und welche Art von Korrektur erforderlich ist.

Für die Modellverbesserung werden zwei Trainingsformen betrachtet. Beim Supervised Fine-Tuning erhält das Modell Beispiele, bei denen eine Eingabe einer gewünschten Ausgabe zugeordnet ist. In diesem Fall besteht die Eingabe aus dem Schema, der fehlerhaften oder unvollständigen SQL-Anfrage und den Analyseinformationen. Die gewünschte Ausgabe ist eine bessere SQL-Anfrage. Das Modell lernt dadurch direkt, wie eine Anfrage auf Basis des Feedbacks korrigiert werden soll.

Beim Preference-Training wird das Modell nicht nur mit einer einzelnen Zielausgabe trainiert. Stattdessen werden zwei Varianten miteinander verglichen: eine bessere und eine schlechtere SQL-Anfrage. Das Modell soll lernen, die bessere Variante zu bevorzugen. Dieser Ansatz passt besonders gut zu den erzeugten Analyseergebnissen, weil aus den Scores und Fehlerlisten nachvollziehbar abgeleitet werden kann, welcher Kandidat qualitativ besser ist.

Die optionale Modellanpassung stellt damit das Reparaturmodell bereit, das im folgenden Verbesserungsschritt eingesetzt wird.

3.2.6. Verbesserung der SQL-Anfragen

Aus der vorherigen Kandidatenauswahl liegt für jeden Durchlauf die am besten bewertete SQL-Anfrage vor. Die Auswahl des bestbewerteten Kandidaten bedeutet jedoch nicht, dass dieser bereits den Full Score erreicht hat. Ausgewählte Kandidaten mit Full

Score können unmittelbar weiterverwendet werden. Die übrigen ausgewählten Kandidaten werden dem Verbesserungsschritt zugeführt.

Für diesen Schritt wird aus den bereits vorhandenen Informationen eine gezielte Reparaturaufgabe gebildet. Das Sprachmodell erhält erneut das Datenbankschema und die Beispielwerte aus dem zugehörigen Datenbankzustand sowie die Beschreibung des geforderten Schwierigkeitsgrads. Ergänzt wird diese Eingabe um die ausgewählte SQL-Anfrage und die aus den nicht bestandenen Prüfschritten abgeleiteten Korrekturhinweise.

Das Modell wird angewiesen, die festgestellten Fehler unter Berücksichtigung des Schemas und der geforderten SQL-Schwierigkeitskategorie zu beheben. Im Unterschied zur initialen Generierung steht damit nicht die Erzeugung einer unabhängigen Anfrage, sondern die gezielte Überarbeitung des ausgewählten Kandidaten im Mittelpunkt.

Die Reparatur kann grundsätzlich von einem geeigneten Sprachmodell durchgeführt werden. Wird die optionale Modellanpassung eingesetzt, übernimmt diese Aufgabe das zuvor mit den aus der Analyse abgeleiteten SFT- oder DPO-Daten trainierte Reparaturmodell. In dieser Arbeit werden die im optionalen Schritt trainierten Modelle für die Verbesserung verwendet. Die überarbeitete SQL-Anfrage gilt nach der Modellausgabe noch nicht als korrekt, sondern wird im folgenden Schritt erneut der vollständigen Analyse unterzogen.

3.2.7. Erneute Analyse der verbesserten SQL-Anfragen

Die durch das Reparaturmodell überarbeiteten SQL-Anfragen werden nicht unmittelbar als korrekt übernommen. Da die Korrektur unvollständig sein oder neue Fehler hervorbringen kann, werden ausschließlich diese verbesserten Anfragen erneut nach denselben Kriterien untersucht wie die initial erzeugten Kandidaten.

Jede verbesserte SQL-Anfrage wird erneut geparst und gegen das zugehörige Datenbankschema validiert. Anschließend wird ihre Ausführbarkeit auf dem entsprechenden Datenbankzustand geprüft und kontrolliert, ob ein nicht leeres Ergebnis mit einer verwertbaren Ergebnisvorschau entsteht. Darauf folgen die Überprüfung der verwendeten Fremdschlüsselbeziehungen, die semantische Analyse durch SQLlint und

der Vergleich der tatsächlich enthaltenen SQL-Konstrukte mit der geforderten SQL-Schwierigkeitskategorie. Durch die Verwendung derselben Prüfkriterien bleiben die Bewertungen der initialen und der verbesserten Anfragen miteinander vergleichbar.

Aus den Ergebnissen dieser erneuten Prüfung wird für jede verbesserte SQL-Anfrage ein neuer Score berechnet. Frühere Punktwerte werden dabei nicht übernommen. Ausschlaggebend ist ausschließlich die überarbeitete Fassung. Jeder bestandene Prüfschritt trägt entsprechend seiner Gewichtung zum Score bei, sodass die maximale Punktzahl nur erreicht wird, wenn sämtliche Prüfungen bestanden wurden. Anschließend werden sämtliche neu bewerteten Reparaturergebnisse unabhängig von ihrem erreichten Score mit den ausgewählten Kandidaten zusammengeführt, die bereits bei der initialen Analyse den Full Score erreicht haben.

3.2.8. Auswahl geprüfter Musterlösungs-Anfragen

Nach der Zusammenführung umfasst die Datenmenge sowohl die bereits bei der initialen Analyse ausgewählten Kandidaten mit Full Score als auch sämtliche verbesserten und erneut bewerteten SQL-Anfragen. Erst in diesem Schritt wird entschieden, welche Einträge für die nachgelagerte Erzeugung von Aufgabentexten verwendet werden dürfen.

Die zusammengeführte Menge bewerteter Ergebnisse wird als B_1 bezeichnet. Ein Element dieser Menge wird als Tupel (q, b) dargestellt. Dabei bezeichnet q die SQL-Anfrage und b das zugehörige Bewertungsergebnis, das den berechneten Score score_b sowie die Resultate der einzelnen Prüfschritte enthält. Die maximal erreichbare Punktzahl wird als $\text{score}_{\max}$ bezeichnet.

Weiterverwendet werden ausschließlich Anfragen, die alle zuvor beschriebenen Prüfungen bestanden und damit diese maximale Punktzahl erreicht haben. Die Menge der mit Full Score bewerteten Musterlösungs-Anfragen wird als Q_{valid} bezeichnet und ergibt sich damit als

$$Q_{\text{valid}} = \{q \mid (q, b) \in B_1 \wedge \text{score}_b = \text{score}_{\max}\}\,.$$

Diese Menge bildet die Grundlage für die nachgelagerte Erzeugung natürlichsprachlicher Aufgabentexte. Dadurch wird das Risiko reduziert, dass eine sprachlich korrekt

wirkende Aufgabe auf einer bereits fehlerhaften oder nur teilweise geprüften SQL-Anfrage basiert.

3.3. Phase 2: Erzeugung vollständiger SQL-Übungsaufgaben

Aufbauend auf den geprüften Musterlösungs-Anfragen konkretisieren die folgenden Abschnitte die Erzeugung, Auswahl und Validierung der zugehörigen Aufgabentexte. Die Verarbeitung wechselt dabei von der SQL-to-Question-Richtung zur Text-to-SQL-Rekonstruktion und prüft anschließend die Konsistenz des entstehenden Paars aus Musterlösungs-Anfrage und Aufgabentext.

3.3.1. Erzeugung natürlichsprachlicher Aufgabentexte

Aus dem vorherigen Schritt liegen mit Full Score bewertete Musterlösungs-Anfragen vor. Diese SQL-Anfragen sind bereits als Musterlösungs-Anfragen geeignet, bilden aber noch keine vollständigen Übungsaufgaben, da der natürlichsprachliche Aufgabentext fehlt. Ziel dieses Schrittes ist es daher, zu jeder geprüften SQL-Anfrage mehrere mögliche Aufgabenformulierungen zu erzeugen.

Dazu wird ein Sprachmodell verwendet. Die Eingabe für das Modell besteht aus dem Datenbankschema und der jeweiligen SQL-Anfrage. Auf dieser Grundlage erzeugt das Modell natürlichsprachliche Aufgabentext-Kandidaten, die dieselbe fachliche Bedeutung wie die SQL-Anfrage ausdrücken sollen. Für jede geprüfte Musterlösungs-Anfrage werden insgesamt fünf solcher Kandidaten erzeugt.

Dieses Vorgehen greift die in OmniSQL [22] verwendete Idee auf, zu einer gegebenen SQL-Anfrage mehrere natürlichsprachliche Formulierungen zu erzeugen. In dieser Arbeit wird dieser Ansatz genutzt, um aus geprüften Musterlösungs-Anfragen mögliche Aufgabentexte für SQL-Übungsaufgaben abzuleiten.

3.3.2. Semantische Auswahl des Aufgabentextes

Aus dem vorherigen Schritt liegen zu jeder geprüften Musterlösungs-Anfrage mehrere natürlichsprachliche Aufgabentext-Kandidaten vor. Aus diesen Kandidaten wird zu-

nächst ein Aufgabentext ausgewählt, dessen Übereinstimmung mit der ursprünglichen Musterlösungs-Anfrage anschließend gesondert geprüft wird.

Die Auswahl orientiert sich am Semantic Consistency Selector aus OmniSQL [22]. Die zugrunde liegende Annahme ist, dass ein geeigneter Kandidat den gemeinsamen semantischen Kern der Kandidatenmenge möglichst gut repräsentiert. Dazu werden die Aufgabentext-Kandidaten durch ein Embedding-Modell in numerische Satzvektoren fester Länge überführt. Die Verwendung solcher Satzvektoren ist an Sentence-BERT angelehnt. Reimers und Gurevych zeigen, dass sich Satz-Embeddings für semantische Ähnlichkeitsvergleiche eignen und effizient über die Kosinusähnlichkeit vergleichen lassen [7].

Die Menge der zu einer SQL-Anfrage q erzeugten Aufgabentext-Kandidaten wird als N_q bezeichnet. Ein Aufgabentext-Kandidat aus dieser Menge wird mit n_i, ein weiterer mit n_j bezeichnet. Die Funktion $\mathrm{sim}(n_i, n_j)$ gibt die Kosinusähnlichkeit ihrer Satzvektoren an. Der Ausdruck $|N_q| - 1$ bezeichnet die Anzahl der übrigen Kandidaten, mit denen n_i verglichen wird. Für jeden Kandidaten wird die durchschnittliche Ähnlichkeit zu allen anderen Kandidaten berechnet. Der Kandidat mit dem höchsten Durchschnittswert wird als n_q^* ausgewählt:

$$n_q^* = \arg\max_{n_i \in N_q} \frac{1}{|N_q| - 1} \sum_{j \neq i} \mathrm{sim}(n_i, n_j).$$

Die aus OmniSQL übernommene Auswahl [22] entspricht konzeptionell der Bestimmung eines Medoids bezüglich der Kosinusdistanz. Wird die Distanz als $d_{\cos}(n_i, n_j) = 1 - \mathrm{sim}(n_i, n_j)$ aufgefasst, besitzt der gewählte Kandidat die kleinste durchschnittliche Distanz zu den übrigen Kandidaten. Im Unterschied zu einem Zentroid wird dabei kein neuer mittlerer Vektor gebildet. Ausgewählt wird stets einer der tatsächlich erzeugten Aufgabentexte. Semantische Ausreißer weisen im Regelfall eine geringere durchschnittliche Ähnlichkeit auf und werden dadurch seltener ausgewählt.

Bei nur einem vorhandenen Kandidaten wird dieser direkt übernommen. Bei zwei Kandidaten weist das Ähnlichkeitskriterium beiden Formulierungen denselben Wert zu und ermöglicht daher keine begründete Präferenz. In diesem Fall wird einer der beiden Kandidaten ohne semantische Bevorzugung ausgewählt. Ab drei Kandidaten wird die beschriebene semantische Auswahl angewendet.

3.3.3. Erzeugung einer rekonstruierten SQL-Anfrage

Die semantische Auswahl bestimmt den Aufgabentext-Kandidaten, der die Kandidatenmenge am besten repräsentiert. Sie gewährleistet jedoch nicht, dass der ausgewählte Aufgabentext vollständig mit der Musterlösungs-Anfrage übereinstimmt, da mehrere Kandidaten dieselbe Auslassung oder Fehlinterpretation enthalten können.

Zur unabhängigen Prüfung erhält ein weiteres Text-to-SQL-Sprachmodell ausschließlich den ausgewählten Aufgabentext und das zugehörige Datenbankschema. Die ursprüngliche SQL-Anfrage wird nicht übergeben. Aus diesen Informationen erzeugt das Modell eine SQL-Anfrage, die seine Interpretation des Aufgabentextes ausdrückt. Diese rekonstruierte Anfrage wird nicht als Musterlösungs-Anfrage gespeichert, sondern im folgenden Schritt ausschließlich mit der ursprünglichen Anfrage verglichen.

3.3.4. Vergleich der Ausführungsergebnisse

Die ursprüngliche Musterlösungs-Anfrage und die rekonstruierte SQL-Anfrage werden auf demselben SQLite-Datenbankzustand ausgeführt. Verglichen werden die vollständigen Ergebniszeilen und nicht nur eine Vorschau. Enthält eine der Anfragen eine explizite Sortierung, muss auch die Reihenfolge der Zeilen übereinstimmen. Andernfalls erfolgt der Vergleich unabhängig von der Reihenfolge als Multimenge, sodass mehrfach vorkommende Zeilen weiterhin berücksichtigt werden. Abweichende Aliasnamen der Ausgabespalten beeinflussen den Vergleich nicht.

Stimmen die Ausführungsergebnisse überein, wird angenommen, dass der ausgewählte Aufgabentext und die Musterlösungs-Anfrage dieselbe Bedeutung ausdrücken. Bei abweichenden Ergebnissen oder einem Ausführungsfehler wird der Kandidat nicht weiterverwendet. Die Prüfung belegt keine allgemeine Äquivalenz der beiden SQL-Anfragen, liefert jedoch einen automatischen semantischen Vergleich auf dem vorliegenden Datenbankzustand.

3.3.5. Zusätzlicher Qualitätsfilter

Aus dem vorherigen Schritt liegen Aufgabentext-Kandidaten vor, die die Rekonstruktionsprüfung bestanden haben. Diese Prüfung zeigt jedoch lediglich, dass die ursprüngliche Musterlösungs-Anfrage und die aus dem Aufgabentext rekonstruierte SQL-Anfrage auf dem verwendeten Datenbankzustand dasselbe Ergebnis liefern. Sie ersetzt keine vollständige inhaltliche Sichtung der erzeugten Aufgaben.

Da die Menge der Kandidaten, die die Rekonstruktionsprüfung bestanden haben, mehrere tausend Einträge umfasst, war eine vollständige manuelle Sichtung nicht praktikabel. Der Datenbestand wurde daher einem Sprachmodell mit der offenen Anweisung vorgelegt, Aufgabentexte zu identifizieren, die nicht zur zugehörigen SQL-Anfrage passen, sprachliche oder inhaltliche Fehler enthalten oder anderweitig auffällig erscheinen. Diese Sichtung folgte keinem formal definierten Bewertungsverfahren und die Einschätzungen des Modells wurden nicht unmittelbar als Ausschlussentscheidungen übernommen. Sie dienten ausschließlich dazu, mögliche Problemfälle und wiederkehrende Fehlermuster im Datenbestand aufzufinden.

Die identifizierten Auffälligkeiten werden anschließend in konkrete, automatisiert prüfbare Regeln überführt. Der Qualitätsfilter unterscheidet dabei zwischen harten Ausschlussregeln und Review-Regeln. Harte Ausschlussregeln erfassen Merkmale, die eine Aufgabe unmittelbar ungeeignet machen, beispielsweise Selbstreferenzen auf die Musterlösungs-Anfrage, systematische Fehlübersetzungen von Schemaelementen oder nachweisbare Widersprüche zwischen Aufgabentext und SQL-Anfrage. Kandidaten, auf die eine solche Regel zutrifft, werden aus dem Aufgabenpool entfernt.

Review-Regeln erfassen dagegen Auffälligkeiten, die nicht zwangsläufig einen fachlichen Fehler darstellen, aber einer zusätzlichen Beurteilung bedürfen. Die betroffenen Aufgaben verbleiben im Datenbestand und werden für eine abschließende fachliche Kontrolle markiert. Auf diese Weise führt eine unsichere automatische Einschätzung nicht unmittelbar zum Ausschluss einer potenziell geeigneten Aufgabe. Das entwickelte Filterprogramm wendet beide Regelarten reproduzierbar auf sämtliche zuvor durch die Rekonstruktionsprüfung übernommenen Kandidaten an und speichert sowohl die Ausschlussentscheidung als auch eine mögliche Review-Markierung.

3.3.6. Finaler Aufgabenpool

Der finale Aufgabenpool enthält Aufgaben, deren Musterlösungs-Anfrage den Full Score erreicht hat, deren Aufgabentext die Rekonstruktionsprüfung bestanden hat und auf die keine harte Ausschlussregel des zusätzlichen Qualitätsfilters zutrifft. Für eine Überprüfung markierte Aufgaben bleiben im Datenbestand, sind jedoch vor einem produktiven Einsatz fachlich zu prüfen. Jedes Element dieser Menge besteht aus einem Datenbankschema, einem natürlichsprachlichen Aufgabentext und einer geprüften Musterlösungs-Anfrage.

Trotz dieser mehrstufigen Prüfung ist der finale Aufgabenpool nicht als garantiert fehlerfrei zu verstehen. Die einzelnen Prüfschritte reduzieren die Wahrscheinlichkeit fehlerhafter Aufgaben, können aber nicht alle Sonderfälle vollständig abdecken. Eine SQL-Anfrage kann trotz maximaler Bewertung einen nicht erkannten semantischen Fehler enthalten. Ebenso kann ein Aufgabentext trotz bestandener Rekonstruktionsprüfung in Einzelfällen nicht exakt zur Musterlösungs-Anfrage passen. Diese Einschränkung ist auch aus der Evaluation von Text-to-SQL-Systemen bekannt: Ausführungsbasierte Vergleiche können Hinweise auf semantische Äquivalenz liefern, garantieren diese aber nicht vollständig, da unterschiedliche SQL-Anfragen auf einem konkreten Datenbankzustand zufällig dieselbe Ergebnisrelation liefern können [16]. Die Pipeline liefert daher einen automatisch stark vorgefilterten Aufgabenpool, ersetzt aber keine abschließende fachliche Kontrolle, wenn einzelne Aufgaben produktiv eingesetzt werden sollen.

Mit Abschluss dieses Schrittes ist das Ziel der Pipeline erreicht: Aus den ursprünglich gegebenen Datenbankschemata wurden vollständige SQL-Übungsaufgaben mit zugehöriger Musterlösungs-Anfrage erzeugt. Dieser finale Aufgabenpool bildet damit das zentrale Ergebnis der in dieser Arbeit entwickelten Vorgehensweise.

4. Implementierung

Dieses Kapitel beschreibt die konkrete Umsetzung der Pipeline zur Generierung, Analyse und Verbesserung von SQL-Anfragen und der Erzeugung der dazugehörigen natürlichsprachlichen Aufgabentexte. Die Implementierung folgt dem in Kapitel 3

beschriebenen Ablauf: Zunächst werden die Datenbankschemata aus dem Spider-Datensatz aufbereitet, anschließend werden Promptvarianten erzeugt und zur initialen SQL-Generierung verwendet. Die generierten SQL-Anfragen werden danach automatisch analysiert, bewertet und für weitere Verarbeitungsschritte genutzt. Zur vollständigen Pipeline gehört außerdem die anschließende Erzeugung eines natürlichsprachlichen Aufgabentextes aus dem Datenbankschema und der geprüften SQL-Anfrage. Erst durch die Kombination aus Schema, Aufgabentext und Musterlösungs-Anfrage entsteht eine vollständige SQL-Übungsaufgabe.

Ein besonderer Schwerpunkt liegt auf der automatischen Prüfung der generierten SQL-Anfragen. Da die Ausgabe eines Sprachmodells nicht zuverlässig als korrekt angenommen werden kann, werden mehrere unabhängige Prüfschritte kombiniert. Dazu gehören Parsing und Validierung, Ausführung auf der Datenbank, Fremdschlüsselprüfung, Linting sowie die Prüfung der vorgegebenen SQL-Schwierigkeitskategorie. Die Ergebnisse dieser Schritte werden in einem gemeinsamen Datenformat gespeichert und später für Score-Berechnung, Kandidatenauswahl, Trainingsdatenerzeugung und den Verbesserungsschritt weiterverwendet.

Die rechenintensiven Generierungs- und Trainingsschritte wurden auf der Recheninfrastruktur der Martin-Luther-Universität Halle-Wittenberg ausgeführt. Die benötigten Ressourcen wurden über Slurm [29] angefordert, die Ausführung erfolgte in einer mit Docker umgesetzten containerisierten Umgebung [30]. Dadurch konnten die verschiedenen Generierungs- und Trainingsläufe unter vergleichbaren technischen Bedingungen durchgeführt werden.

Die Pipeline ist als Folge eigenständiger Verarbeitungsschritte umgesetzt, die ihre Ergebnisse über strukturierte JSON-Dateien austauschen. Python wird für die Datenaufbereitung, die Modellaufrufe, die Zusammenführung der Analyseergebnisse und die nachgelagerten Filter verwendet. Die SQL-Analyse ist überwiegend in Java implementiert und nutzt Apache Calcite [31] zur strukturellen Verarbeitung der Anfragen. Für Ausführungstests werden die SQLite-Datenbanken [32] des Spider-Datensatzes verwendet. Diese Aufteilung ermöglicht es, einzelne Verarbeitungsschritte getrennt auszuführen und ihre Zwischenergebnisse zu kontrollieren.

4.1. Aufbereitung des Spider-Datensatzes

Als Datengrundlage werden die relationalen Datenbanken des Trainingssplits des Spider-Datensatzes verwendet [3]. Die zugehörige Datensatzdatei enthält 7 000 Einträge für 140 unterschiedliche Datenbankkennungen. Für die Aufbereitung wird jede dieser Datenbanken genau einmal berücksichtigt und um die für die nachfolgenden Verarbeitungsschritte benötigten Schema- und Metadaten ergänzt. Die im Spider-Datensatz enthaltenen natürlichsprachlichen Fragen und Referenzanfragen werden nicht übernommen, da die Pipeline neue SQL-Anfragen und Aufgabentexte erzeugt.

Die Aufbereitung umfasst die Namen der Tabellen und Spalten, die Datentypen sowie die Primär- und Fremdschlüsselbeziehungen. Diese Informationen werden vorrangig aus den SQL-Schemadefinitionen ermittelt. Können einzelne Angaben dort nicht eindeutig bestimmt werden, werden ergänzend die Metadaten des Spider-Datensatzes verwendet. Dabei werden auch zusammengesetzte Primär- und Fremdschlüssel berücksichtigt.

Zusätzlich werden aus dem zugehörigen SQLite-Datenbankzustand für jede Tabelle bis zu drei Zeilen zufällig ausgewählt. Die darin enthaltenen Werte werden den jeweiligen Spalten als Beispielwerte zugeordnet. Doppelte Werte werden nicht mehrfach aufgenommen. Liegen für eine Spalte weniger als drei unterschiedliche Werte vor, werden die fehlenden Einträge durch zum jeweiligen Datentyp passende Platzhalter ergänzt. Die ausgewählten Werte bilden keinen vollständigen Wertebereich ab, liefern dem Sprachmodell jedoch Hinweise auf mögliche Ausprägungen und die inhaltliche Bedeutung eines Attributs.

Die extrahierten Informationen werden zunächst in einem strukturierten Zwischenformat gespeichert. Dieses enthält für jede Datenbank eine eindeutige Kennung, die Primär- und Fremdschlüsselbeziehungen, die Datentypen der Spalten sowie die ausgewählten Beispielzeilen. Für die spätere Übergabe an das Sprachmodell werden diese Bestandteile in eine kompakte textuelle Schemabeschreibung überführt.

Ein gekürzter Ausschnitt für die Tabelle `department` lautet:

```
department(
    Department_ID(5, 8, 12),
    Name('Interior', 'Labor', 'Energy'),
    Creation('1849', '1913', '1977'),
    Ranking(5, 8, 12)
)
```

Primär- und Fremdschlüsselbeziehungen werden zusätzlich separat aufgeführt, damit die Beziehungen zwischen den Tabellen eindeutig erhalten bleiben. Das Ergebnis dieses Schrittes ist ein strukturierter Datensatz für jede verwendete Spider-Datenbank. Dieser stellt sowohl die Informationen für die anschließende Promptgenerierung als auch die in späteren Analyse- und Ausführungsschritten benötigten Metadaten bereit.

4.2. Implementierung der Promptvarianten für die initiale SQL-Generierung

Die in Abschnitt 3.2.2 definierten SQL-Schwierigkeitskategorien werden in der Implementierung durch die Kennungen `diff1` bis `diff8` repräsentiert. Zur Erzeugung eines Prompts werden die aufbereitete Schemabeschreibung, die Primär- und Fremdschlüsselbeziehungen sowie die Beschreibung der ausgewählten SQL-Schwierigkeitskategorie zusammengeführt.

Die vier Promptvarianten werden als unterschiedliche Nachrichtenfolgen für das Sprachmodell umgesetzt. Der direkte Prompt besteht aus einer Systemanweisung und einer einzelnen Nutzereingabe, welche die Datenbankinformationen und die SQL-Schwierigkeitskategorie enthält. Beim Divide-and-Conquer-Prompt wird die Bearbeitung in die interne Analyse des Schemas, die Planung der benötigten SQL-Konstrukte und die abschließende Erzeugung der Anfrage gegliedert.

Der mehrstufige Planungsdialog verteilt die Auswahl der Tabellen und Verknüpfungen, die Festlegung der benötigten Bedingungen und die Planung der SQL-Klauseln auf mehrere aufeinanderfolgende Nachrichten. Beim Few-Shot-Prompt werden zusätzlich drei Beispielanfragen der jeweils geforderten SQL-Schwierigkeitskategorie übergeben. Diese dienen als strukturelle Orientierung und stammen nicht aus der aktuell verarbeiteten Datenbank.

Unabhängig von der Promptvariante wird das Modell angewiesen, genau eine SQL-Anfrage zu erzeugen. Die Ausgabe muss ausschließlich die Anfrage zwischen den Markierungen `<SQL>` und `</SQL>` enthalten. Die vollständigen Nachrichtenfolgen aller vier Promptvarianten sind anhand eines echten Beispiels in Anhang B dokumentiert.

4.3. Initiale SQL-Generierung

Die initiale SQL-Generierung erzeugt aus den vorbereiteten Prompts SQL-Anfragen mit einem Sprachmodell. Für jede Datenbank und SQL-Schwierigkeitskategorie werden die vier Promptvarianten ausgeführt. Diese vier Generierungen bilden gemeinsam einen Durchlauf und liefern jeweils einen SQL-Anfrage-Kandidaten. Jede Modellausgabe wird anschließend gespeichert und mit den zugehörigen Metadaten verknüpft.

Die Generierung wurde mit Ollama umgesetzt. Ollama ist eine Software zur lokalen Bereitstellung und Ausführung von Sprachmodellen und stellt für Modellaufrufe eine einheitliche Programmierschnittstelle bereit [33]. Die für die Modellaufrufe verwendeten Inferenzparameter sind in Anhang A dokumentiert. Das für einen Durchlauf verwendete Sprachmodell konnte konfiguriert werden, ohne die nachfolgenden Verarbeitungsschritte zu verändern. Dadurch ließen sich unterschiedliche Modelle mit identischen Eingaben und Ausgabeanforderungen untersuchen. Die verwendeten Modelle und die Ergebnisse des Vergleichs werden in Kapitel 5 gegenübergestellt.

Die erzeugte SQL-Anfrage wird gemeinsam mit der Datenbankkennung, der SQL-Schwierigkeitskategorie, der verwendeten Promptvariante und den zugehörigen Schemainformationen gespeichert.

Für jeden vorgesehenen Kandidaten werden höchstens drei Generierungsversuche durchgeführt. Nach jedem Versuch wird zunächst der zwischen den Markierungen `<SQL>` und `</SQL>` enthaltene Inhalt extrahiert. Fehlen diese Markierungen, versucht die Implementierung, eine SQL-Anfrage aus einem Codeblock oder anhand typischer SQL-Schlüsselwörter zu bestimmen. Für eine erste syntaktische Plausibilitätsprüfung wird SQLGlot eingesetzt. SQLGlot ist ein in Python implementierter SQL-Parser und -Transpiler, der SQL-Anfragen in einen Syntaxbaum überführen und in unterschiedlichen SQL-Dialekten ausgeben kann [34]. Als vorläufig plausibel gilt eine

extrahierte Anfrage, wenn sie mit einem unterstützten SQL-Schlüsselwort beginnt und durch SQLGlot geparst werden kann. Sobald diese Bedingungen erfüllt sind, werden die weiteren Generierungsversuche abgebrochen. Andernfalls wird das Modell erneut aufgerufen. Kann in drei Versuchen keine plausible SQL-Anfrage erzeugt werden, wird der Datensatz mit einem Generierungsfehler gespeichert.

Innerhalb eines Generierungsversuchs wird ein Aufruf der Modellschnittstelle bei einem Übertragungsfehler, einem fehlerhaften HTTP- oder JSON-Ergebnis oder einer leeren Modellantwort ebenfalls höchstens dreimal wiederholt. Diese Wiederholungen behandeln technische Fehler der Kommunikation und sind von den drei Generierungsversuchen zur Erzeugung einer plausiblen SQL-Anfrage zu unterscheiden.

Nachbearbeitung der SQL-Anfrage Nach der Extraktion aus der Modellantwort nutzt die Implementierung SQLGlot außerdem zur Nachbearbeitung der Bezeichner. Dazu wird die Anfrage zunächst im PostgreSQL-Dialekt geparst. Die im Schema bekannten Tabellen- und Spaltennamen sowie Aliasnamen werden im Syntaxbaum als quotierte Bezeichner markiert. Der resultierende Baum wird danach im MySQL-Dialekt serialisiert, um diese Bezeichner einheitlich mit Backticks zu maskieren. Die so erzeugte Schreibweise wird von SQLite akzeptiert, die Serialisierung dient hier daher der konsistenten Maskierung und nicht der Festlegung des späteren Ausführungsdialekts. Kann SQLGlot eine Anfrage nicht parsen, bleibt die extrahierte Anfrage unverändert und wird den nachfolgenden Prüfschritten übergeben. Diese Nachbearbeitung ist somit keine Validierung und ersetzt insbesondere nicht die anschließende Prüfung mit Calcite und SQLite.

Die Generierung selbst wird bewusst von der Bewertung getrennt. Das Modell erzeugt zunächst nur Kandidaten. Ob diese Kandidaten korrekt, ausführbar und für die vorgegebene SQL-Schwierigkeitskategorie geeignet sind, wird erst in den nachgelagerten Analyseschritten entschieden.

4.4. Automatische Analyse der generierten SQL-Anfragen

Nach der initialen Generierung werden die SQL-Anfragen einer mehrstufigen Analyse unterzogen. Dabei werden unterschiedliche Fehlerarten getrennt erfasst, da ein

einzelner Prüfschritt keine hinreichende Aussage über die Eignung einer Anfrage als Musterlösungs-Anfrage ermöglicht. Eine Anfrage kann beispielsweise syntaktisch gültig sein, aber nicht zum Datenbankschema passen, bei der Ausführung scheitern oder die vorgegebene SQL-Schwierigkeitskategorie verfehlen.

Die automatische Analyse umfasst die folgenden Prüfschritte:

1. syntaktisches Parsen der SQL-Anfrage,
2. semantische Validierung gegenüber dem Datenbankschema,
3. Ausführung der Anfrage auf der zugehörigen Datenbank,
4. Prüfung des erzeugten Anfrageergebnisses,
5. Analyse der verwendeten Fremdschlüsselbeziehungen,
6. Linting zur Erkennung semantisch auffälliger SQL-Konstrukte,
7. Prüfung der vorgegebenen SQL-Schwierigkeitskategorie.

Für jeden Prüfschritt werden der jeweilige Prüfstatus und gegebenenfalls konkrete Fehlerhinweise festgehalten. Die Einzelergebnisse werden anschließend zu einem gewichteten Score zusammengeführt. Dieser ermöglicht die vergleichende Bewertung der generierten Anfragen, während die zugehörigen Fehlerhinweise als Grundlage für die spätere Verbesserung fehlerhafter Kandidaten erhalten bleiben. Die folgenden Unterabschnitte beschreiben die Umsetzung der einzelnen Prüfschritte.

4.4.1. Parsing und semantische Validierung

Die Prüfschritte 1 und 2 werden mit Apache Calcite umgesetzt [31]. Apache Calcite ist ein Java-basiertes Framework zur Verarbeitung relationaler Anfragen und stellt unter anderem einen SQL-Parser, einen Validator sowie interne Repräsentationen für SQL-Ausdrücke bereit. In dieser Arbeit dient Calcite als Analysewerkzeug und nicht als ausführendes Datenbanksystem.

Beim Parsing wird die generierte SQL-Anfrage zunächst auf Übereinstimmung mit der SQL-Grammatik geprüft und in einen `SqlNode`-Baum überführt. Ein `SqlNode` ist Calcites strukturierte Repräsentation einer SQL-Anfrage. Die Knoten dieses Baums

bilden unter anderem die `SELECT`-Liste, die `FROM`-Klausel, Tabellenverknüpfungen und Bedingungen ab. Kann aus der Eingabe kein solcher Baum erzeugt werden, wird der Parsing-Schritt als fehlgeschlagen gewertet und die zugehörige Fehlermeldung festgehalten.

Für die anschließende Validierung wird das Datenbankschema in eine von Calcite verarbeitbare Schema-Repräsentation überführt. Dazu wird jede Tabelle mit ihren Spalten und Datentypen registriert. Die im Spider-Datensatz verwendeten SQLite-Datentypen werden dabei auf entsprechende Calcite-Typen abgebildet: Texttypen werden als `VARCHAR`, Ganzzahlen als `INTEGER`, Fließkommazahlen als `FLOAT` und Datums- beziehungsweise Zeitangaben als `TIMESTAMP` repräsentiert. Nicht eindeutig zuordenbare Typen werden als allgemeiner Typ behandelt. Tabellen- und Spaltennamen werden für die Verarbeitung vereinheitlicht.

Der Calcite-Validator prüft den beim Parsing erzeugten `SqlNode`-Baum gegen diese Schema-Repräsentation. Dabei werden Tabellen-, Spalten- und Aliasnamen aufgelöst und auf ihre Existenz sowie eindeutige Zuordenbarkeit geprüft. Auf diese Weise werden Anfragen erkannt, die zwar der SQL-Grammatik entsprechen, jedoch nicht mit dem zugrunde liegenden Datenbankschema vereinbar sind.

Als Ergebnis werden die strukturierte Darstellung der geparsten Anfrage, die validierte Darstellung sowie der jeweilige Prüfstatus gespeichert. Schlägt das Parsing oder die Validierung fehl, wird die Fehlermeldung übernommen und die Anfrage in diesem Analyselauf nicht ausgeführt oder durch die nachfolgenden strukturabhängigen Prüfungen verarbeitet. Bei erfolgreicher Validierung kann die ursprünglich generierte Anfrage den weiteren Analyseschritten übergeben werden.

4.4.2. Ausführung der SQL-Anfragen

Die zuvor erzeugten SQL-Anfragen stellen zunächst Kandidaten dar, aus denen im weiteren Verlauf die Musterlösungs-Anfragen ausgewählt werden. Im Prüfschritt 3 wird jeder Kandidat auf der zugehörigen Spider-Datenbank ausgeführt. Da diese Datenbanken im SQLite-Format vorliegen, erfolgt die Ausführung im SQLite-Dialekt. Als Ergebnis wird festgehalten, ob die Ausführung erfolgreich abgeschlossen wurde oder welche

Fehlermeldung aufgetreten ist. Die Ausführbarkeit ist eine grundlegende Voraussetzung für die Verwendung einer Anfrage als Musterlösungs-Anfrage und wird daher bei der späteren Score-Berechnung besonders stark gewichtet.

Bei einer erfolgreichen Ausführung werden zusätzlich die Spaltenbezeichnungen, die Gesamtzahl der Ergebniszeilen und die ersten fünf Zeilen des Ergebnisses gespeichert. Diese Daten bilden die in Prüfschritt 4 betrachtete Ergebnisvorschau. Anhand dieser Vorschau lässt sich prüfen, ob die Anfrage auf dem vorliegenden Datenbankzustand ein nicht leeres und strukturell nachvollziehbares Ergebnis erzeugt, ohne das vollständige Anfrageergebnis speichern zu müssen.

Ein leeres Ergebnis stellt keinen Ausführungsfehler dar und kann auch aus einer fachlich korrekten Anfrage hervorgehen. Für den Einsatz als Übungs- oder Klausuraufgabe ist eine solche Anfrage jedoch in der Regel weniger geeignet. Die Lernenden sollen das Ergebnis ihrer Lösung anhand konkreter Datensätze nachvollziehen und überprüfen können. Eine Anfrage ohne Ergebniszeilen bietet hierfür keine ausreichende Grundlage. Zudem besitzt die Übereinstimmung zweier leerer Ergebnisse beim späteren Ausführungsvergleich nur eine geringe Aussagekraft, da auch semantisch unterschiedliche Anfragen zufällig jeweils keine Zeilen liefern können. Leere Ergebnisse werden deshalb gesondert erfasst und bei der Bewertung gegenüber nicht leeren Ergebnissen nachrangig behandelt.

4.4.3. Erkennung von Fremdschlüsselverwendungen in SQL-Anfragen

Prüfschritt 5 untersucht, ob die in einer SQL-Anfrage ausgedrückten Spaltenbeziehungen mit den im Datenbankschema definierten Fremdschlüsseln übereinstimmen. Hierzu wurde ein eigener Fremdschlüsselchecker entwickelt, der Beziehungen aus unterschiedlichen SQL-Konstrukten extrahiert, normalisiert und mit den Schemavorgaben vergleicht. Eine Übereinstimmung belegt die Verwendung einer definierten Beziehung. Sie bestätigt für sich genommen noch nicht die fachliche Eignung der gesamten Anfrage. Umgekehrt gilt ein nicht verwendeter Fremdschlüssel nicht als Fehler, da nicht jede Anfrage sämtliche Beziehungen eines Schemas benötigt.

Eingaben und Datenbasis Die Prüfung verwendet die erzeugte SQL-Anfrage, die Tabellen- und Spalteninformationen sowie die Fremdschlüsseldefinitionen der jeweils zugehörigen Datenbank. Die Spalteninformationen ermöglichen es, Bezeichner auf Tabellen zurückzuführen und implizite Join-Bedingungen aufzulösen. Die vollständige Eingabestruktur einschließlich der intern verwendeten Feldbezeichnungen ist in Anhang C dargestellt.

Algorithmischer Ablauf Der Fremdschlüsselchecker parst die Anfrage mit Calcite und verarbeitet den entstehenden abstrakten Syntaxbaum. Zunächst werden Aliasnamen den referenzierten Tabellen zugeordnet. Für Unteranfragen werden die Zuordnungen des jeweiligen Anfragekontexts berücksichtigt, damit qualifizierte und eindeutig zuordenbare unqualifizierte Spaltenreferenzen aufgelöst werden können. Anschließend durchsucht eine rekursive Traversierung den Syntaxbaum nach Spaltenbeziehungen. Eine exemplarische Darstellung der dabei ausgewerteten Syntaxbaumknoten ist in Anhang D enthalten. Der vollständige Pseudocode ist als Algorithmus 1 in Anhang E dokumentiert.

Die konstruktbezogene Auswertung unterscheidet die in Tabelle 1 aufgeführten Formen. Ausführliche Beispiele zu den einzelnen Konstrukten enthält Anhang F.1.

Konstrukt	**Auswertung**
`NATURAL JOIN`	Ableitung von Gleichheiten aus gleichnamigen Spalten beider Tabellen.
`JOIN USING`	Zuordnung der angegebenen Spalten zu den beiden Join-Operanden.
`JOIN ON`, `WHERE`	Extraktion expliziter Spaltengleichheiten einschließlich konjunktiv verknüpfter und mehrspaltiger Bedingungen.
`IN`, `NOT IN`	Positionsweise Zuordnung äußerer Spalten zu den Ausgabespalten der Unteranfrage.
`EXISTS`, `NOT EXISTS`	Untersuchung der innerhalb der Unteranfrage ausgedrückten und mit dem äußeren Anfragekontext verbundenen Spaltenbeziehungen.

Tabelle 1: Vom Fremdschlüsselchecker ausgewertete SQL-Konstrukte

Für den abschließenden Abgleich werden Aliase durch Tabellennamen ersetzt, Spalten vollständig qualifiziert und Groß- und Kleinschreibung vereinheitlicht. Die Reihenfolge der Operanden einer Gleichheit bleibt unberücksichtigt. Für jeden im Schema definierten Fremdschlüssel wird anschließend geprüft, ob sämtliche zugehörigen Spaltenpaare in der Anfrage vorkommen. Zusätzliche Gleichheiten ohne passende Fremdschlüsseldefinition werden als auffällig ausgewiesen. Definierte, aber nicht benötigte Beziehungen werden davon getrennt als nicht verwendet dokumentiert. Das Analyseergebnis hält außerdem fest, ob ein Selbstverbund erkannt wurde. Ein vollständiges Ergebnisobjekt ist in Anhang F.2 dargestellt.

Validierung der Fremdschlüsselanalyse Die Implementierung wird mit 42 parameterisierten Unit-Tests gegen manuell erstellte Referenzauswertungen geprüft. Die Tests verteilen sich auf `JOIN ON` (20 Fälle), `JOIN USING` (4), `NATURAL JOIN` (4), Fremdschlüsselbedingungen in `WHERE` (8) sowie `IN` und `NOT IN` (6). Neben regulären Verwendungen enthalten sie fehlerhafte Zuordnungen, Aliase, mehrteilige Bedingungen und Selbstverbunde. Für jeden Fall werden Parsingstatus und sämtliche Felder des normalisierten Analyseergebnisses mit der Referenz verglichen. Die detaillierte Teststruktur und ein exemplarischer Testablauf sind in Anhang F.3 dokumentiert. Die Tests decken die für diese Arbeit implementierten Ausdrucksformen ab. Darüber hinausgehende SQL-Konstrukte erfordern zusätzliche Prüffälle.

4.4.4. Linting der SQL-Anfragen

Der Prüfschritt 6 setzt SQLlint ein, um ergänzend zu den bisherigen Prüfungen semantisch auffällige Konstruktionen in den generierten SQL-Anfragen zu erkennen. Grundlage sind die von Brass und Goldberg beschriebenen Fehlerklassen [13]. SQLlint untersucht die Anfragestruktur anhand implementierter Prüfregeln und bezieht dabei Informationen aus dem Datenbankschema ein. Anders als der Ausführungstest betrachtet es damit nicht allein, ob eine Anfrage verarbeitet werden kann, sondern auch, ob ihre Formulierung auf einen möglichen Bedeutungsfehler oder eine unnötige Konstruktion hinweist.

SQLlint erkennt unter anderem problematische Vergleiche mit `NULL` sowie redundante

Konstruktionen in Aggregationen. Beispielsweise kann ein Vergleich mit `NULL` syntaktisch gültig und ausführbar sein, aufgrund der dreiwertigen SQL-Logik jedoch nicht die beabsichtigte Filterwirkung besitzen. Solche Befunde ergänzen die vorherigen Prüfungen um Hinweise auf mögliche semantische Fehler und unnötige Konstruktionen.

Für die technische Einbindung wird SQLlint in einem Docker-Container betrieben. Die öffentlich zugängliche Projektseite dokumentiert den SQLlint-Prototyp und die zugehörigen Veröffentlichungen [14]. Die verwendete Docker-Konfiguration wurde aus einem universitätsinternen Software-Repository zur Evaluation von Äquivalenzbeweisern für SQL-Anfragen übernommen [35] und stellt SQLlint einschließlich seiner Laufzeitumgebung zur Verfügung. Ein Python-Skript steuert den Dienst über Docker Compose und ruft die Prüfung für die einzelnen Anfragen im laufenden Container auf. Schema- und Anfragedateien werden über ein zwischen Host und Container eingebundenes Verzeichnis bereitgestellt. Über dasselbe Verzeichnis wird anschließend die von SQLlint erzeugte CSV-Ausgabe eingelesen.

Vor dem Aufruf wird das zugehörige Schema an die von SQLlint unterstützte Schemaformulierung angepasst. Dazu gehören die Abbildung nicht unterstützter Datentypen und die Vereinheitlichung von Fremdschlüsseldefinitionen. Die eingelesenen Befunde werden mit Fehlercode, Beschreibung und Häufigkeitsangaben in die JSON-Analyseausgabe übernommen. Scheitert der Werkzeugaufruf, werden stattdessen Statusinformationen und die verfügbare Fehlermeldung gespeichert. Ein technisch nicht abgeschlossener Prüflauf ist damit von einem Prüflauf ohne gemeldete Auffälligkeiten unterscheidbar.

Eine Sonderbehandlung betrifft den Code `PORT_COL_NAME`, der mögliche Portabilitätsprobleme impliziter Ausgabespaltennamen bezeichnet. Diese Meldung wurde für die verwendeten Anfragen nicht als geeignetes Bewertungskriterium übernommen und wird beim Einlesen der Ergebnisse herausgefiltert. Die übrigen Befunde bleiben für die anschließende Bewertung und Fehleranalyse erhalten.

Die Prüfung ist auf die tatsächlich implementierten Regeln beschränkt und deckt nicht sämtliche Fehlerklassen des zugrunde liegenden Katalogs ab. Insbesondere stellt ein Prüflauf ohne Lint-Meldung keinen Nachweis der fachlichen Fehlerfreiheit dar. Ob eine

Anfrage den beabsichtigten Aufgabentext beantwortet, wird durch diese regelbasierte Untersuchung allein nicht festgestellt.

4.4.5. Prüfung der erwarteten SQL-Schwierigkeitskategorie

Der abschließende Prüfschritt 7 untersucht, ob die Struktur der generierten SQL-Anfrage der für den jeweiligen Lauf vorgegebenen SQL-Schwierigkeitskategorie entspricht. Hierzu wird die von Calcite erzeugte relationale Darstellung der Anfrage rekursiv nach den für die Kategorien maßgeblichen Konstrukten durchsucht. Erfasst werden unter anderem Joins, Aggregationen, `GROUP BY`, `HAVING`, Mengenoperationen sowie `NOT EXISTS` und `NOT IN`.

Die Zuordnung erfolgt anhand der Kombination dieser Merkmale. So wird beispielsweise zwischen Aggregationen ohne Gruppierung, Aggregationen mit `GROUP BY` und Aggregationen mit zusätzlicher `HAVING`-Bedingung unterschieden. Weitere Kategorien erfassen unter anderem reguläre Joins, Left Joins, Mengenoperationen sowie Anfragen mit `NOT EXISTS` oder `NOT IN` ohne Aggregation. Die zuletzt genannte Kategorie bildet damit eine bestimmte Form von Negation ab, ist jedoch nicht mit der allgemeinen Klasse aller nicht-monotonen Anfragen gleichzusetzen. Insbesondere können auch gruppierte Aggregationen nicht-monotones Verhalten aufweisen.

Die ermittelte Kategorie wird mit der für den Lauf vorgegebenen Kategorie verglichen. Stimmen beide Werte überein, gilt der Prüfschritt als bestanden. Kann keine der definierten Kategorien zugeordnet werden oder weicht die ermittelte Kategorie von der Vorgabe ab, schlägt die Prüfung fehl. Das Ergebnis wird zusammen mit der erwarteten und der tatsächlich ermittelten Kategorie in der Analyseausgabe gespeichert und anschließend in die Score-Berechnung einbezogen.

Auf diese Weise wird überprüft, ob eine formal gültige und ausführbare Anfrage auch die für die spätere Übungsaufgabe vorgesehene SQL-Struktur verwendet. Die Prüfung stellt jedoch keine allgemeine Bestimmung der semantischen oder mathematischen Schwierigkeit einer Anfrage dar, sondern eine regelbasierte Zuordnung zu den in dieser Arbeit festgelegten Kategorien.

4.5. Berechnung des Scores

Die Ergebnisse der sieben Prüfschritte werden für jede SQL-Anfrage sowohl einzeln gespeichert als auch zu einem gewichteten Score zusammengeführt. Für jeden bestandenen Prüfschritt wird die ihm zugeordnete Punktzahl zum Score addiert. Ein nicht bestandener Prüfschritt trägt keine Punkte bei.

Die verwendete Gewichtung lautet:

- Parsing: 3 Punkte,
- Validierung: 5 Punkte,
- Ausführung: 12 Punkte,
- Fremdschlüsselprüfung: 6 Punkte,
- Linting: 4 Punkte,
- Prüfung der SQL-Schwierigkeitskategorie: 6 Punkte,
- Ergebnisvorschau: 1 Punkt.

Der maximal erreichbare Score beträgt 37 Punkte. Da alle Gewichte positiv sind, wird dieser Wert genau dann erreicht, wenn sämtliche Prüfschritte bestanden wurden. Die Ausführbarkeit erhält mit zwölf Punkten das höchste Gewicht, da eine nicht ausführbare Anfrage unabhängig von ihren übrigen Eigenschaften nicht als Musterlösungs-Anfrage verwendet werden kann. Die Ergebnisvorschau wird dagegen mit einem Punkt gewichtet: Ein leeres Ergebnis vermindert die Eignung für eine Übungsaufgabe, stellt jedoch keinen ebenso grundlegenden Fehler wie eine nicht ausführbare Anfrage dar.

Der Score verdichtet die Einzelergebnisse zu einer vergleichbaren Kennzahl, ersetzt jedoch nicht die Betrachtung der einzelnen Prüfungen. Für die Kandidatenauswahl wird zunächst der gewichtete Score herangezogen. Erreichen mehrere Kandidaten denselben maximalen Score, wird der in der festen Eingabereihenfolge zuerst auftretende Kandidat ausgewählt. Die detaillierten Fehlerhinweise bleiben außerdem für die Trainingsdatenerzeugung und den Verbesserungsschritt erhalten.

Für die nachfolgende Datenaufbereitung erfüllt der Score ergänzende Funktionen. Er ermöglicht die Identifikation aller Anfragen, die mit 37 Punkten sämtliche Prüfschritte bestanden haben, und quantifiziert den Qualitätsabstand zwischen unterschiedlich bewerteten Anfragen. Dieser Abstand wird bei der Bildung der DPO-Präferenzpaare verwendet. Zusätzlich müssen dabei festgelegte verpflichtende Prüfungen bestanden sein. Die Trainingsdaten werden somit nicht allein anhand eines Zahlenwertes gefiltert, sondern anhand einer Kombination aus Score, Score-Abstand und konkreten Testergebnissen.

4.6. Auswahl des besten Kandidaten aus mehreren Promptvarianten

Für jeden Durchlauf liegen vier analysierte SQL-Anfrage-Kandidaten vor, die mit den vier Promptvarianten für dieselbe Datenbank und dieselbe SQL-Schwierigkeitskategorie erzeugt wurden. Aus dieser Gruppe wird ein Kandidat für die weitere Verarbeitung ausgewählt. Dadurch reduziert sich die Datenmenge auf eine SQL-Anfrage pro Durchlauf.

Grundlage der Auswahl ist der in Abschnitt 4.5 beschriebene gewichtete Score. Für jeden Durchlauf wird zunächst der höchste unter den vier Kandidaten erreichte Score bestimmt. Liegt nur ein Kandidat mit diesem Score vor, wird er unmittelbar übernommen. Erreichen mehrere Kandidaten denselben maximalen Score, wird der in der festen Reihenfolge der Promptvarianten zuerst auftretende Kandidat ausgewählt. Dadurch bleibt die Entscheidung bei einem Gleichstand reproduzierbar.

Für den ausgewählten Kandidaten werden neben der SQL-Anfrage auch die Analyseergebnisse und die zugehörige Promptvariante gespeichert. Dadurch bleibt nachvollziehbar, aus welcher Promptvariante die weiterverwendete Anfrage hervorgegangen ist. Diese Information ermöglicht es, die Ergebnisse später getrennt nach Promptvarianten auszuwerten.

4.7. Erzeugung der Trainingsdaten für SFT und DPO

Die Ergebnisse der automatischen Analyse werden zusätzlich zur Erstellung von Trainingsdaten für das Reparaturmodell verwendet. Dafür werden SQL-Anfragen zusammengefasst, die unter denselben Bedingungen, also für dieselbe Datenbank, dieselbe SQL-Schwierigkeitskategorie und dieselbe Promptvariante, erzeugt wurden. Die zugehörigen Scores und Fehlerhinweise ermöglichen es, besser und schlechter bewertete Anfragen innerhalb dieser Gruppen zu unterscheiden.

Für das Supervised Fine-Tuning (SFT) werden Korrekturbeispiele gebildet. Die Eingabe enthält das aufbereitete Schema, eine fehlerhafte oder niedriger bewertete SQL-Anfrage sowie die bei ihrer Analyse ermittelten Fehlerhinweise. Diese Hinweise teilen dem Modell mit, welche Bestandteile der Anfrage verbessert werden sollen. Als Zielausgabe wird eine unter denselben Generierungsbedingungen erzeugte Anfrage verwendet, die den maximalen Score erreicht. Das Modell lernt dadurch, aus einer analysierten Anfrage und dem zugehörigen Korrekturfeedback eine besser bewertete Anfrage zu erzeugen.

Für die Direct Preference Optimization (DPO) [10] werden dagegen Präferenzpaare erstellt. Ein solches Paar besteht aus einer bevorzugten und einer verworfenen Antwort für denselben Eingabekontext. Als bevorzugte Antwort dient eine Anfrage, welche die maßgeblichen Prüfschritte bestanden und einen hohen Score erreicht hat. Ihr wird eine deutlich niedriger bewertete Anfrage gegenübergestellt. Auf diese Weise lernt das Modell nicht unmittelbar eine festgelegte Zielausgabe nachzubilden, sondern zwischen unterschiedlich geeigneten Antworten zu unterscheiden und die besser bewertete Anfrage zu bevorzugen.

Vor der Speicherung werden unvollständige und doppelte Trainingsbeispiele entfernt. Die so erzeugten SFT- und DPO-Datensätze bilden die Eingabe für die anschließende Anpassung des Reparaturmodells. Gekürzte Beispiele beider Trainingsformate sind in Anhang G dargestellt.

4.8. Fine-Tuning mit Axolotl

Die in Abschnitt 4.7 erzeugten SFT- und DPO-Datensätze bilden die Grundlage für die Anpassung der im Reparaturschritt eingesetzten Sprachmodelle. Das Training wird mit Axolotl durchgeführt. Axolotl ist ein Open-Source-Framework für das Fine-Tuning großer Sprachmodelle und übernimmt unter anderem das Einlesen und Tokenisieren der Trainingsdaten, die Anwendung des modellspezifischen Chat-Templates sowie die Konfiguration und Ausführung des Adaptertrainings [36].

Die Trainingsdaten liegen in den von Axolotl erwarteten JSONL-Strukturen vor. Beim SFT wird jeder Datensatz als Nachrichtenfolge aus Systemanweisung, Reparaturauftrag und Zielantwort eingelesen. Für das DPO-Training werden der gemeinsame Eingabekontext sowie die bevorzugte und die verworfene Antwort den entsprechenden Feldern der Trainingskonfiguration zugeordnet.

Für das SFT wird LoRA eingesetzt. LoRA ist ein Verfahren zur parametereffizienten Modellanpassung, bei dem die Parameter des Basismodells unverändert bleiben und lediglich zusätzlich eingefügte Matrizen mit niedrigem Rang trainiert werden [8]. Die Adapter werden in ausgewählte Projektionsschichten des Aufmerksamkeitsmechanismus und des Feedforward-Netzes integriert. Das Basismodell wird dabei ohne Quantisierung geladen. Zur Ausführung des verteilten Trainings wird `torchrun` eingesetzt. Dieses zu PyTorch gehörende Kommandozeilenwerkzeug startet und koordiniert die Trainingsprozesse für die beteiligten GPUs [37]. Als Parallelisierungsverfahren kommt Fully Sharded Data Parallelism (FSDP) zum Einsatz. FSDP verteilt Modellparameter, Gradienten und Optimierungszustände auf die Trainingsprozesse und setzt die jeweils benötigten Parameter während der Berechnung temporär zusammen. Dadurch sinkt der Speicherbedarf pro GPU, sodass das Modell auf den verfügbaren GPUs trainiert werden kann [38].

Für das Präferenztraining wird DPO mit QLoRA kombiniert. DPO optimiert das Modell so, dass die bevorzugte Antwort für einen gegebenen Eingabekontext eine höhere Wahrscheinlichkeit als die verworfene Antwort erhält, ohne dass dafür ein separates Reward-Modell trainiert werden muss [10]. QLoRA verbindet das Adaptertraining mit einer quantisierten Repräsentation des Basismodells [9]. In der verwendeten Konfiguration wird das Basismodell in 4-Bit-Darstellung geladen, während ausschließlich

die zusätzlichen LoRA-Parameter trainiert werden. Dadurch sinkt insbesondere der Speicherbedarf des Präferenztrainings.

Die Trainingsläufe werden als Slurm-Aufträge in einer selbst erstellten Containerumgebung ausgeführt. Als Ergebnis erzeugt Axolotl jeweils einen Adapter mit den zusätzlich trainierten Parametern. Das Basismodell selbst wird nicht überschrieben. Für die Verwendung im Reparaturschritt wird der Adapter separat mit einem Werkzeug aus `llama.cpp` in das GGUF-Format konvertiert [39]. Das Basismodell und der konvertierte Adapter werden anschließend gemeinsam über Ollama [33] bereitgestellt. Die Erstellung und Einbindung der Containerumgebung, die Konvertierung der Adapter sowie die Hyperparameter der Trainingsläufe sind in Anhang H dokumentiert.

4.9. Verbesserung generierter SQL-Anfragen durch ein zweites Modell

Die in Abschnitt 4.8 trainierten Adapter werden anschließend wieder in die Pipeline zurückgeführt. Zusammen mit dem jeweiligen Basismodell bilden sie die im zweiten Modellschritt eingesetzten Reparaturmodelle. Dieser Schritt verarbeitet die ausgewählten Kandidaten der initialen Kandidatenauswahl, deren Bewertung unterhalb des Full Scores liegt. Kandidaten, die bereits 37 Punkte erreicht haben, werden unverändert für die spätere Zusammenführung aufbewahrt.

Die Eingabe des Reparaturmodells umfasst die aufbereitete Schemabeschreibung mit den Beispieldaten, die vorgegebene SQL-Schwierigkeitskategorie, die zu verbessernde SQL-Anfrage und deren Analyseergebnis. Die fehlgeschlagenen Prüfschritte und die dazugehörigen Fehlermeldungen werden als Korrekturfeedback in den Prompt übernommen. Das Modell erhält damit die Anweisung, die vorhandene Anfrage anhand dieses Feedbacks zu überarbeiten und genau eine korrigierte SQL-Anfrage zurückzugeben.

Neben den trainierten SFT- und DPO-Varianten werden auch die zugehörigen Basismodelle getestet. Dadurch lässt sich in der Evaluation unterscheiden, ob eine Verbesserung bereits durch den zusätzlichen Modellaufruf entsteht oder auf das Fine-Tuning zurück-

zuführen ist. Die erzeugten Anfragen werden im selben strukturierten Format wie die initialen Ergebnisse gespeichert und anschließend erneut analysiert.

4.10. Erneute Analyse der verbesserten SQL-Anfragen

Die verbesserten SQL-Anfragen werden erneut mit denselben Prüfschritten analysiert, die bereits auf die initial erzeugten Kandidaten angewendet wurden. Dazu gehören Parsing, Validierung, Ausführung, Ergebnisvorschau, Fremdschlüsselprüfung, Linting und die Prüfung der erwarteten SQL-Schwierigkeitskategorie. Aus den neuen Prüfergebnissen wird nach derselben Gewichtung erneut ein Score berechnet. Dadurch lässt sich für jede verbesserte Anfrage nachvollziehen, ob das übergebene Fehlerfeedback erfolgreich umgesetzt wurde und welche Auffälligkeiten weiterhin bestehen.

Nach der erneuten Analyse werden sämtliche neu bewerteten Reparaturergebnisse mit den ausgewählten Kandidaten zusammengeführt, die bereits bei der initialen Analyse den Full Score erreicht haben. Die ursprünglichen Full-Score-Anfragen bleiben dabei unverändert. Aus der zusammengeführten Datenmenge werden anschließend alle Anfragen mit einem Score von 37 Punkten ausgewählt. Nur diese Anfragen werden als Musterlösungs-Anfragen an die Erzeugung der Aufgabentexte übergeben.

Diese Auswahl bildet den Abschluss der SQL-orientierten Pipeline-Stufe. Die so entstandene Menge der mit Full Score bewerteten Musterlösungs-Anfragen dient anschließend als Eingabe für die Erzeugung der Aufgabentexte.

4.11. Erzeugung natürlichsprachlicher Aufgabentexte

Dieser Schritt greift die bereits in OmniSQL genutzte Idee auf, aus SQL-Anfragen und Schemata natürlichsprachliche Formulierungen abzuleiten [22]. Dafür wird ein weiteres Sprachmodell verwendet, das aus einer mit Full Score bewerteten Musterlösungs-Anfrage und dem zugehörigen Datenbankschema einen passenden deutschsprachigen Aufgabentext erzeugt. Ein gekürzter Ausschnitt eines tatsächlich verwendeten Prompts ist in Anhang I dokumentiert.

Für die Erzeugung wurde das Sprachmodell `Qwen3-Coder-30B-A3B-Instruct` eingesetzt [40]. Zu jeder Musterlösungs-Anfrage werden fünf voneinander getrennte Aufgabentext-Kandidaten angestrebt. Das Modell erzeugt dabei pro Aufruf genau einen Kandidaten. Für jeden noch fehlenden Kandidaten sind bis zu drei Modellaufrufe vorgesehen. Leere Antworten und Duplikate werden nicht übernommen. Bereits akzeptierte Formulierungen werden in den folgenden Aufrufen mitgegeben, damit weitere Varianten erzeugt werden. Sobald fünf unterschiedliche Kandidaten vorliegen, endet die Generierung. Datensätze mit mindestens drei verwertbaren Aufgabentexten werden durch das semantische Auswahlverfahren verarbeitet. Liegen nur ein oder zwei Kandidaten vor, greifen die im Abschnitt 4.12 beschriebenen Ersatzregeln.

4.12. Semantische Auswahl des Aufgabentextes

Das in Abschnitt 3.3.2 konzeptionell beschriebene Auswahlverfahren wird auf die zu einer Musterlösungs-Anfrage erzeugten Aufgabentext-Kandidaten angewendet. Vor der Berechnung werden die Formulierungen normalisiert, indem führende und nachfolgende Leerzeichen entfernt und mehrere aufeinanderfolgende Leerraumzeichen zusammengefasst werden. Kandidaten ohne verwertbaren Aufgabentext bleiben unberücksichtigt.

Für Kandidatenmengen mit mindestens drei Aufgabentexten wird das Modell `sentence-transformers/all-mpnet-base-v2` verwendet [41]. Es überführt die Aufgabentexte in Vektoren, die anschließend auf ihre euklidische Norm normalisiert werden. Durch die Multiplikation der normalisierten Vektormatrix mit ihrer Transponierten entsteht die Matrix der paarweisen Kosinusähnlichkeiten. Für jeden Kandidaten wird daraus die Durchschnittsähnlichkeit zu allen übrigen Kandidaten berechnet. Die Selbstähnlichkeit auf der Hauptdiagonalen wird nicht berücksichtigt. Ausgewählt wird der Kandidat mit dem höchsten Durchschnittswert. Bei einem Gleichstand wird der in der Eingabereihenfolge zuerst auftretende Kandidat übernommen.

Liegt kein verwertbarer Kandidat vor, wird die Auswahl als fehlgeschlagen markiert. Ein einzelner Kandidat wird unmittelbar übernommen. Bei genau zwei Kandidaten kann das Ähnlichkeitskriterium nicht zwischen den Formulierungen unterscheiden, da beide nur miteinander verglichen werden. In diesem Fall erfolgt die Auswahl pseudozufällig mithilfe eines mit dem festen Startwert 42 initialisierten Zufallszahlengenerators.

Dadurch liefert das Verfahren bei identischer Eingabe und Eingabereihenfolge dasselbe Ergebnis. Das Embedding-Modell wird nur geladen, wenn mindestens ein Datensatz drei oder mehr Kandidaten enthält.

Für jeden Datensatz werden die angewendete Auswahlmethode, die Anzahl der verwertbaren Kandidaten, die berechneten Durchschnittsähnlichkeiten und der ausgewählte Aufgabentext gespeichert. Ein vollständiges Beispiel mit der Musterlösungs-Anfrage, den Aufgabentext-Kandidaten und ihren Ähnlichkeitswerten ist in Anhang J dokumentiert.

4.13. Rekonstruktionsprüfung des Aufgabentextes

Der ausgewählte Aufgabentext ist noch kein ausreichender Nachweis dafür, dass er inhaltlich zur Musterlösungs-Anfrage passt. Deshalb wird aus dem Aufgabentext und dem Datenbankschema erneut eine SQL-Anfrage erzeugt. Die ursprüngliche Musterlösungs-Anfrage wird dem Modell dabei nicht übergeben. Als Eingabe dienen nur der Aufgabentext, die Datenbankkennung und das Schema.

Für diese Rekonstruktion wird das Sprachmodell `ArcticSQL` eingesetzt. Es gehört zur Open-Source-Modellfamilie Arctic-Text2SQL-R1, die für Text-to-SQL-Aufgaben entwickelt wurde. Im zugehörigen Paper wurde die Modellfamilie unter anderem anhand der BIRD-Benchmark evaluiert und gehörte zum Zeitpunkt der Veröffentlichung zu den führenden Einträgen der Rangliste [42].

Für jeden Datensatz sind höchstens drei Modellaufrufe vorgesehen. Aus der Antwort wird zunächst der als SQL gekennzeichnete Bereich extrahiert. Fehlt eine solche Markierung, werden alternativ SQL-Codeblöcke beziehungsweise mit `SELECT` oder `WITH` beginnende Textabschnitte berücksichtigt. Nur Antworten, aus denen sich auf diese Weise eine plausible SQL-Anfrage gewinnen lässt, werden für den Ausführungsvergleich verwendet.

Die rekonstruierte und die ursprüngliche Musterlösungs-Anfrage werden anschließend auf demselben SQLite-Datenbankzustand ausgeführt. Für jede Ausführung gilt ein Zeitlimit von 15 Sekunden. Außerdem werden höchstens 100 000 Ergebniszeilen verarbeitet.

Fehlerhafte Ausführungen und Zeitüberschreitungen werden als eigener Validierungsstatus gespeichert.

Vor dem Vergleich werden die Ergebniswerte vereinheitlicht. Fließkommazahlen werden auf zwölf Nachkommastellen gerundet, ganzzahlige Fließkommawerte als Ganzzahlen und Bytefolgen hexadezimal dargestellt. Enthält mindestens eine der beiden Anfragen ein `ORDER BY`, werden die Ergebniszeilen positionsweise verglichen. Andernfalls erfolgt der Vergleich als Multimenge, sodass die Reihenfolge der Zeilen keine Rolle spielt, ihre Häufigkeit jedoch erhalten bleibt. Unterschiedliche Spaltenaliasnamen beeinflussen die Entscheidung nicht. Zusätzlich werden Hashwerte der normalisierten Ergebnisrelationen gespeichert.

Die Datenbank `music_2` aus dem Spider-Datensatz wird ausgeschlossen, da sie zwar Tabellen, aber keine Daten enthält. Ebenso werden Fälle nicht übernommen, in denen beide Anfragen ein leeres Ergebnis liefern. Ein vollständiges Beispiel einer erfolgreichen Rekonstruktionsprüfung ist in Anhang K dokumentiert.

4.14. Implementierung des zusätzlichen Qualitätsfilters

Der in Abschnitt 3.3.5 beschriebene Qualitätsfilter wird als regelbasiertes Programm auf die Aufgaben angewendet, die die Rekonstruktionsprüfung bestanden haben. Die Regeln wurden aus einer vorgelagerten modellgestützten Sichtung des Datenbestands abgeleitet. Hierfür wurde ChatGPT mit dem Modell GPT-5.6 Sol verwendet [43]. Das Sprachmodell erhielt die offene Anweisung, unpassende, fehlerhafte oder anderweitig auffällige Aufgabentexte zu identifizieren. Diese Sichtung war kein automatisierter Bestandteil des späteren Filterlaufs. Die gefundenen Fälle wurden zunächst kontrolliert und anschließend in reproduzierbar anwendbare Suchregeln überführt.

Das Filterprogramm untersucht den Aufgabentext und die zugehörige Musterlösungs-Anfrage. Für einfache sprachliche Merkmale wird eine von Groß- und Kleinschreibung unabhängige Textsuche verwendet. Komplexere Zusammenhänge zwischen Aufgabentext und SQL-Anfrage werden durch reguläre Ausdrücke geprüft. Jede erkannte Auffälligkeit wird durch einen eindeutigen Grundcode dokumentiert.

Die harten Ausschlussregeln erfassen zunächst Selbstreferenzen auf die bereits vorhandene SQL-Anfrage, beispielsweise Formulierungen wie „entsprechend der gegebenen SQL-Abfrage“. Weitere Regeln erkennen wiederkehrende Fehlübersetzungen englisch benannter Schemaelemente. So wurde `bikes_available` als „verfügbare Fähren“ statt als „verfügbare Fahrräder“ und `Num_of_shops` als „Anzahl der Schäden“ statt als „Anzahl der Geschäfte“ wiedergegeben. In der verwendeten Sportdatenbank wurde außerdem `Goals` mit „Ziele“ statt mit „Tore“ übersetzt, während `Tries` dort Versuche bezeichnet, jedoch ebenfalls als „Tore“ wiedergegeben wurde.

Zusätzlich werden Widersprüche bei Sortierungen und Ergebnisbegrenzungen geprüft. Verlangt der Aufgabentext eine Sortierung, ohne dass die Musterlösungs-Anfrage eine entsprechende `ORDER BY`-Klausel enthält, wird die Aufgabe ausgeschlossen. Dasselbe gilt, wenn die Anfrage eine erkennbare Sortierung enthält, diese im Aufgabentext jedoch nicht zum Ausdruck kommt. Entsprechend werden Abweichungen zwischen sprachlich geforderten Begrenzungen und der Verwendung von `LIMIT` in beiden Richtungen erfasst. Sobald mindestens eine harte Ausschlussregel zutrifft, wird die Aufgabe aus dem finalen Aufgabenpool entfernt.

Review-Regeln behandeln dagegen Auffälligkeiten, die nicht eindeutig auf eine fachlich falsche Aufgabe schließen lassen. Dazu zählen rohe SQL-Begriffe im Aufgabentext, technische Formulierungen zu Join-Operationen sowie Fälle, in denen die Musterlösungs-Anfrage `DISTINCT` verwendet, ohne dass der Aufgabentext die Eindeutigkeit der Ergebnisse ausdrücklich erkennen lässt. Die betroffenen Aufgaben verbleiben im Datenbestand, werden jedoch zusätzlich für eine fachliche Kontrolle markiert.

Das Programm speichert die übernommenen Aufgaben, die ausgeschlossenen Aufgaben und die Review-Fälle in getrennten Ausgaben. Für ausgeschlossene und markierte Aufgaben werden neben den ausgelösten Regeln auch die Datensatz-ID, die Datenbank, die Schwierigkeitskategorie, der Aufgabentext, die Musterlösungs-Anfrage, die rekonstruierte SQL-Anfrage und eine Vorschau des Ausführungsergebnisses festgehalten. Damit bleiben die Filterentscheidungen nachvollziehbar und können anschließend fachlich überprüft werden.

4.15. Struktur und Speicherung des finalen Aufgabenpools

Der in Abschnitt 3.3.6 konzeptionell beschriebene Aufgabenpool wird als strukturierter JSON-Datensatz gespeichert. Ausgangspunkt sind die Aufgaben, die die Rekonstruktionsprüfung bestanden haben. Datensätze, auf die mindestens eine harte Ausschlussregel des zusätzlichen Qualitätsfilters zutrifft, werden nicht übernommen. Durch eine Review-Regel markierte Aufgaben verbleiben dagegen im Aufgabenpool und werden zusätzlich in einer separaten Ausgabe für die fachliche Kontrolle erfasst.

Jeder Datensatz enthält eine eindeutige Kennung, die zugehörige Datenbank, die SQL-Schwierigkeitskategorie, das aufbereitete Datenbankschema, den ausgewählten Aufgabentext und die geprüfte Musterlösungs-Anfrage. Damit sind alle Bestandteile vorhanden, die zur Darstellung einer vollständigen SQL-Übungsaufgabe benötigt werden.

Zusätzlich bleiben die Informationen aus der Rekonstruktionsprüfung erhalten. Dazu gehören die aus dem Aufgabentext rekonstruierte SQL-Anfrage, der verwendete Vergleichsmodus, die Zeilenanzahlen und Spaltenbezeichnungen beider Ausführungsergebnisse sowie Vorschauen der Ergebnisrelationen. Außerdem wird gespeichert, ob die Ergebnisse übereinstimmen und auf welcher Grundlage diese Übereinstimmung festgestellt wurde. Diese Angaben gehören nicht zum eigentlichen Aufgabentext, ermöglichen jedoch eine spätere Kontrolle der automatischen Validierung.

Als Musterlösungs-Anfrage wird ausschließlich die zuvor analysierte und mit Full Score bewertete SQL-Anfrage gespeichert. Die rekonstruierte SQL-Anfrage dient nur der Überprüfung des Aufgabentextes und ersetzt die ursprüngliche Musterlösungs-Anfrage nicht. Auf oberster Ebene enthält die Ausgabedatei darüber hinaus Angaben zur Anzahl der übernommenen, ausgeschlossenen und zur Kontrolle markierten Aufgaben sowie eine Zusammenfassung der ausgelösten Filterregeln.

Ein vollständiges Beispiel für die Darstellung einer Aufgabe aus dem finalen Aufgabenpool ist in Anhang L enthalten. Dort werden das Datenbankschema, der Aufgabentext und die zugehörige Musterlösungs-Anfrage in der für eine SQL-Übungsaufgabe vorgesehenen Form dargestellt.

5. Evaluierung

Von den 5 600 gestarteten Pipeline-Durchläufen erreichen 5 059 Musterlösungs-Anfragen nach Abschluss der ersten Phase den Full Score und werden in die zweite Phase übernommen. Für jede dieser Anfragen wird ein Aufgabentext erzeugt und ausgewählt, sodass 5 059 Paare aus Musterlösungs-Anfrage und Aufgabentext in die Rekonstruktionsprüfung eingehen. Nach dieser Prüfung und der Anwendung des zusätzlichen Qualitätsfilters verbleiben 4 109 SQL-Übungsaufgaben im finalen Aufgabenpool. Dies entspricht 81,22 % der in die Rekonstruktionsprüfung eingegangenen Paare und 73,38 % aller gestarteten Pipeline-Durchläufe. Der letztgenannte Wert beschreibt die quantitative Ausbeute der gesamten Pipeline und ist nicht als Genauigkeits- oder Fehlerfreiheitsmaß zu verstehen. Von den finalen Aufgaben sind 76 für eine zusätzliche fachliche Kontrolle markiert.

Ausgehend von diesem Gesamtergebnis werden die in den beiden Pipelinephasen beobachteten Veränderungen der Datenbestände untersucht. Für die Musterlösungs-Anfragen betrifft dies die Schemaübergabe, die eingesetzten Sprachmodelle, die Promptvarianten, die scorebasierte Kandidatenauswahl und die Reparatur fehlerhafter Anfragen. In der zweiten Phase werden die Erzeugung und semantische Auswahl der Aufgabentexte, die Rekonstruktionsprüfung und der zusätzliche Qualitätsfilter ausgewertet. Abschließend werden die Aussagekraft des Ergebnisses und die Grenzen der automatischen Prüfung diskutiert.

5.1. Gesamtergebnis und Evaluationsgrundlage

Ausgangspunkt der Evaluation sind die 140 Datenbanken des Trainingssplits von Spider 1 [3]. Für jede Datenbank und jede der acht SQL-Schwierigkeitskategorien werden mit den vier Promptvarianten vier SQL-Anfrage-Kandidaten erzeugt. Diese Kombination aus einer Datenbank, einer SQL-Schwierigkeitskategorie und den vier zugehörigen SQL-Anfrage-Kandidaten wird im Folgenden als Durchlauf bezeichnet.

Um für jede Kombination mehrere voneinander unabhängige Ergebnisse zu erhalten, wird dieser Vorgang fünfmal durchgeführt. Aus 140 Datenbanken, acht SQL-

Schwierigkeitskategorien und fünf Wiederholungen ergeben sich somit

$$140 \cdot 8 \cdot 5 = 5\,600$$

Durchläufe. Da jeder Durchlauf vier SQL-Anfrage-Kandidaten umfasst, enthält die initiale Datenmenge insgesamt

$$5\,600 \cdot 4 = 22\,400$$

Kandidaten.

Innerhalb jedes Durchlaufs werden die vier Kandidaten mit denselben Prüfschritten analysiert und anhand ihrer Bewertung miteinander verglichen. Für die weitere Verarbeitung wird jeweils der bestbewertete Kandidat ausgewählt. Dadurch reduziert sich die Datenmenge von 22 400 Kandidaten auf 5 600 ausgewählte SQL-Anfragen. Anfragen, die zu diesem Zeitpunkt noch nicht den Full Score erreichen, werden im Reparaturschritt überarbeitet und erneut analysiert. Die Ergebnisse der erfolgreichsten Reparaturvariante werden anschließend mit den Anfragen zusammengeführt, die bereits bei der initialen Analyse den Full Score erreicht haben. Daraus entstehen die Musterlösungs-Anfragen für die Erzeugung der Aufgabentexte.

Abbildung 2 stellt den quantitativen Verlauf der Evaluation von den 140 Spider-Datenbanken bis zum finalen Aufgabenpool dar. Sie zeigt die Veränderung der jeweils in die nächste Verarbeitungsstufe übernommenen Datenmenge.

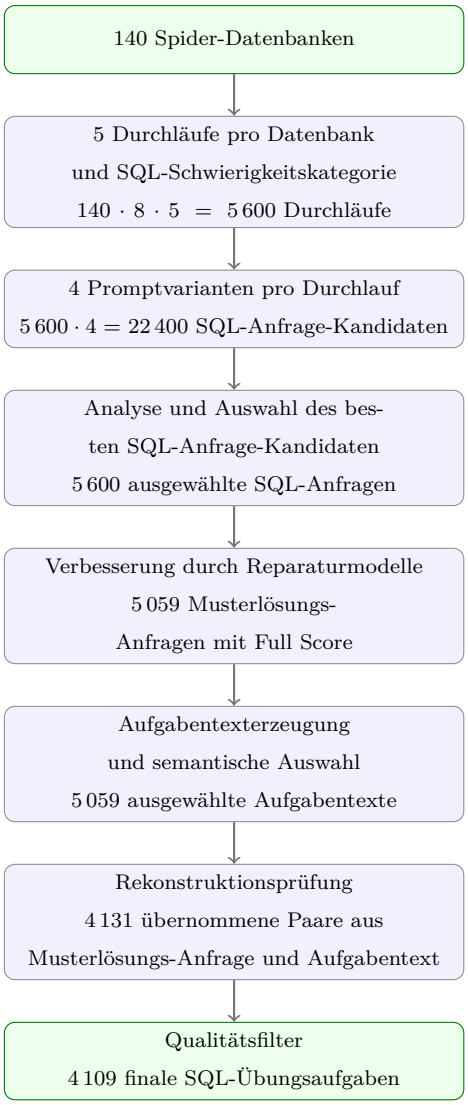


Abbildung 2: Chronologischer Mengenfluss der Evaluation von den Spider-Datenbanken bis zum finalen Aufgabenpool.

Eine Anfrage erreicht in der Evaluation nur dann den Full Score, wenn sie alle vorgesehenen Prüfschritte besteht und damit die maximale Punktzahl erreicht. Die Berechnung dieses Scores ist in Abschnitt 4.5 beschrieben. Die folgenden Abschnitte werten die Pipeline daher in derselben Reihenfolge aus, in der die Daten entstehen und weitergegeben werden.

5.2. Evaluation der Musterlösungs-Anfragen

Die erste Pipelinephase bewertet die initial erzeugten SQL-Anfragen. Auf dieser Grundlage wird bestimmt, welche Anfragen als Musterlösungs-Anfragen für die Aufgabener-

zeugung verwendet werden können. Zunächst werden die Entscheidungen zur Schemaübergabe sowie zur Wahl des Sprachmodells und der Promptvarianten untersucht. Anschließend wird ausgewertet, wie sich die Full-Score-Anteile nach der scorebasierten Kandidatenauswahl und nach dem Reparaturschritt verändern.

5.2.1. Promptentwicklung für die initiale SQL-Generierung

Für die initiale SQL-Generierung wurden zwei Formen der Schemaübergabe miteinander verglichen. In der älteren Variante wurde das Schema durch die Tabellen, die zugehörigen Spalten und deren Datentypen beschrieben. Die überarbeitete Variante ersetzt die Datentypangaben durch bis zu drei konkrete Beispielwerte je Spalte. Tabellenbeziehungen sowie Primär- und Fremdschlüssel werden in beiden Varianten separat übergeben. Die Umsetzung der überarbeiteten Schemaaufbereitung ist in Abschnitt 4.1 beschrieben.

Für beide Varianten wurden mit ArcticSQL jeweils 5 600 Durchläufe ausgewertet. Abbildung 3 vergleicht den jeweils nach der Kandidatenauswahl erreichten Full-Score-Anteil. Bei der Schemaübergabe mit Datentypen erreichen 4 596 der 5 600 ausgewählten Anfragen den Full Score. Bei der Übergabe von Beispielwerten steigt diese Zahl auf 4 817 Anfragen. Dies entspricht einem Anstieg des Full-Score-Anteils von 82,07 % auf 86,02 % beziehungsweise um 3,95 Prozentpunkte.

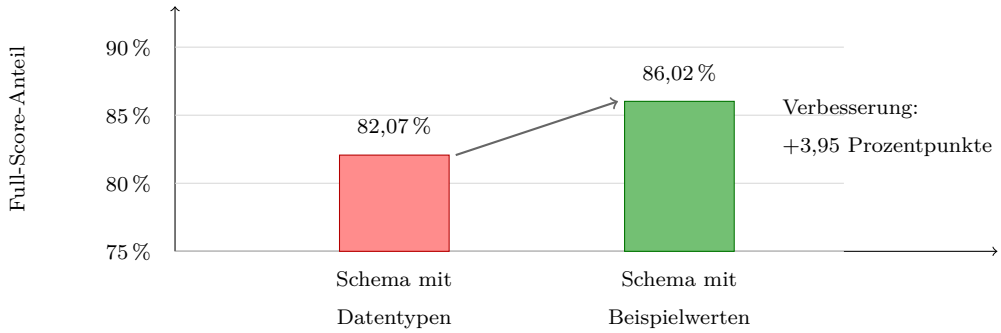


Abbildung 3: Full-Score-Anteil der initialen SQL-Generierung bei der Schemaübergabe mit Datentypen beziehungsweise Beispielwerten.

Unter den untersuchten Bedingungen erzielt die Schemaübergabe mit Beispielwerten den höheren Full-Score-Anteil. Die konkreten Werte liefern dem Modell Hinweise auf

die mögliche Bedeutung und Ausprägung der Spalten, die aus einer reinen Typangabe nicht unmittelbar hervorgehen. Auf Grundlage dieses Ergebnisses wird für die weiteren Versuche die Schemaübergabe mit Beispielwerten verwendet. Aus dem Vergleich lässt sich jedoch nicht ableiten, dass diese Darstellungsform unabhängig vom eingesetzten Modell und Datenbankschema grundsätzlich überlegen ist.

5.2.2. Vergleich verschiedener Modelle in der initialen Generierung

Neben der Form der Schemaübergabe wurde untersucht, welches der vier betrachteten Sprachmodelle für die initiale Erzeugung der SQL-Anfragen am besten geeignet ist. Verglichen werden Llama 8B aus der Llama-3-Familie [44], GPT-OSS 20B [45], Meta Llama 4 Scout [46] und ArcticSQL [42]. Für jedes Modell werden 5 600 Durchläufe mit demselben Bewertungsverfahren ausgewertet. Maßgeblich ist jeweils der Full-Score-Anteil nach der Auswahl des bestbewerteten SQL-Anfrage-Kandidaten eines Durchlaufs. Dadurch werden die Modelle nicht anhand einzelner Antworten, sondern anhand des Ergebnisses nach SQL-Analyse und Kandidatenauswahl verglichen. Abbildung 4 stellt die dabei erreichten Full-Score-Anteile gegenüber.

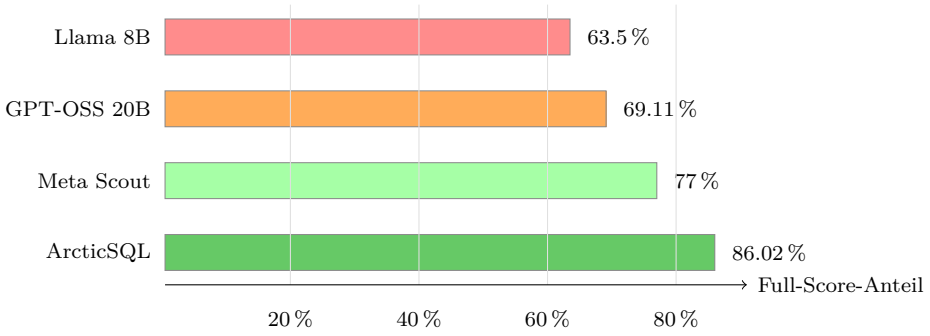


Abbildung 4: Vergleich der getesteten Modelle für die initiale SQL-Generierung.

Llama 8B erreicht in 3 556 von 5 600 Durchläufen den Full Score und damit einen Full-Score-Anteil von 63,50 %. GPT-OSS 20B steigert diesen Wert auf 3 870 Durchläufe beziehungsweise 69,11 %. Meta Scout erreicht in 4 312 Durchläufen den Full Score und damit 77,00 %. Das beste Ergebnis liefert ArcticSQL mit 4 817 Anfragen mit Full Score beziehungsweise einem Full-Score-Anteil von 86,02 %.

Unter den untersuchten Bedingungen unterscheiden sich die Full-Score-Anteile der vier Sprachmodelle deutlich. ArcticSQL [42] wurde für Text-to-SQL-Aufgaben entwickelt, wird in diesem Schritt jedoch nicht zur Übersetzung eines vorgegebenen Aufgabentextes verwendet. Stattdessen erzeugt das Modell auf Grundlage des Datenbankschemas und einer SQL-Schwierigkeitskategorie eine neue SQL-Anfrage. Das Ergebnis deutet darauf hin, dass seine SQL-spezifischen Fähigkeiten auch für diese Form der SQL-Generierung von Vorteil sind. Da ArcticSQL mit 86,02 % den höchsten Full-Score-Anteil der verglichenen Modelle erreicht, wird es für die nachfolgenden Auswertungen der initialen SQL-Generierung verwendet.

5.2.3. Initiale SQL-Generierung und Promptauswahl mit ArcticSQL

Nach der Auswahl von ArcticSQL wird untersucht, wie sich die vier Promptvarianten innerhalb der 5 600 Durchläufe verhalten. Jeder Durchlauf umfasst vier SQL-Anfrage-Kandidaten, die für dieselbe Datenbank und SQL-Schwierigkeitskategorie mit den vier Promptvarianten erzeugt wurden. Damit liegen insgesamt 22 400 initial erzeugte Kandidaten vor.

Die Auswertung beantwortet zwei Fragen. Zunächst wird untersucht, wie stark der Full-Score-Anteil zwischen den Promptvarianten und SQL-Schwierigkeitskategorien variiert. Anschließend wird der Full-Score-Anteil der in Abschnitt 4.6 beschriebenen Auswahl des bestbewerteten Kandidaten eines Durchlaufs dem Ergebnis einer festen Promptvariante gegenübergestellt. Die Kurzbezeichnungen der SQL-Schwierigkeitskategorien und die jeweils geforderten SQL-Konstrukte sind in Abschnitt 3.2.2 definiert.

Die in Abschnitt 3.2.2 konzeptionell eingeführten vier Promptvarianten verfolgen unterschiedliche Generierungsstrategien. *Prompt 1* ist ein direkter Schema-Prompt, *Prompt 2* zerlegt die Aufgabe nach dem Divide-and-Conquer-Prinzip, *Prompt 3* verwendet einen mehrstufigen Planungsdialog und *Prompt 4* ergänzt die Eingabe um Few-Shot-Beispiele. Während sich der zuvor angegebene Full-Score-Anteil von 86,02 % auf die 5 600 nach der Kandidatenauswahl vorliegenden Anfragen bezieht, betrachtet die folgende Auswertung zunächst alle 22 400 einzeln erzeugten Rohkandidaten. Von diesen erreichen 12 907 den Full Score, was einem Anteil von 57,62 % entspricht. Abbildung 5 schlüsselt diesen Anteil nach Promptvariante und SQL-Schwierigkeitskategorie auf.

	diff1	*diff2*	*diff3*	*diff4*	*diff5*	*diff6*	*diff7*	*diff8*
Prompt 1	75.3 %	70.6 %	38.6 %	71 %	58.6 %	44.7 %	58.1 %	66.7 %
Prompt 2	76.3 %	67.6 %	36 %	68.4 %	50.1 %	26.1 %	39.6 %	76.6 %
Prompt 3	75.3 %	61.1 %	30.7 %	73.1 %	37 %	20 %	32.4 %	60.3 %
Prompt 4	69 %	81.6 %	39.6 %	88 %	75.1 %	49.1 %	50.9 %	76.3 %

Abbildung 5: Full-Score-Anteile der initial erzeugten ArcticSQL-Anfrage-Kandidaten nach Promptvariante und SQL-Schwierigkeitskategorie.

Die Heatmap zeigt, dass die Full-Score-Anteile sowohl zwischen den Promptvarianten als auch zwischen den SQL-Schwierigkeitskategorien variieren. Der Few-Shot-Ansatz *Prompt 4* erzielt vergleichsweise hohe Werte bei *diff2*, das Joins ohne Aggregation verlangt, bei *diff4* mit Aggregation ohne `GROUP BY` sowie bei *diff5* mit Aggregation und `GROUP BY`. Deutlich niedrigere Werte treten bei *diff3* mit `NOT EXISTS` oder `NOT IN` und bei *diff6* mit `GROUP BY` und `HAVING` auf. Die Ergebnisse sprechen somit gegen die Annahme, dass eine Promptvariante bei allen geforderten SQL-Konstrukten gleichermaßen wirksam ist.

Abbildung 6 fasst die Ergebnisse über alle acht SQL-Schwierigkeitskategorien zusammen. Sie beantwortet die Frage, welche Promptvariante bei einer einheitlichen Verwendung für alle Durchläufe den höchsten Full-Score-Anteil erzielen würde.

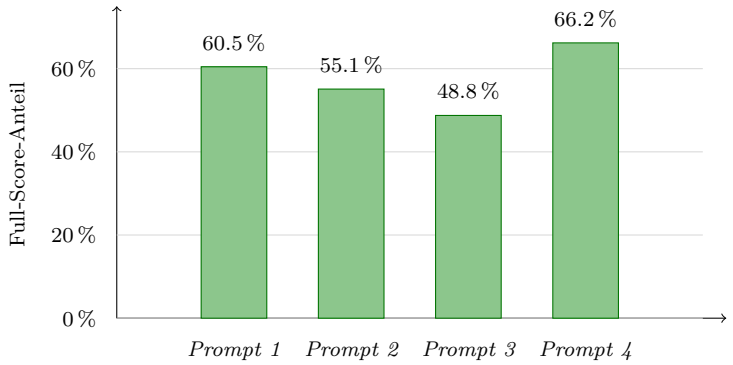


Abbildung 6: Full-Score-Anteil der initial erzeugten ArcticSQL-Anfrage-Kandidaten nach Promptvariante.

Prompt 4, die Few-Shot-Variante, erreicht mit 66,20 % den höchsten aggregierten Full-Score-Anteil. Es folgen *Prompt 1* mit 60,45 %, *Prompt 2* mit 55,09 % und *Prompt 3*

mit 48,75 %. Bei einer Festlegung auf eine einzelne Promptvariante wäre *Prompt 4* damit die beste der untersuchten Optionen. Dieser aggregierte Vergleich berücksichtigt jedoch nicht, dass innerhalb eines Durchlaufs auch eine andere Promptvariante den bestbewerteten Kandidaten erzeugen kann.

Zur Bewertung der scorebasierten Kandidatenauswahl wird ihr Ergebnis deshalb mit *Prompt 4* als bester fester Promptvariante verglichen. Beide Werte beziehen sich auf dieselben 5 600 Durchläufe. Abbildung 7 stellt diesen Vergleich dar.

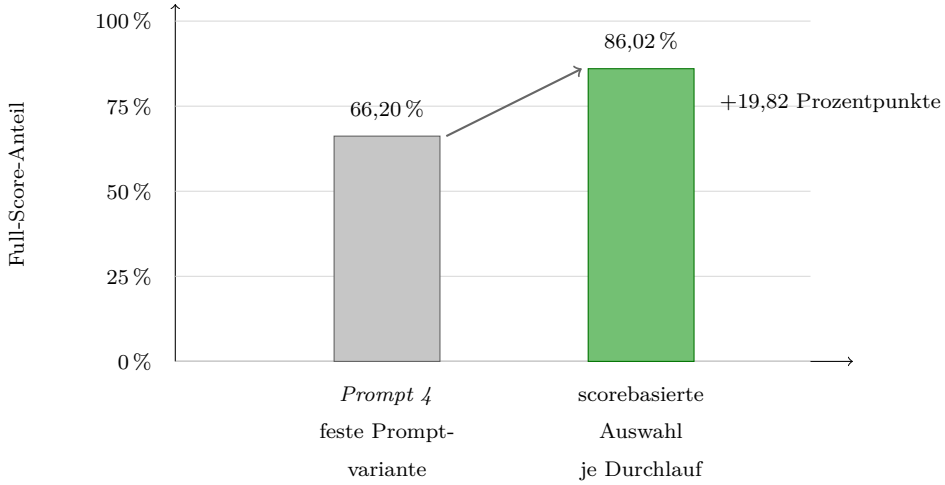


Abbildung 7: Full-Score-Anteil bei Verwendung der besten festen Promptvariante und bei scorebasierter Auswahl des besten Kandidaten je Durchlauf.

Bei ausschließlicher Verwendung von *Prompt 4* erreichen 66,20 % der Anfragen den Full Score. Durch die scorebasierte Auswahl des besten Kandidaten aus allen vier Promptvarianten steigt dieser Anteil auf 86,02 %. Die Differenz beträgt 19,82 Prozentpunkte. In 4 817 der 5 600 Durchläufe liegt somit mindestens ein Kandidat mit Full Score vor, der aufgrund seiner Bewertung ausgewählt wird. Die Auswahl verbessert keine bereits erzeugte Anfrage, sondern nutzt die unterschiedlichen Ergebnisse der vier Promptvarianten und entfernt die geringer bewerteten Alternativen eines Durchlaufs.

Der Nutzen dieses Vorgehens wird bei *diff3* und *diff6* besonders deutlich. In beiden Kategorien erzielt *Prompt 4* unter den einzelnen Promptvarianten den höchsten Full-Score-Anteil. Abbildung 8 vergleicht diese feste Promptvariante mit der scorebasierten Auswahl.

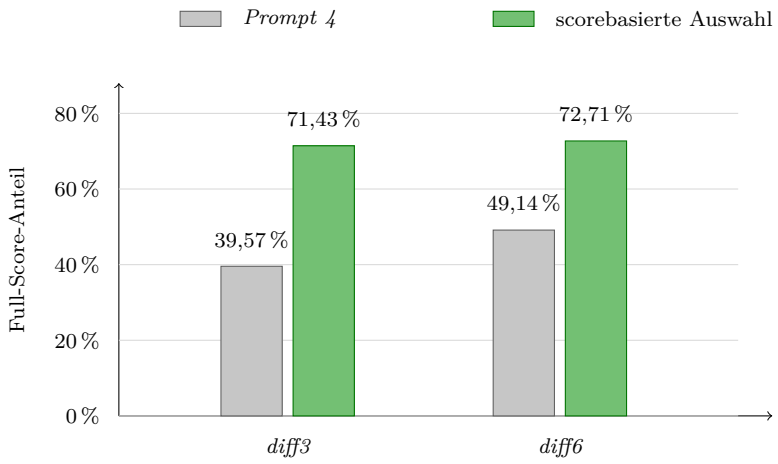


Abbildung 8: Full-Score-Anteile von *Prompt 4* und der scorebasierten Kandidatenauswahl bei *diff3* und *diff6*.

Bei *diff3* steigt der Full-Score-Anteil durch die Auswahl von 39,57 % auf 71,43 % und damit um 31,86 Prozentpunkte. Bei *diff6* erhöht er sich von 49,14 % auf 72,71 %, was einer Differenz von 23,57 Prozentpunkten entspricht. In diesen zusätzlichen Durchläufen scheitert *Prompt 4*, während mindestens eine der anderen Promptvarianten einen Kandidaten mit Full Score erzeugt. Die erfolgreichen Kandidaten verteilen sich folglich auf unterschiedliche Promptvarianten. Für die weitere Pipeline wird deshalb kein fester Prompt gewählt, sondern in jedem Durchlauf der Kandidat mit dem höchsten Score übernommen.

5.2.4. Qualität nach SQL-Schwierigkeitskategorie

Der Full-Score-Anteil von 86,02 % ist ein Gesamtwert über alle SQL-Schwierigkeitskategorien. Abbildung 9 schlüsselt diesen Wert für die jeweils 700 Durchläufe pro Kategorie auf.

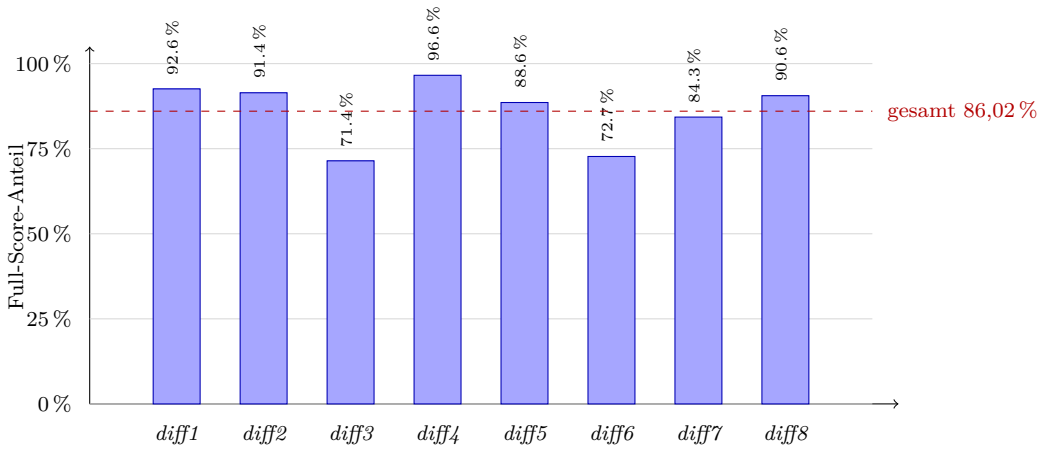


Abbildung 9: Full-Score-Anteil der ausgewählten SQL-Anfragen nach SQL-Schwierigkeitskategorie.

Die Full-Score-Anteile unterscheiden sich deutlich zwischen den Kategorien. Den höchsten Wert erreicht *diff4*, das Aggregationen ohne `GROUP BY` verlangt, mit 96,57 %. Ebenfalls oberhalb des Gesamtwerts von 86,02 % liegen *diff1*, *diff2*, *diff5* und *diff8*. Unterhalb des Gesamtwerts liegen *diff7* mit 84,29 %, *diff6* mit 72,71 % und *diff3* mit 71,43 %.

Die Bezeichnungen *diff1* bis *diff8* unterscheiden die geforderten SQL-Konstrukte und stellen keine aufsteigende Rangfolge dar. Zwischen dem höchsten und dem niedrigsten Full-Score-Anteil liegt eine Differenz von 25,14 Prozentpunkten. Die Kandidatenauswahl gleicht die Qualitätsunterschiede zwischen den Kategorien somit nicht vollständig aus.

Von den insgesamt 783 Durchläufen ohne Full Score entfallen 200 auf *diff3* und 191 auf *diff6*. Auf beide Kategorien zusammen entfallen damit 391 beziehungsweise 49,94 % der nach der Auswahl verbleibenden Fälle ohne Full Score. Für den anschließenden Reparaturschritt sind sie daher von besonderer Bedeutung.

5.2.5. Fehleranalyse vor und nach der Auswahl des besten SQL-Anfrage-Kandidaten

Die bisherige Auswertung betrachtet die Qualität anhand des zusammengefassten Scores. Im Folgenden wird untersucht, welche einzelnen Prüfschritte von den initial er-

zeugten Anfragen nicht bestanden werden und welche dieser Probleme nach der Kandidatenauswahl weiterhin auftreten.

Dazu werden zunächst alle 22 400 initial erzeugten ArcticSQL-Kandidaten ausgewertet. Anschließend erfolgt dieselbe Auswertung für die 5 600 bestbewerteten Kandidaten der einzelnen Durchläufe. Die Auswahl korrigiert dabei keine Anfrage. Unterschiede zwischen beiden Datenmengen entstehen dadurch, dass geringer bewertete Kandidaten nicht weiterverwendet werden.

Abbildung 10 zeigt für jeden Prüfschritt den Anteil der bestandenen Tests. Bei den initialen Kandidaten werden Parsing und Validierung jeweils von 91,86 % bestanden. Die niedrigsten Erfolgsanteile weisen das Linting mit 67,96 %, die Ergebnisvorschau mit 82,33 % und die Fremdschlüsselprüfung mit 85,01 % auf. Nach der Auswahl bestehen alle 5 600 übernommenen Anfragen das Parsing und die Validierung. 5 597 Anfragen sind auf der jeweiligen Datenbank ausführbar.

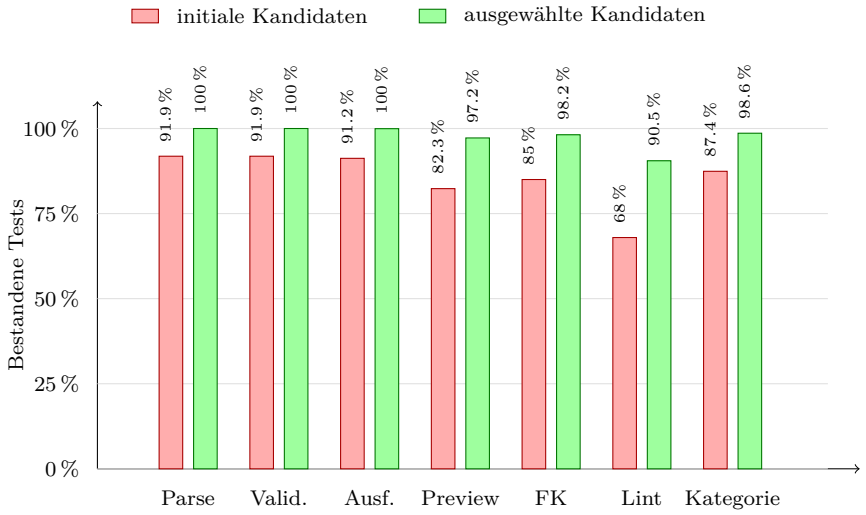


Abbildung 10: Anteil bestandener Prüfschritte vor und nach der Auswahl des besten SQL-Anfrage-Kandidaten.

Das Linting bleibt auch nach der Kandidatenauswahl der Prüfschritt mit dem niedrigsten Erfolgsanteil. Deshalb werden die dabei am häufigsten auftretenden Fehlercodes gesondert untersucht. Die BGE-Codes bezeichnen Fehlerklassen des verwendeten SQLlint-Regelwerks. Diese greifen semantisch auffällige SQL-Muster auf, wie sie von Brass und Goldberg systematisiert wurden [13]. Die Einbindung des Regelwerks und die

Bedeutung ausgewählter BGE-Codes sind in Abschnitt 4.4.4 beschrieben. Die übrigen Codes bezeichnen technische Fehler, die während der Verarbeitung im Linting-Schritt auftreten.

Vor der Auswahl enthalten 7 176 der 22 400 initialen Kandidaten mindestens eine Lint-Meldung. Dies entspricht 32,04 %. Nach der Auswahl betrifft dies noch 531 der 5 600 übernommenen Anfragen beziehungsweise 9,48 %. Tabelle 2 erläutert die häufigsten Codes und stellt ihre absoluten Häufigkeiten sowie ihre Anteile an der jeweiligen Datenmenge gegenüber.

Fehlercode	**Bedeutung**	**Initial**	**Ausgewählt**
`SEM_PARSING_ERR`	Anfrage konnte durch die semantische Analyse nicht verarbeitet werden.	2390 (10,67 %)	115 (2,05 %)
`OTHER_PARSER`	Sonstiges Parserproblem innerhalb des Linting-Schrittes.	1570 (7,01 %)	161 (2,88 %)
`BGE_17`	Unnötiges Argument in einer `COUNT`-Aggregation.	1432 (6,39 %)	142 (2,54 %)
`BGE_27`	Fehlende oder durch SQLlint nicht erkannte Join-Bedingung.	581 (2,59 %)	39 (0,70 %)
`BGE_31`	Vergleich von Werten unterschiedlicher Domänen.	500 (2,23 %)	28 (0,50 %)
`BGE_10`	Problematische Verwendung von `NULL` in einer Unteranfrage.	313 (1,40 %)	26 (0,46 %)
`INCORRECT_SQL`	Eingabe konnte im Linting-Schritt nicht als SQL-Anfrage verarbeitet werden.	221 (0,99 %)	0 (0,00 %)
`OTHER_LINT`	Sonstiger technischer Fehler während des Linting-Schrittes.	81 (0,36 %)	7 (0,13 %)

Tabelle 2: Bedeutung und Häufigkeit der am häufigsten auftretenden Linting-Fehlercodes vor und nach der Kandidatenauswahl. Die Anteile beziehen sich auf 22 400 initiale beziehungsweise 5 600 ausgewählte Kandidaten.

Eine Anfrage kann mehrere Lint-Meldungen enthalten. Außerdem beschränkt sich die Tabelle auf die am häufigsten auftretenden Codes. Die aufgeführten Häufigkeiten addieren sich daher nicht zur Gesamtzahl der vom Linting betroffenen Anfragen.

Nach der Kandidatenauswahl treten sämtliche dargestellten Linting-Fehlercodes in der weiterverwendeten Datenmenge anteilig seltener auf. Besonders deutlich sinken die An-

teile von `SEM_PARSING_ERR`, `BGE_17` und `OTHER_PARSER`. Die Auswahl korrigiert diese Fehler nicht, bevorzugt jedoch innerhalb eines Durchlaufs Kandidaten mit besseren Analyseergebnissen.

Mit 531 betroffenen Anfragen verbleibt das Linting dennoch als häufigste Fehlerquelle unter den sieben Prüfschritten. Die zugehörigen Meldungen bilden deshalb einen wesentlichen Bestandteil des Fehlerfeedbacks für den anschließenden Reparaturschritt.

5.2.6. Evaluation der Reparaturmodelle

Die Implementierung des Fine-Tunings und des anschließenden Reparaturschritts ist in den Abschnitten 4.8 und 4.9 beschrieben. Nach der Kandidatenauswahl liegen 5 600 SQL-Anfragen vor. Davon erreichen 4 817 bereits den Full Score und werden unverändert übernommen. Die verbleibenden 783 Anfragen werden von den jeweiligen Reparaturmodellen überarbeitet und erneut analysiert. Anschließend werden sämtliche neu bewerteten Reparaturergebnisse mit den 4 817 ursprünglichen Full-Score-Anfragen zusammengeführt.

Die Reparaturmodelle basieren auf der Qwen2.5-Coder- beziehungsweise Qwen3-Modellfamilie [47, 48]. Sie werden anhand des Full-Score-Anteils in der jeweils zusammengeführten Datenmenge von 5 600 Anfragen verglichen. Als Referenz dient der Stand unmittelbar nach der ArcticSQL-Kandidatenauswahl mit einem Full-Score-Anteil von 86,02 %. Abbildung 11 stellt diesen Referenzwert den Ergebnissen der untersuchten Reparaturvarianten gegenüber.

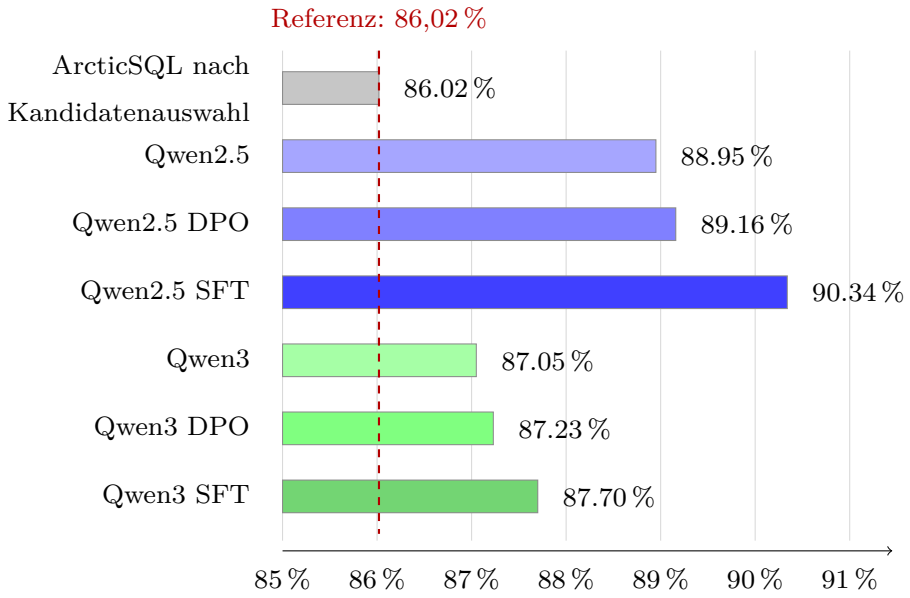


Abbildung 11: Full-Score-Anteile nach der Reparatur und Zusammenführung mit den bereits zuvor erfolgreichen ArcticSQL-Anfragen. Die Skala beginnt bei 85 %, um die Unterschiede zwischen den Modellvarianten sichtbar zu machen.

Alle untersuchten Reparaturvarianten erhöhen den Full-Score-Anteil gegenüber dem Referenzwert von 86,02 %. Das beste Ergebnis erreicht Qwen2.5 SFT mit 5 059 Full-Score-Anfragen beziehungsweise 90,34 %. Durch den Reparaturschritt werden damit 242 der zuvor 783 nicht erfolgreichen Durchläufe zusätzlich bis zum Full Score verbessert.

Qwen2.5 DPO erreicht 89,16 %, während das zugehörige Basismodell auf 88,95 % kommt. Bei der Qwen3-Modellfamilie werden 87,05 % ohne Fine-Tuning, 87,23 % mit DPO und 87,70 % mit SFT erreicht. SFT erzielt damit bei beiden untersuchten Modellfamilien das jeweils beste Ergebnis. Für die nachfolgenden Pipeline-Schritte wird deshalb Qwen2.5 SFT verwendet.

Neben dem Full-Score-Anteil wird untersucht, welche Fehler nach der Reparatur mit der ausgewählten Modellvariante verbleiben. Bei Qwen2.5 SFT erreichen 541 der 5 600 Durchläufe keinen Full Score. Abbildung 12 vergleicht die betroffenen Prüfschritte mit dem Stand nach der ArcticSQL-Kandidatenauswahl. Da eine Anfrage mehrere Prüfschritte verfehlen kann, addieren sich die einzelnen Fehlerzahlen nicht zur Zahl der

Durchläufe ohne Full Score.

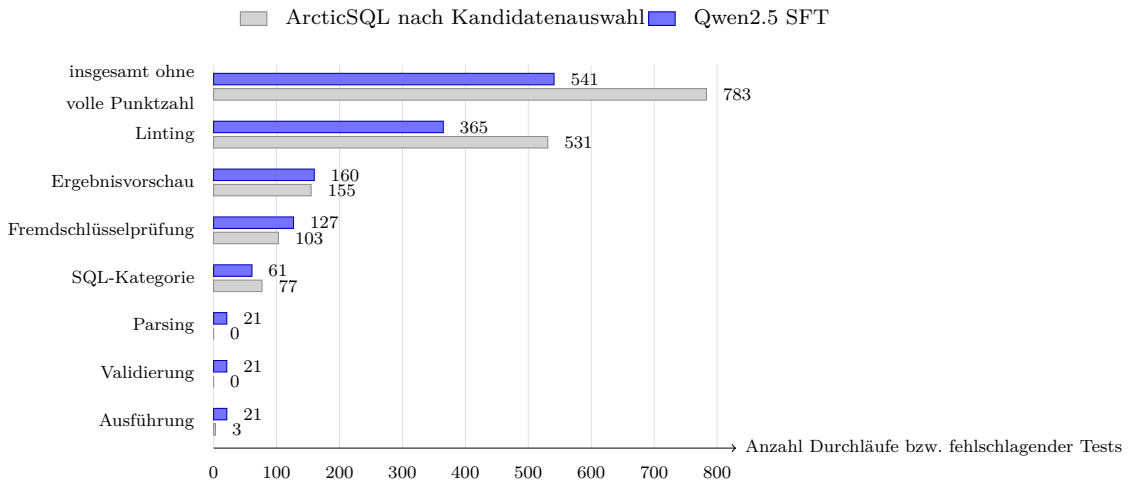


Abbildung 12: Fehlschlagende Prüfschritte nach der ArcticSQL-Kandidatenauswahl und nach der Reparatur mit Qwen2.5 SFT.

Die Zahl der Durchläufe ohne Full Score sinkt insgesamt von 783 auf 541. Der deutlichste Rückgang betrifft das Linting: Die Anzahl der betroffenen Anfragen verringert sich von 531 auf 365. Auch Abweichungen von der geforderten SQL-Schwierigkeitskategorie nehmen von 77 auf 61 ab. Gleichzeitig steigen die Fehlerzahlen bei der Ergebnisvorschau von 155 auf 160 und bei der Fremdschlüsselprüfung von 103 auf 127. Parsing, Validierung und Ausführung schlagen nach der Reparatur jeweils in 21 Fällen fehl, obwohl diese Prüfungen zuvor von nahezu allen ausgewählten Anfragen bestanden wurden.

Das Reparaturmodell korrigiert somit einen Teil der übergebenen Probleme, kann bei der Überarbeitung jedoch neue Fehler in zuvor bestandenen Prüfschritten erzeugen. Bezogen auf die 783 zur Reparatur übergebenen Anfragen führt die beste Modellvariante 242 weitere Anfragen zum Full Score. Dies entspricht 30,91 % der zuvor nicht erfolgreichen Fälle und belegt einen quantitativen Nutzen des Reparaturschritts. Für den praktischen Einsatz ist dieser Zugewinn gegen die zusätzliche Generierungszeit des Reparaturaufrufs abzuwägen.

Innerhalb der verbleibenden Lint-Fehler treten insbesondere die bereits in Tabelle 2 erläuterten Parser- und BGE-Codes auf. Die häufigsten Meldungen nach Qwen2.5 SFT sind `OTHER_PARSER` mit 160 Fällen, `SEM_PARSING_ERR` mit 107 Fällen, `BGE_27` mit 38

Fällen, `BGE_31` mit 22 Fällen und `BGE_17` mit 20 Fällen. Das Linting bleibt damit auch nach der Reparatur die häufigste Fehlerquelle.

Als empirisches Ergebnis ist festzuhalten, dass Qwen2.5 SFT unter den untersuchten Konfigurationen den höchsten Full-Score-Anteil erreicht. Die Evaluation erlaubt dagegen keine allgemeine Aussage darüber, welche Modellarchitektur grundsätzlich am besten für die Reparatur von SQL-Anfragen geeignet ist.

5.2.7. Ergebnisse der SQL-Qualitätssicherung

Die Bewertung steuert, welcher Kandidat eines Durchlaufs weiterverarbeitet wird. Bei durchgängiger Verwendung der besten festen Promptvariante erreichen 66,20 % der Durchläufe den Full Score. Die scorebasierte Auswahl greift dagegen auf vier unterschiedlich erzeugte Kandidaten desselben Durchlaufs zurück und übernimmt jeweils den Kandidaten mit dem höchsten Score. Für diese Auswahl liegt der Full-Score-Anteil bei 86,02 %. Die Differenz von 19,82 Prozentpunkten beschreibt den beobachteten Vorteil der fallbezogenen Auswahl gegenüber der besten einzelnen Promptvariante unter den untersuchten Bedingungen. Sie ist mit vier Modellaufrufen pro Durchlauf verbunden und isoliert daher nicht allein die Wirkung der Score-Berechnung von der zusätzlichen Kandidatenzahl.

Nach der anschließenden Reparatur steigt der Full-Score-Anteil von 86,02 % auf 90,34 %. Damit stehen 5 059 Musterlösungs-Anfragen für die zweite Pipelinephase zur Verfügung. Auch diese Veränderung beschreibt das Ergebnis des vollständigen Reparatur- und Analysepfads. Sie erlaubt keine getrennte kausale Zuordnung zu einzelnen Analyseinformationen oder Trainingsentscheidungen.

An die Erzeugung der Aufgabentexte werden ausschließlich Anfragen übergeben, die den Full Score erreichen und somit alle vorgesehenen Prüfschritte bestanden haben. Die Full-Score-Filterung bildet damit die festgelegte Qualitätsschwelle zwischen der Erzeugung der Musterlösungs-Anfragen und der Erzeugung der vollständigen Aufgaben. Sie verhindert, dass bereits durch die Prüfschritte erkannte Fehler in die nachfolgende Aufgabentexterzeugung übernommen werden. Unerkannte Fehler können dennoch bestehen bleiben. Diese Grenze wird in Abschnitt 5.4 diskutiert.

Die Ergebnisse zeigen damit, dass die kombinierte Prüf-, Auswahl- und Reparaturstrategie einen deutlich größeren Bestand an Anfragen mit Full Score bereitstellt als die Verwendung einer einzelnen festen Promptvariante. Welchen Anteil einzelne Prüfschritte kausal an der Qualität der finalen Aufgaben haben, lässt sich aus den vorliegenden Daten hingegen nicht bestimmen.

5.3. Evaluation der vollständigen SQL-Übungsaufgaben

Die zweite Pipelinephase ergänzt die ausgewählten Musterlösungs-Anfragen um natürlichsprachliche Aufgabentexte und prüft die daraus entstehenden vollständigen Aufgaben. Bewertet werden zunächst die Erzeugung und semantische Auswahl der Aufgabentexte und anschließend die Rekonstruktionsprüfung sowie der zusätzliche Qualitätsfilter.

5.3.1. Erzeugung und Auswahl der Aufgabentexte

Aus den 5 059 mit Full Score bewerteten Musterlösungs-Anfragen der Qwen2.5-SFT-Variante werden natürlichsprachliche Aufgabentexte erzeugt. Für jeden Datensatz werden fünf unterschiedliche Aufgabentext-Kandidaten angestrebt. Das Sprachmodell erzeugt jeden Kandidaten in einem separaten Aufruf. Für jede noch unbesetzte Kandidatenposition sind höchstens drei Versuche vorgesehen. Leere Antworten und bereits erzeugte Formulierungen werden nicht übernommen. Daher liegen in der Praxis nicht für jeden Datensatz fünf unterschiedliche Kandidaten vor. Für die Erzeugung wurde das Modell `Qwen3-Coder-30B-A3B-Instruct` eingesetzt.

Abbildung 13 zeigt die Verteilung der erzeugten Kandidatenanzahl.

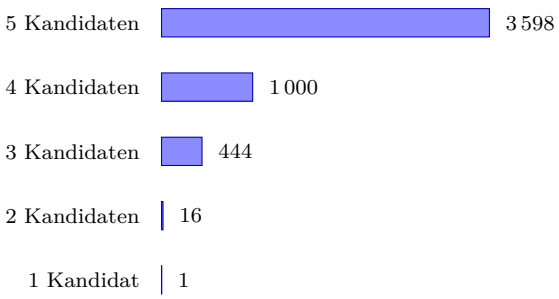


Abbildung 13: Anzahl erzeugter Aufgabentext-Kandidaten pro Musterlösungs-Anfrage.

Für die in Abschnitt 4.12 beschriebene Auswahl wird das Embedding-Modell `sentence-transformers/all-mpnet-base-v2` verwendet [41]. Bei genau zwei Kandidaten kann das Ähnlichkeitskriterium keine Präferenz bestimmen. Daher kommt die pseudozufällige Ersatzregel zum Einsatz.

In 3 598 Datensätzen (71,12 %) werden die angestrebten fünf Aufgabentext-Kandidaten erreicht. In 1 000 Datensätzen (19,77 %) liegen vier und in 444 Datensätzen (8,78 %) drei Kandidaten vor. Damit kann das semantische Auswahlverfahren bei 5 042 Datensätzen (99,66 %) regulär angewendet werden. Nur bei 16 Datensätzen (0,32 %) mit genau zwei Kandidaten kommt diese Ersatzregel zum Einsatz. In einem Datensatz (0,02 %) wird der einzige vorhandene Kandidat unmittelbar übernommen.

Die Ergebnisse zeigen, dass die Aufgabentexterzeugung nahezu immer genügend unterschiedliche Kandidaten für die vorgesehene semantische Auswahl bereitstellt. Die Ersatzregeln sind lediglich für 17 der 5 059 Datensätze (0,34 %) erforderlich, sodass für jede Musterlösungs-Anfrage ein Aufgabentext ausgewählt werden kann. Das Auswahlkriterium bewertet dabei die semantische Zentralität eines Kandidaten innerhalb der jeweiligen Kandidatenmenge: Bevorzugt wird die Formulierung mit der höchsten mittleren Kosinusähnlichkeit zu den übrigen Formulierungen. Es trifft hingegen keine unmittelbare Aussage über die fachliche Korrektheit oder die vollständige inhaltliche Übereinstimmung mit der Musterlösungs-Anfrage. Diese Eigenschaften werden durch die nachgelagerte Rekonstruktionsprüfung und den zusätzlichen Qualitätsfilter untersucht.

5.3.2. Rekonstruktionsprüfung und Qualitätsfilter

Die Implementierung der Rekonstruktionsprüfung und des Qualitätsfilters ist in den Abschnitten 4.13 und 4.14 beschrieben. Ausgangspunkt der beiden abschließenden Prüfschritte sind 5 059 Paare aus jeweils einer mit Full Score bewerteten Musterlösungs-Anfrage und einem ausgewählten Aufgabentext. Abbildung 14 zeigt, wie sich diese Datenmenge durch die Rekonstruktionsprüfung und den zusätzlichen Qualitätsfilter verändert. Die zur fachlichen Kontrolle markierten Aufgaben werden dabei gesondert ausgewiesen, da sie im finalen Aufgabenpool verbleiben.

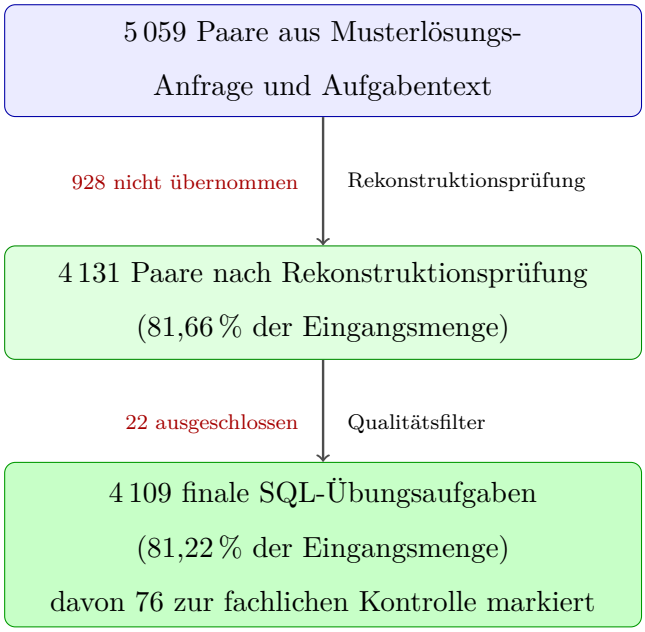


Abbildung 14: Veränderung der Datenmenge durch Rekonstruktionsprüfung und Qualitätsfilter.

Von den 5 059 Paaren aus Musterlösungs-Anfrage und Aufgabentext bestehen 4 131 die Rekonstruktionsprüfung. Dies entspricht 81,66 % der Eingangsmenge. Insgesamt werden in diesem Schritt 928 Paare (18,34 %) nicht übernommen. Den mit Abstand größten Anteil bilden 851 Fälle (16,82 %) mit abweichenden Ausführungsergebnissen. Weitere 67 Paare (1,32 %) scheitern aufgrund eines Ausführungsfehlers. Je fünf Fälle (jeweils 0,10 %) werden wegen eines leeren Ergebnisses beziehungsweise mangels eines geeigneten Datenbankzustands nicht weiterverwendet.

Die Verteilung zeigt, dass technische Ausführungsfehler nur einen vergleichsweise kleinen Teil der ausgeschlossenen Paare erklären. Ausschlaggebend sind überwiegend un-

terschiedliche Ergebnisse der Musterlösungs-Anfrage und der rekonstruierten SQL-Anfrage. Für diese Paare kann auf dem verwendeten Datenbankzustand keine Ergebnisäquivalenz bestätigt werden. Die Abweichung lässt sich jedoch nicht ausschließlich dem erzeugten Aufgabentext zuschreiben, da auch das für die Rekonstruktion eingesetzte Sprachmodell eine ungeeignete Anfrage erzeugen kann. Der Anteil von 18,34 % bewertet somit den gesamten Rekonstruktions- und Vergleichsschritt und ist nicht als unmittelbare Fehlerquote der erzeugten Aufgabentexte zu interpretieren.

Der zusätzliche Qualitätsfilter führt bei 22 der 4 131 zuvor von der Rekonstruktionsprüfung übernommenen Paare (0,53 %) zu einem harten Ausschluss. Damit verbleiben 4 109 Aufgaben im finalen Pool. Weitere 76 Paare (1,84 %) erfüllen mindestens eine Review-Regel. Sie werden für eine fachliche Kontrolle markiert, aber nicht aus dem finalen Aufgabenpool entfernt. Dieser umfasst folglich 4 033 Aufgaben ohne Markierung und 76 Review-Fälle. Die geringe Anzahl harter Ausschlüsse zeigt, dass die aus der vorgelagerten Modellprüfung abgeleiteten Ausschlussregeln nur bei einem kleinen Teil der von der Rekonstruktionsprüfung übernommenen Paare greifen. Die Regeln wurden explorativ aus Auffälligkeiten desselben Datenbestands abgeleitet, auf den sie anschließend angewendet wurden. Die 22 Ausschlüsse belegen daher ihre Abdeckung innerhalb dieses Aufgabenpools, erlauben jedoch keine Aussage über Präzision, Vollständigkeit oder Übertragbarkeit auf neue Aufgaben. Dafür wäre eine unabhängig annotierte Evaluationsmenge erforderlich. Ebenso ist die geringe Zahl kein Nachweis dafür, dass der verbleibende Aufgabenbestand frei von weiteren sprachlichen oder didaktischen Auffälligkeiten ist.

Bezogen auf die 5 059 eingehenden Paare umfasst der finale Aufgabenpool 4 109 Aufgaben beziehungsweise 81,22 %. Der größte quantitative Rückgang entsteht damit während der Rekonstruktionsprüfung. Da deren Ergebnis sowohl von der Qualität des Aufgabentextes als auch von der Zuverlässigkeit des Rekonstruktionsmodells abhängt, kann die Ursache anhand dieser Auswertung nicht eindeutig einem der beiden Schritte zugeordnet werden. Insgesamt werden etwa vier von fünf Paaren aus Musterlösungs-Anfrage und Aufgabentext in den finalen Aufgabenpool übernommen.

5.4. Diskussion

Neben dem Gesamtumfang des Aufgabenpools ist zu untersuchen, an welchen Stellen die Bestände der acht SQL-Schwierigkeitskategorien zurückgehen. Da für jede Kategorie ursprünglich 700 Durchläufe vorliegen, können ihre absoluten Anzahlen unmittelbar miteinander verglichen werden. Abbildung 15 stellt den Bestand nach Phase 1, nach der Rekonstruktionsprüfung und nach dem Qualitätsfilter gegenüber. Die Definitionen der Kategorien sind in Abschnitt 3.2.2 aufgeführt.

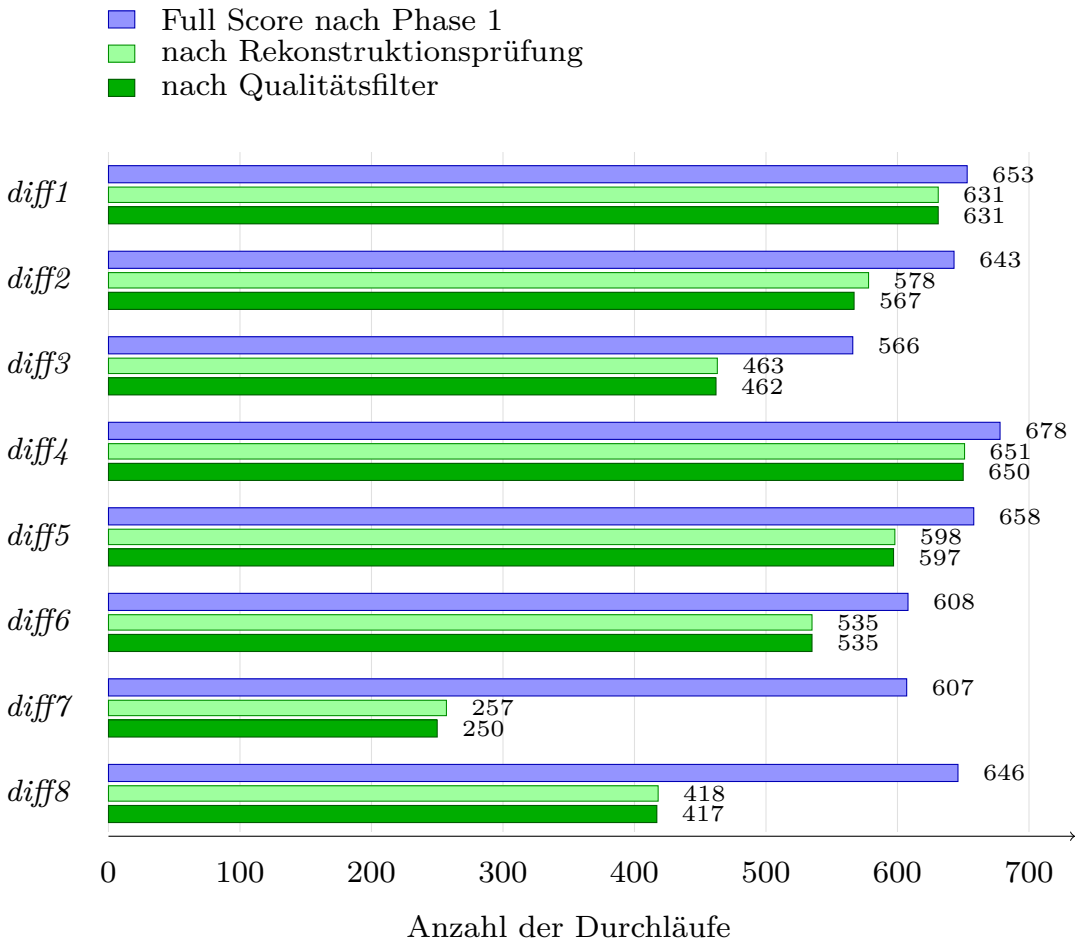


Abbildung 15: Bestand je SQL-Schwierigkeitskategorie nach Phase 1, nach der Rekonstruktionsprüfung und nach dem Qualitätsfilter.

Der finale Aufgabenpool deckt alle acht SQL-Schwierigkeitskategorien ab. Nach Phase 1 liegen die Bestände zwischen 566 Anfragen in *diff3* und 678 Anfragen in *diff4* noch vergleichsweise nah beieinander. Der wesentliche kategoriespezifische Rückgang entsteht während der Rekonstruktionsprüfung. Besonders deutlich ist dies für *diff7*:

Von 607 Musterlösungs-Anfragen mit Full Score bestehen 257 die Rekonstruktionsprüfung. Nach dem Qualitätsfilter verbleiben 250 Aufgaben. Bei *diff8* sinkt der Bestand in diesem Schritt von 646 auf 418 und anschließend auf 417 Aufgaben. Demgegenüber verändern sich die Bestände von *diff1* und *diff4* zwischen den drei dargestellten Stufen nur geringfügig.

Der zusätzliche Qualitätsfilter verändert die Verteilung mit insgesamt 22 Ausschlüssen nur noch in geringem Umfang. Die Unterschiede im finalen Aufgabenpool werden daher überwiegend durch die Rekonstruktionsprüfung geprägt und nicht durch die harten Regeln des Qualitätsfilters. Sie zeigen, dass sich die Konsistenz zwischen Aufgabentext und Musterlösungs-Anfrage für die geforderten SQL-Konstrukte unterschiedlich häufig bestätigen lässt. Da die Bezeichnungen *diff1* bis *diff8* unterschiedliche Anfrageformen und keine aufsteigende Rangfolge kennzeichnen, ist daraus keine allgemeine Abnahme der Qualität mit zunehmender Kategorienummer abzuleiten. Insgesamt erreicht die Pipeline ihr zentrales Ziel, aus Datenbankschemata automatisiert einen umfangreichen Pool vorgeprüfter SQL-Übungsaufgaben zu erzeugen. Die beobachteten Bestände verändern sich dabei entlang mehrerer Verarbeitungsschritte: Nach der Auswahl des bestbewerteten SQL-Anfrage-Kandidaten liegt der Full-Score-Anteil über dem Wert der besten festen Promptvariante. Nach dem Reparaturschritt steigt er von 86,02 % auf 90,34 %. In der zweiten Phase reduzieren die Rekonstruktionsprüfung und der zusätzliche Qualitätsfilter die Datenmenge auf solche Aufgaben, die die Rekonstruktionsprüfung bestanden haben und durch keine harte Qualitätsregel ausgeschlossen wurden.

Die automatischen Prüfungen erlauben dennoch keine Garantie für die fachliche Fehlerfreiheit der finalen Aufgaben. Eine Musterlösungs-Anfrage kann trotz Full Score einen von den Prüfverfahren nicht erfassten semantischen Fehler enthalten. Die Prüfung der SQL-Schwierigkeitskategorie stellt zudem fest, ob die geforderten SQL-Konstrukte vorhanden sind, bewertet aber nicht, ob ihre Verwendung für die jeweilige fachliche Fragestellung erforderlich oder didaktisch zweckmäßig ist. Eine formal passende Anfrage kann daher unnötig komplex sein.

Auch die Rekonstruktionsprüfung besitzt eine begrenzte Aussagekraft. Gleiche Ausführungsergebnisse auf einem konkreten Datenbankzustand belegen keine allgemeine logische Äquivalenz zweier SQL-Anfragen. Umgekehrt kann ein abweichendes Ergebnis

sowohl auf einen ungenauen Aufgabentext als auch auf eine fehlerhafte Rekonstruktion durch das eingesetzte Sprachmodell zurückgehen. Der gemessene Übernahmeanteil dieses Schrittes bewertet deshalb den vollständigen Rekonstruktions- und Vergleichspfad. Sprachliche oder didaktische Auffälligkeiten können außerdem bestehen bleiben, wenn sie weder durch die Rekonstruktionsprüfung noch durch die abgeleiteten Qualitätsregeln erfasst werden.

Aus diesen Grenzen folgt, dass der erzeugte Aufgabenpool als automatisiert vorgeprüfte Grundlage und nicht als unmittelbar freizugebender Aufgabenbestand zu verstehen ist. Vor einem Einsatz in Lehrveranstaltungen oder Prüfungen bleibt eine abschließende Kontrolle durch eine fachlich verantwortliche Person erforderlich. Die 76 durch Review-Regeln markierten Aufgaben können dabei priorisiert geprüft werden. Die Sichtung der übrigen Aufgaben bleibt dennoch notwendig. Die Pipeline folgt damit einem Human-in-the-Loop-Ansatz, bei dem die automatische Verarbeitung die fachliche Beurteilung unterstützt, aber nicht ersetzt. Dies entspricht Befunden zu KI-generierten Bildungsinhalten, deren Qualität trotz automatischer Erzeugung kritisch geprüft werden sollte [49].

Die modulare Struktur der Pipeline ermöglicht es, einzelne Verarbeitungsschritte abhängig vom jeweiligen Anwendungsszenario weiterzuentwickeln, auszutauschen oder auszulassen. Der Reparaturschritt erhöht den Full-Score-Anteil beispielsweise um 4,32 Prozentpunkte. Wird dieser Zugewinn im Verhältnis zum zusätzlichen Rechenaufwand als zu gering bewertet, kann stattdessen auf den Reparaturschritt verzichtet und eine größere Anzahl initialer Kandidaten erzeugt werden. Entsprechend lassen sich auch die eingesetzten Modelle, Promptstrategien und Prüfverfahren unabhängig voneinander anpassen. Jede solche Änderung erfordert jedoch eine erneute Evaluation der nachfolgenden Ergebnisse, da sie sowohl die Qualität als auch die Größe des finalen Aufgabenpools beeinflussen kann.

Schließlich sind die automatisierte Laufzeit der Pipeline und der menschliche Arbeitsaufwand getrennt zu betrachten. Generierung und Validierung benötigen Rechenzeit, laufen jedoch ohne fortlaufende manuelle Eingriffe ab. Der menschliche Arbeitsanteil verschiebt sich dadurch von der vollständigen Erstellung einer Aufgabe hin zur fachlichen Kontrolle und Auswahl bereits erzeugter Aufgaben. Für den praktischen Einsatz

ist daher die zusätzliche Generierungszeit gegen den erzielten Qualitätsgewinn und die Verringerung des manuellen Erstellungsaufwands abzuwägen.

6. Fazit und Ausblick

Dieses Kapitel führt die Ergebnisse der Arbeit auf die in Abschnitt 1.1 formulierte Zielsetzung und Forschungsfrage zurück. Zunächst werden die zentralen Resultate zusammengeführt und hinsichtlich der Forschungsfrage eingeordnet. Darauf aufbauend werden die Grenzen des entwickelten Verfahrens und mögliche Weiterentwicklungen dargestellt.

6.1. Beantwortung der Forschungsfrage

Ausgangspunkt der Arbeit war die Frage, welcher Anteil der Generierungsdurchläufe einer sprachmodellgestützten und automatisch geprüften Pipeline für vorgegebene Datenbankschemata und unterschiedliche SQL-Schwierigkeitskategorien zu vollständigen SQL-Übungsaufgaben führt, die sämtliche operationalisierten Qualitätskriterien erfüllen. Ergänzend wurde untersucht, welche Unterschiede bei verschiedenen Formen der Schemaübergabe, Promptgestaltungen und Sprachmodellen auftreten und wie sich die nachgelagerte Verbesserung fehlerhafter Anfragen in den beobachteten Zwischenergebnissen widerspiegelt.

Die Ergebnisse zeigen, dass eine solche Erzeugung in großem Umfang möglich ist, wenn die Modellausgaben nicht unmittelbar übernommen, sondern schrittweise analysiert, ausgewählt und verbessert werden. Entscheidend ist dabei die getrennte Qualitätssicherung der Musterlösungs-Anfragen und der anschließend erzeugten Aufgabentexte.

Nach der Auswahl aus vier Promptvarianten erreichen 4 817 der 5 600 ausgewählten SQL-Anfragen den Full Score. Der Reparaturschritt erhöht diese Zahl auf 5 059 beziehungsweise 90,34 %. Aus den daraufhin erzeugten Paaren aus Musterlösungs-Anfrage und Aufgabentext bestehen 4 131 die Rekonstruktionsprüfung. In diesem zentralen Validierungsschritt werden 928 Paare beziehungsweise 18,34 % nicht übernommen, weil

die Konsistenz von Aufgabentext und Musterlösungs-Anfrage durch den Ausführungsvergleich nicht bestätigt werden kann. Der zusätzliche Qualitätsfilter schließt weitere 22 Aufgaben aus. Damit verbleiben 4 109 Aufgaben im finalen Pool. Dies entspricht 81,22 % der in die zweite Phase eingegangenen Paare und 73,38 % der 5 600 Durchläufe. Der letztgenannte Anteil beschreibt die quantitative Ausbeute der Pipeline und nicht die fachliche Fehlerfreiheit der Aufgaben.

Der deutliche Rückgang während der Rekonstruktionsprüfung zeigt, dass eine geprüfte Musterlösungs-Anfrage und ein sprachlich plausibler Aufgabentext für sich genommen noch keine konsistente Übungsaufgabe gewährleisten. Die gesonderte Prüfung ihrer Zuordnung bildet daher einen wesentlichen Bestandteil der Qualitätssicherung.

Die ergänzenden Auswertungen zeigen deutliche Unterschiede zwischen den untersuchten Sprachmodellen sowie zwischen den beiden Formen der Schemaübergabe. Die vier Promptvarianten erreichen abhängig von der geforderten Anfrageform unterschiedliche Full-Score-Anteile. Bei der fallbezogenen Auswahl aus vier Kandidaten liegt der beobachtete Anteil höher als bei der durchgängigen Verwendung der besten festen Promptvariante. Der Reparaturschritt erhöht den Anteil mit Full Score bewerteter Musterlösungs-Anfragen, beseitigt verbleibende Fehler jedoch nicht vollständig.

Die Grenzen dieser automatischen Bewertung werden in Abschnitt 5.4 ausführlich diskutiert. Insbesondere prüfen die Verfahren nur die modellierten Fehlerfälle, und die Ergebnisgleichheit auf einem einzelnen Datenbankzustand belegt keine allgemeine semantische Äquivalenz. Auch die Regeln des zusätzlichen Qualitätsfilters wurden aus dem untersuchten Datenbestand abgeleitet. Ihre Übertragbarkeit auf neue Aufgaben wurde nicht unabhängig evaluiert. Der finale Aufgabenpool ist daher als automatisiert vorgeprüfte Grundlage zu verstehen.

Für den praktischen Einsatz bleibt eine abschließende Freigabe durch eine fachlich verantwortliche Person erforderlich. Die 76 durch Review-Regeln markierten Aufgaben können dabei priorisiert kontrolliert werden, ohne dass die übrigen Aufgaben von der Sichtung ausgenommen sind. Der Human-in-the-Loop ergänzt somit die automatische Vorprüfung um die fachliche Letztentscheidung.

6.2. Ausblick

Eine mögliche Weiterentwicklung besteht darin, die Aussagekraft der automatischen Prüfung zu erhöhen. Der Vergleich der Ausführungsergebnisse könnte auf mehreren gezielt erzeugten Datenbankzuständen wiederholt werden, um zufällige Ergebnisgleichheiten zu reduzieren. Zusätzliche semantische Tests und weitere Prüf- und Reparaturzyklen könnten insbesondere Fehler erfassen, die von den gegenwärtigen Verfahren noch nicht abgedeckt werden. Die nach der Reparatur verbleibenden Fehlermuster bieten hierfür einen konkreten Ausgangspunkt.

Auch die Validierung durch Rekonstruktion lässt sich robuster gestalten. Gegenwärtig hängt die rekonstruierte SQL-Anfrage von einem einzelnen Text-to-SQL-Modell ab und kann daher dessen modellspezifische Fehler übernehmen. Künftig könnten mehrere Modelle unabhängig voneinander SQL-Anfragen aus demselben Aufgabentext erzeugen. Eine Mehrheitsentscheidung auf Grundlage der Ausführungsergebnisse würde einzelne abweichende Rekonstruktionen weniger stark gewichten. Auch ein solcher Konsens wäre allerdings kein Nachweis allgemeiner semantischer Äquivalenz, sondern ein zusätzlicher Plausibilitätsindikator.

Eine weitere Verbesserungsmöglichkeit liegt in der gezielten Untersuchung unterschiedlicher Sprachmodelle für die einzelnen Aufgaben der Pipeline. Ein Modell mit guten Ergebnissen bei der SQL-Generierung muss nicht gleichermaßen für die Reparatur, die Erzeugung von Aufgabentexten oder die Rekonstruktion geeignet sein. Eine systematische Evaluation unterschiedlicher Modellkombinationen könnte deshalb weitere Qualitätsgewinne erschließen. Ebenso könnte untersucht werden, ob eine zunächst englischsprachige Generierung mit anschließender Übersetzung ins Deutsche die inhaltliche Genauigkeit verbessert. Damit ließe sich der Einfluss der Sprache der Aufgabentexterzeugung auf die inhaltliche Genauigkeit gesondert untersuchen.

Die wiederkehrenden Auffälligkeiten des zusätzlichen Qualitätsfilters können zudem in die Erzeugung der Aufgabentexte zurückgeführt werden. Entsprechende Fehler könnten bereits durch explizite Vorgaben bei der Generierung verringert werden. Dazu zählen Vorgaben gegen Verweise auf die Musterlösungs-Anfrage, unnötige Nennungen von SQL-Konstrukten und Fehlübersetzungen von Schemabezeichnern. Ob solche Ergän-

zungen tatsächlich zu besseren Aufgabentexten führen, wäre anhand einer erneuten Auswertung der Kandidaten und des finalen Aufgabenpools zu überprüfen.

Der modulare Aufbau ermöglicht es außerdem, einzelne Verarbeitungsschritte unabhängig voneinander weiterzuentwickeln oder an den jeweiligen Einsatzkontext anzupassen. So kann etwa abgewogen werden, ob der durch die Reparatur erzielte Anstieg des Full-Score-Anteils den zusätzlichen Modelllauf rechtfertigt oder ob stattdessen mehr Kandidaten in der initialen Generierung erzeugt werden könnten. Das Verhältnis zwischen zusätzlicher Generierungszeit und erzieltem Qualitätsgewinn könnte durch ein leistungsfähigeres und zugleich recheneffizienteres Basismodell verbessert werden. Darüber hinaus könnten umfangreichere und repräsentativere Trainingsdaten weitere Fehlermuster abdecken und den Anteil erfolgreich reparierter Anfragen erhöhen. Bei entsprechenden Änderungen wäre eine erneute Evaluation sinnvoll, da sie sowohl die Fehlerverteilung als auch den Umfang des finalen Aufgabenpools beeinflussen können.

Als Anschlussarbeit bietet sich die Überführung der Pipeline in eine bedienfreundliche Anwendung an. Ein solches System könnte Lehrenden ermöglichen, Datenbankschemata und gewünschte Anfrageformen auszuwählen, Aufgaben automatisch zu erzeugen, Prüfergebnisse nachzuvollziehen und einzelne Aufgaben fachlich freizugeben und zu exportieren. Auf diese Weise ließe sich der entwickelte Prototyp zu einem praktisch nutzbaren Werkzeug für die Erstellung von SQL-Übungs- und Prüfungsaufgaben weiterentwickeln.

Anhang

A. Modellaufrufe und Inferenzparameter

Tabelle 3 dokumentiert die für die abschließend verwendeten Generierungsschritte konfigurierten Inferenzparameter. Die maximale Zahl der Versuche bezeichnet erneute Modellaufrufe, wenn keine verwertbare Ausgabe erzeugt wurde. Technische Wiederholungen aufgrund von Übertragungsfehlern werden davon getrennt behandelt.

Schritt	**Modell/Konfiguration**	**Temp.**	***top-p***	**Token**	**Versuche**
Initiale SQL-Generierung	ArcticSQL (7B, F16)	0,3	0,9	3 000	3
Verbesserung der SQL-Anfrage	untersuchte Qwen2.5- und Qwen3-Varianten	0,2	0,9	3 000	3
Erzeugung der Aufgabentexte	`Qwen3-Coder-30B-A3B-Instruct`	0,3	0,9	700	3
Rekonstruktion der SQL-Anfrage	ArcticSQL (7B, F16)	0,0	0,9	700	3

Tabelle 3: Konfiguration der sprachmodellgestützten Generierungsschritte

Die Temperatur steuert die Zufälligkeit der Tokenauswahl, während *top-p* die Auswahl auf die wahrscheinlichsten Token bis zur angegebenen kumulierten Wahrscheinlichkeit begrenzt. Die Spalte „Token“ gibt die konfigurierte Obergrenze der erzeugten Token pro Antwort an. Für die initiale SQL-Generierung und die Verbesserung wird die Ausgabe zusätzlich am Abschlussmarker `</SQL>` beendet.

B. Promptvarianten der initialen SQL-Generierung

Im Folgenden werden die vier Promptvarianten anhand eines echten Beispiels aus dem Lauf für die Datenbank `department_management` und den SQL-Typ `diff3` dargestellt.

Die Kennzeichnungen `system:` und `user:` geben die Rollen der einzelnen Nachrichten innerhalb eines Modellaufrufs an. Die Systemnachricht legt übergeordnete Verhaltens- und Ausgabevorgaben für das Sprachmodell fest. Die Nutzernachrichten enthalten die konkrete Generierungsaufgabe sowie die dafür erforderlichen Schema- und Steuerinformationen. Mehrere aufeinanderfolgende `user:`-Blöcke bilden entsprechend mehrere Nachrichten innerhalb desselben Dialogs ab.

Promptvariante 1: Direkter Schema-Prompt

Die erste Variante gibt dem Modell direkt das Schema und den gewünschten SQL-Typ. Das Modell soll ohne weitere Zwischenschritte eine passende SQL-Anfrage erzeugen.

```
system:
You are a database expert. You receive database schema information
and a difficulty level. Your task is to generate a single SQL query
that matches the requested difficulty and can serve as a basis for
creating exam questions. [...]

user:
You are looking at the database department_management.
Schema: department(Department_ID(5, 8, 12), Name('Interior',
'Labor', 'Energy'), Creation('1849', '1913', '1977'),
Ranking(5, 8, 12), Budget_in_Billions(10.7, 59.7, 21.5),
Num_Employees(71436.0, 17347.0, 116100.0))
head(head_ID(2, 9, 3), name(...), born_state(...), age(...))
management(department_ID(11, 7, 2), head_ID(10, 3, 6),
temporary_acting('No', 'Yes', 'ExampleA'),
Department_ID=department.Department_ID, head_ID=head.head_ID)

Primary keys:
- department: Department_ID
- head: head_ID
- management: department_ID, head_ID
```

```
Foreign keys:
- management.Department_ID = department.Department_ID
- management.head_ID = head.head_ID

The SQL query you should generate has to be a SQL statements
with NOT EXISTS or NOT IN without aggregation (non-monotone queries).
Return only the SQL query wrapped in <SQL> and </SQL> with no
additional text.
```

Die erzeugte Anfrage lautet:

```
SELECT d.`Department_ID`, d.`Name`, d.`Creation`, d.`Ranking`,
       d.`Budget_in_Billions`, d.`Num_Employees`
FROM `department` AS d
WHERE NOT EXISTS (
  SELECT 1
  FROM `management` AS m
  WHERE m.`Department_ID` = d.`Department_ID`
    AND m.`temporary_acting` = 'Yes'
)
```

Promptvariante 2: Divide-and-Conquer-Prompt

Die zweite Variante zerlegt die Aufgabe in interne Teilschritte. Das Modell soll zuerst das Schema analysieren, danach die benötigten SQL-Konstrukte planen und erst anschließend die finale Anfrage ausgeben. Diese Zwischenschritte dürfen nicht in der Antwort erscheinen.

```
system:
You are a database expert using an internal three-step
divide-and-conquer strategy. Use the steps only as internal reasoning:
do not describe them or your thought process in the output.
```

```
Step 1 (internal): analyse the schema and relationships.
Step 2 (internal): plan the joins, filters, aggregations and structure
that match the requested difficulty.
Step 3 (output): write the final SQL query.

user:
Step 1 (internal): analyse the database department_management using
the information below and identify the key tables, attributes,
relationships, and any useful example values. Do not output this
analysis.

Schema: department(...), head(...), management(...)
Primary keys: [...]
Foreign keys: [...]

user:
Step 2 (internal): based on Step 1, decide which joins, filters,
aggregations or other constructs are needed for a query with difficulty
'SQL statements with NOT EXISTS or NOT IN without aggregation
(non-monotone queries)'. Do not output this reasoning.

user:
Step 3 (output): now produce the final SQL query that implements
the Step 2 plan.
```

Die erzeugte Anfrage lautet:

```
SELECT d.`Name` AS `Department_Name`,
       h.`name` AS `Head_Name`,
       m.`temporary_acting` AS `Acting_Status`
FROM `department` AS d
JOIN `management` AS m
  ON d.`Department_ID` = m.`department_ID`
JOIN `head` AS h
```

```
  ON m.`head_ID` = h.`head_ID`
WHERE NOT EXISTS (
  SELECT 1
  FROM `department`
  WHERE `Department_ID` = d.`Department_ID`
    AND `Ranking` < 10
)
```

Promptvariante 3: Mehrstufiger Planungsdialog

Die dritte Variante nutzt ebenfalls interne Planung, formuliert diese aber als mehrere aufeinanderfolgende Nutzeranweisungen. Dadurch wird das Modell stärker dazu gezwungen, Tabellen, Join-Pfade, Filter und die finale Anfrage getrennt zu planen.

```
user:
Context for database department_management:
Schema: department(...), head(...), management(...)
Primary keys: [...]
Foreign keys: [...]
Target difficulty: SQL statements with NOT EXISTS or NOT IN
without aggregation (non-monotone queries)

user:
Internally (do not show this), identify the relevant tables,
join paths and key attributes based on the schema above.

user:
Internally (do not show this), determine the filters, aggregations,
grouping, HAVING clauses or set operations required by the requested
difficulty. Revise your internal plan if needed.

user:
Internally (do not show this), outline the final SELECT/FROM/WHERE
```

```
and, if needed, GROUP BY/HAVING structure.

user:
Now output only the final SQL query text, wrapped in <SQL> and </SQL>,
with no explanation, no labels and no comments.
```

Die erzeugte Anfrage lautet:

```
SELECT `department`.`Name`,
       `department`.`Budget_in_Billions`,
       `head`.`Name` AS `Head_Name`,
       `head`.`Age` AS `Head_Age`,
       `department`.`Num_Employees`,
       `department`.`Ranking`,
       `department`.`Budget_in_Billions` * `department`.`Num_Employees`
         AS `Total_Budget_Employees`
FROM `department`
JOIN `management`
  ON `department`.`Department_ID` = `management`.`Department_ID`
JOIN `head`
  ON `management`.`head_ID` = `head`.`head_ID`
WHERE NOT EXISTS (
  SELECT 1
  FROM `management`
  WHERE `management`.`Department_ID` = `department`.`Department_ID`
    AND `management`.`temporary_acting` = 'Yes'
)
```

Promptvariante 4: Few-Shot-Prompt mit Beispielanfragen

Die vierte Variante ergänzt das Schema um Beispielanfragen für den gewünschten SQL-Typ. Diese Beispiele stammen nicht aus der konkreten Datenbank, sondern dienen nur als strukturelle Orientierung. Das Modell soll die Beispiele nicht kopieren, sondern eine neue Anfrage auf Basis des gegebenen Schemas erzeugen.

```
user:
Here is the schema for database department_management:
Schema: department(...), head(...), management(...)
Primary keys: [...]
Foreign keys: [...]

user:
Example queries for difficulty 'SQL statements with NOT EXISTS
or NOT IN without aggregation (non-monotone queries)':

- Example 1:
SELECT supplier_name
FROM suppliers s
WHERE NOT EXISTS (
  SELECT 1
  FROM shipments sh
  WHERE sh.supplier_id = s.id
);

- Example 2:
SELECT employee_id
FROM employees e
WHERE NOT EXISTS (
  SELECT 1
  FROM bonus b
  WHERE b.employee_id = e.id
);

- Example 3:
SELECT product_code
FROM catalog
WHERE product_code NOT IN (
```

```
  SELECT product_code
  FROM warehouse_stock
);

Do not copy these verbatim -- use them only as guidance for the
structure and difficulty.

user:
Now create a new SQL query with this difficulty:
SQL statements with NOT EXISTS or NOT IN without aggregation
(non-monotone queries).
```

Die erzeugte Anfrage lautet:

```
SELECT d.`Name`
FROM `department` AS d
WHERE NOT EXISTS (
  SELECT 1
  FROM `management` AS m
  WHERE m.`Department_ID` = d.`Department_ID`
)
```

C. Eingabestruktur der Fremdschlüsselprüfung

Das folgende Beispiel zeigt die Eingabestruktur, die der Fremdschlüsselprüfung übergeben wird.

```
[
  {
    "db_id": "department_management",
    "tables": [
      "Table department columns = [Department_ID, Name, Creation,
      Ranking, Budget_in_Billions, Num_Employees]",
```

```
    "Table head columns = [head_ID, name, born_state, age]",
    "Table management columns = [department_ID, head_ID,
    temporary_acting]"
  ],
  "primary_keys": {
    "department": ["Department_ID"],
    "head": ["head_ID"],
    "management": ["department_ID", "head_ID"]
  },
  "foreign_keys": [
    "management.Department_ID = department.Department_ID",
    "management.head_ID = head.head_ID"
  ],
  "foreign_key_cardinalities": ["N:1", "N:1"],
  "column_types": {
    "department": {
      "Department_ID": "INT",
      "Name": "TEXT",
      "Creation": "TEXT",
      "Ranking": "INT",
      "Budget_in_Billions": "REAL",
      "Num_Employees": "REAL"
    },
    "head": {
      "head_ID": "INT",
      "name": "TEXT",
      "born_state": "TEXT",
      "age": "REAL"
    },
    "management": {
      "department_ID": "INT",
      "head_ID": "INT",
```

```
        "temporary_acting": "TEXT"
      }
    },
    "generated_query": "SELECT d.Department_ID FROM department d JOIN
    management m USING (department_ID)"
  }
]
```


D. Syntaxbaumdarstellung der Fremdschlüsselprüfung

Abbildung 16 zeigt die für die Fremdschlüsselprüfung relevanten Knoten einer Anfrage mit `JOIN USING`. Die Baumdarstellung bildet die Grundlage für die anschließende Aliasauflösung und Extraktion der Spaltenbeziehung.

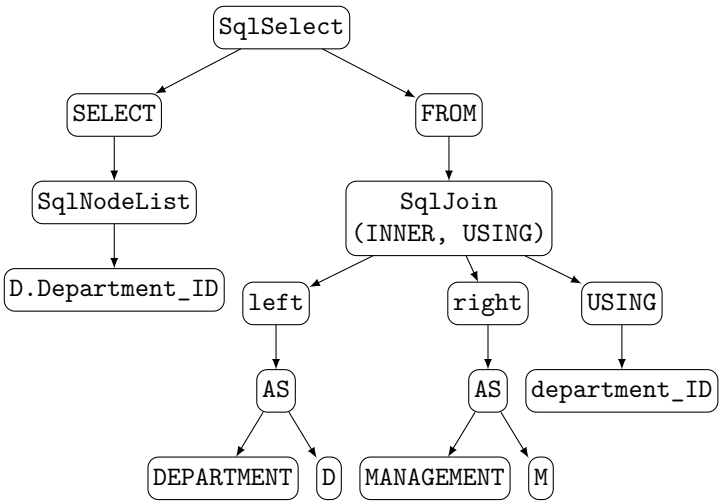


Abbildung 16: Schematische Darstellung der relevanten Syntaxbaumknoten einer SQL-Anfrage mit `JOIN USING`.

E. Pseudocode der Fremdschlüsselprüfung

Algorithm 1 ForeignKeyChecker – Kernalgorithmus

```
1: procedure DETECTFOREIGNKEYS(query, schemaFK)
2:     ast := parseSQL(query)
3:     astRoot := rootOf(ast)
4:     baseScope := collectAliases(fromOf(astRoot))          ▷ Top-Level-Scope
5:     pairs := empty collection          ▷ kanonische, qualifizierte Spaltenpaare
6:     visitor(astRoot, baseScope, pairs)          ▷ rekursiv mit überlagerten Scopes
7:     result := matchAgainstSchema(pairs, schemaFK)    ▷ reihenfolgeunabhängiger
   Abgleich
8:     return result
9: procedure VISITOR(node, scope, pairs)
10:    if isSelect(node) then
11:        localScope := collectAliases(fromOf(node))
12:        curr := mergeScopes(scope, localScope) ▷ innerer Scope überdeckt äußeren
13:    else
14:        curr := scope
15:    if isNaturalJoin(node) then
16:        append pairsFromNatural(node, curr) to pairs
17:    if isUsingJoin(node) then
18:        append pairsFromUsing(node, curr) to pairs
19:    if hasOnCondition(node) then
20:        append pairsFromCondition(onCondition(node), curr) to pairs
21:    if isWhere(node) then
22:        append pairsFromCondition(whereCondition(node), curr) to pairs
23:    if hasInNotInSubquery(node) then
24:        append pairsFromIn(node, curr) to pairs
25:    if hasExistsSubquery(node) then
26:        visitor(existsSubquery(node), curr, pairs)
27:    for each child in childrenOf(node) do
28:        visitor(child, curr, pairs)
```

F. Beispiele zur Fremdschlüsselprüfung

Die folgenden Beispiele ergänzen die Beschreibung der Fremdschlüsselprüfung um konkrete SQL-Konstrukte, ein Analyseergebnis und einen Testablauf.

F.1. Auswertung spezieller SQL-Konstrukte

Die konstruktbezogenen Analyseschritte erhalten den jeweiligen AST-Knoten, die Aliaszuordnungen und die Tabellen- und Spalteninformationen. Erkannte Spaltenbeziehungen werden als normalisierte Vergleichsbedingungen in einer gemeinsamen Menge gesammelt. Mehrspaltige Beziehungen können dabei als zusammengehörige Bedingungen repräsentiert werden. Erst der anschließende Abgleich bestimmt, welche dieser Bedingungen einem definierten Fremdschlüssel entsprechen.

(a) NATURAL JOIN

Gesucht. NATURAL JOIN erzeugt implizite Gleichheitsbedingungen über alle Spalten, die in beiden beteiligten Tabellen denselben Namen tragen. Stimmen mehrere Spaltennamen überein, entsteht dadurch eine zusammengesetzte Join-Bedingung. Alle betroffenen Spaltenpaare gehören gemeinsam zu dieser Verknüpfung.

Beispiel:

```
SELECT d.Department_ID
FROM department d
NATURAL JOIN management m;
```

Algorithmische Verarbeitung. Der Fremdschlüsselprüfer erkennt im AST den NATURAL-JOIN-Knoten und löst zunächst die Aliase auf (hier: `d` steht für `department`, `m` für `management`). Anschließend ermittelt er aus dem Schema alle Spalten, die in beiden Tabellen gleich heißen. Für jede dieser Spalten wird eine Gleichheitsbedingung zwischen den vollständig qualifizierten Spaltennamen erzeugt. Im Beispiel entsteht die Bedingung `department.Department_ID = management.department_ID`. Gibt es mehrere gleichnamige Spalten, werden die zugehörigen Gleichheiten als zusammengehöriges Bündel behandelt (zusammengesetzte Bedingung).

(b) JOIN ... USING

Gesucht. Eine `JOIN USING`-Klausel benennt explizit die Spalten, über die zwei Tabellen verknüpft werden. Für jede genannte Spalte entsteht eine Gleichheitsbedingung zwischen den gleichnamigen Spalten beider Tabellen. Werden mehrere Spalten angegeben, bilden die entstehenden Paare gemeinsam eine zusammengesetzte Join-Bedingung.

Beispiel:

```
SELECT d.Department_ID
FROM department d
JOIN management m USING (department_ID);
```

Algorithmische Verarbeitung. Der Fremdschlüsselprüfer erkennt im AST einen JOIN-Knoten mit der Bedingungsart `USING`. Zuerst werden die Aliase aufgelöst. Dabei steht `d` für `department` und `m` für `management`. Anschließend wird die in `USING` angegebene Spaltenliste ausgelesen (hier: `department_ID`). Für jede dieser Spalten wird eine Gleichheit zwischen den vollständig qualifizierten Spaltennamen erzeugt. Im Beispiel lautet sie `department.Department_ID = management.department_ID`. Sind in `USING` mehrere Spalten genannt, werden alle zugehörigen Gleichheiten als zusammengehöriges Bündel behandelt (zusammengesetzte Bedingung).

(c) JOIN ... ON

Gesucht. Explizite Join-Bedingungen in einer `ON`-Klausel, formuliert als Gleichheiten zwischen Spalten. Eine `ON`-Klausel kann einzelne Vergleiche enthalten oder mehrere, die konjunktiv (`AND`) verbunden sind. Auch Tupelgleichheiten werden dabei spaltenweise in einzelne Vergleiche zerlegt. Treffen in einem Anfrageblock mehrere passende Paare für dasselbe Tabellenpaar zusammen, bilden sie eine zusammengesetzte Join-Bedingung.

Beispiel:

```
SELECT d.Department_ID
FROM department d
JOIN management m ON d.Department_ID = m.department_ID
JOIN head h       ON m.head_ID       = h.head_ID
WHERE h.age > 50;
```

Algorithmische Verarbeitung. Der Fremdschlüsselprüfer erkennt die `JOIN`-Knoten mit `ON`-Bedingungen und löst zunächst die Aliase auf (`d` → `department`, `m` → `management`, `h` → `head`). Die `ON`-Ausdrücke werden als Gleichheiten extrahiert. Konjunktive Ausdrücke werden in einzelne Paare zerlegt, Tupelgleichheiten ebenfalls spaltenweise aufgeteilt. Für das Beispiel entstehen die beiden Gleichheiten

```
department.Department_ID = management.department_ID
management.head_ID       = head.head_ID
```

In diesem Fall liegen zwei getrennte Verknüpfungen vor (`department und management` sowie `management und head`). Wären innerhalb eines Blocks mehrere Paare für dasselbe Tabellenpaar vorhanden, würden sie gemeinsam als zusammengesetzte Bedingung geprüft.

(d) WHERE-Klausel mit Gleichheit

Gesucht. Gleichheitsvergleiche zwischen Spalten, die in der `WHERE`-Klausel stehen. Sie treten häufig bei impliziten Joins mit komma-separierten Tabellen auf und können je nach Anzahl der Vergleiche eine einfache oder eine zusammengesetzte Verknüpfung bilden.

Beispiel:

```
SELECT d.Department_ID
FROM department d, management m, head h
WHERE d.Department_ID = m.department_ID
  AND h.head_ID       = m.head_ID
  AND h.age > 50;
```

Algorithmische Verarbeitung. Der Fremdschlüsselprüfer analysiert die `WHERE`-Klausel, zerlegt konjunktive Ausdrücke in einzelne Bedingungen und löst Aliase zu vollständig qualifizierten Spaltennamen auf. Aus dem Beispiel werden die beiden Tabellengleichheiten

```
department.Department_ID = management.department_ID
management.head_ID       = head.head_ID
```

als potenzielle Fremdschlüsselverwendungen übernommen. Mehrere Gleichheiten, die im selben Anfrageblock für dasselbe Tabellenpaar auftreten, werden gemeinsam als zusammengesetzte Bedingung gewertet. Gleichheiten aus unterschiedlichen Blöcken oder alternativen Pfaden (`OR`) werden nicht zusammengezählt.

(e) IN / NOT IN Subqueries

Gesucht. Vergleiche, bei denen eine Spalte oder eine Spaltenliste der äußeren Abfrage per `IN` bzw. `NOT IN` gegen das Ergebnis einer Subquery geprüft wird. Dabei entstehen positionsweise Zuordnungen zwischen den links stehenden Spalten und den Ausgabespalten der Subquery. Bei mehreren Spalten bilden diese Zuordnungen gemeinsam eine zusammengesetzte Bedingung.

Beispiel:

```
SELECT d.Department_ID
FROM department d
WHERE d.Department_ID IN (
  SELECT m.department_ID FROM management m
);
```

Algorithmische Verarbeitung. Der Fremdschlüsselprüfer erkennt den `IN`-Ausdruck mit einer Subquery auf der rechten Seite. Zunächst werden Aliase aufgelöst (`d` → `department`, `m` → `management`). Danach werden die linke Seite (hier: `d.Department_ID`) und die Ausgabespalten der Subquery (hier: `m.department_ID`) positionsweise einander zugeordnet. Für jedes zuordenbare Paar wird eine Gleichheit zwischen vollständig qualifizierten Spaltennamen erzeugt. Im Beispiel entsteht

```
department.Department_ID = management.department_ID
```

Bei mehrspaltigen `IN`-Ausdrücken (linke Seite mit mehreren Spalten bzw. Subquery mit mehreren Projektionen) werden alle Paare gemeinsam als zusammengesetzte Bedingung behandelt. `NOT IN` wird dabei analog ausgewertet. Die logische Negation ändert nichts an der Zuordnung der beteiligten Spaltenpaare.

Alle erkannten Beziehungen werden für den Fremdschlüsselabgleich normalisiert. Die spezielle Verarbeitung von `IN` und `NOT IN` unterscheidet zwischen einspaltigen und

mehrspaltigen Zuordnungen. Enthält die `FROM`-Klausel der Unteranfrage unmittelbar einen `JOIN`-Knoten, wird die spezielle `IN`-Auswertung ohne Ableitung einer äußeren Spaltenzuordnung beendet. Auch bei nicht unterstützten Projektionen oder nicht zuordenbaren Bestandteilen mehrspaltiger Ausdrücke kann diese Methode vorzeitig zurückkehren.

Wird ihr abschließender Verarbeitungspfad erreicht, übergibt sie eine vorhandene `WHERE`-Bedingung der Unteranfrage an die Auswertung von Spaltenvergleichen. Dies ist kein allgemeiner Rückfall, der jede zuvor fehlgeschlagene Zuordnung ersetzt. Davon zu unterscheiden ist die allgemeine AST-Traversierung, die nach der speziellen `IN`-Behandlung untergeordnete Knoten weiter besuchen kann. Innerhalb der Unteranfrage enthaltene Beziehungen werden damit nach den jeweils implementierten Knotenregeln untersucht.

F.2. Beispiel eines Analyseergebnisses

Beispiel (`department_management`): Die verwendete Anfrage lautet:

```
SELECT d.Department_ID
FROM department d
JOIN management m USING (department_ID)
```

Nach der Normalisierung entspricht die erkannte Gleichheit dem im Schema definierten Fremdschlüssel zwischen `management` und `department`. Sie wird daher in den Feldern `used_fk` und `correctly_used` gespeichert. Der zweite Fremdschlüssel zwischen `management` und `head` bleibt unbelegt und wird als `not_used` klassifiziert. Das resultierende Auswertungsobjekt dokumentiert dies explizit:

```
"foreign_key_check": {
  "predefined_foreign_keys": [
    "management.Department_ID = department.Department_ID",
    "management.head_ID = head.head_ID"
  ],
  "used_fk": [
    "DEPARTMENT.DEPARTMENT_ID = MANAGEMENT.DEPARTMENT_ID"
  ],
  "correctly_used": [
```

```
    "DEPARTMENT.DEPARTMENT_ID = MANAGEMENT.DEPARTMENT_ID"
  ],
  "not_used": [
    "HEAD.HEAD_ID = MANAGEMENT.HEAD_ID"
  ],
  "incorrect": [],
  "selfjoin_used": "no"
}
```

F.3. Beispiel der Testvalidierung

Beispielablauf (`join_on_test`).

```
SELECT d.Department_ID
FROM department d
JOIN management m
  ON d.Department_ID = m.department_ID
JOIN head h
  ON m.head_ID = h.head_ID
WHERE h.age > 50;
```

Für diese Anfrage erwartet die Referenzauswertung, dass beide Fremdschlüsselbeziehungen erkannt und als korrekt verwendet klassifiziert werden:

- `used_fk`:
  `DEPARTMENT.DEPARTMENT_ID = MANAGEMENT.DEPARTMENT_ID`
  `HEAD.HEAD_ID = MANAGEMENT.HEAD_ID`
- `correctly_used`:
  `DEPARTMENT.DEPARTMENT_ID = MANAGEMENT.DEPARTMENT_ID`
  `HEAD.HEAD_ID = MANAGEMENT.HEAD_ID`
- `not_used`: ∅
- `incorrect`: ∅
- `selfjoin_used`: `NO`

Beim Testlauf wird das tatsächliche Analyseergebnis mit dieser Referenzauswertung verglichen. Eine Abweichung, etwa eine nicht erkannte `JOIN ... ON`-Bedingung, eine falsche Aliasauflösung oder eine fehlerhafte Selbst-Join-Klassifikation, führt zu einem Testfehlschlag.

G. Beispiele für SFT- und DPO-Trainingsdaten

Das folgende Beispiel zeigt das Prinzip eines SFT-Records anhand eines echten, gekürzten Falls aus der Datenbank `allergy_1`. Die Eingabe enthält nicht nur den fehlerhaften SQL-Anfrage-Kandidaten, sondern auch das Analysefeedback. Der Kandidat erreicht nur 31 von 37 Punkten, weil er die geforderte SQL-Schwierigkeitskategorie `diff1` verfehlt: Für `diff1` wird eine Anfrage mit `WHERE`-Bedingung erwartet, der Kandidat enthält jedoch keine Einschränkung. Die Zielausgabe ist ein anderer Kandidat desselben Falls, der alle Prüfschritte besteht und damit 37 von 37 Punkten erreicht.

SFT-Beispiel: fehlerhafte Eingabe und korrekte Zielausgabe

Gekürzte Eingabe an das Modell:

```
Datenbank: allergy_1
Schema: Allergy_Type(...), Has_Allergy(...),
        Student(StuID, LName, Fname, Age, Sex, Major, Advisor, city_code)
Fremdschlüssel:
- Has_Allergy.StuID = Student.StuID
- Has_Allergy.Allergy = Allergy_Type.Allergy

Gewünschter Schwierigkeitsgrad: diff1
Beschreibung: eine Tabelle, kein JOIN, keine Aggregation, mit WHERE

Fehlerhafter Kandidat (Score 31/37):
<SQL>SELECT * FROM `Student`</SQL>

Analysefeedback:
- test1_parse: bestanden
- test2_validate: bestanden
- test3_execution: bestanden
- test4_preview: bestanden
- test5_foreign_keys: bestanden
- test6_lint: bestanden
- test7_difficulty: fehlgeschlagen, expected diff1; actual_diff = -1
```

Zielausgabe im SFT-Datensatz (Score 37/37):

```
<SQL>SELECT `StuID`, `LName`, `Fname` FROM `Student` WHERE `Age` > 18</SQL>
```

Im DPO-Datensatz wird derselbe Grundgedanke als Präferenzentscheidung formuliert. Der Prompt enthält wieder Schema, Kandidat und Fehlerfeedback. Statt einer einzelnen Zielausgabe werden jedoch zwei Antworten gegenübergestellt. Die als `chosen` markierte Antwort hat die volle Punktzahl, während die als `rejected` markierte Antwort denselben Fehler wie der ursprüngliche Kandidat enthält.

DPO-Beispiel: bevorzugte und verworfene Antwort

Gekürzter Prompt:

```
Datenbank: allergy_1
Schema: Allergy_Type(...), Has_Allergy(...),
        Student(StuID, LName, Fname, Age, Sex, Major, Advisor, city_code)
Gewünschter Schwierigkeitsgrad: diff1
Fehlerhafter Kandidat:
<SQL>SELECT * FROM Student</SQL>
Fehlerfeedback:
- test7_difficulty: expected diff1; got unknown
```

chosen, Score 37/37:

```
<SQL>SELECT StuID, LName, Fname FROM Student WHERE Age > 18</SQL>
```

rejected, Score 31/37:

```
<SQL>SELECT * FROM Student</SQL>
```

H. Technische Einstellungen der Fine-Tuning-Läufe

Containerisierte Trainingsumgebung

Für die Trainingsläufe wurden auf Grundlage eingerichteter Docker-Images [30] eigene Containerabbilder im SquashFS-Format erzeugt. Die Slurm-Aufträge [29] binden das jeweils benötigte Abbild über `--container-image` ein. Das Verzeichnis mit Trainingsdaten, Basismodellen und Konfigurationen wird über `--container-mounts` im Container bereitgestellt. Dadurch stehen innerhalb der jeweiligen Trainingsumgebung die benötigten Versionen von Axolotl [36], PyTorch [50], CUDA [51] und FlashAttention [52] bereit. Vor dem Trainingsstart prüfen die Startskripte die Sichtbarkeit der GPUs sowie die Verfügbarkeit von Axolotl und FlashAttention.

Das SFT wird über `torchrun` auf zwei GPU-Prozesse verteilt. Axolotl verwendet dabei Fully Sharded Data Parallelism [38], um Modellparameter, Gradienten und weitere Trainingszustände auf die GPUs zu verteilen. Für den DPO-Lauf fordert der Slurm-Auftrag vier GPUs an und startet das Training ebenfalls innerhalb einer containerisierten Serverumgebung.

Konvertierung und Bereitstellung der Adapter

Axolotl speichert nach jedem Trainingslauf einen Adapter, der nur die zusätzlich trainierten Parameter enthält. Dieser Adapter wird nicht vorab mit dem Basismodell zusammengeführt. Stattdessen wird er mit dem dafür vorgesehenen Konvertierungswerkzeug von `llama.cpp` separat in das GGUF-Format überführt [39]. In der anschließenden Ollama-Konfiguration werden das Basismodell und der zugehörige GGUF-Adapter gemeinsam referenziert. Beim Aufruf des daraus erzeugten Ollama-Modells werden somit die unveränderten Parameter des Basismodells mit den trainierten Adapterparametern kombiniert.

Hyperparameter

Für das Supervised Fine-Tuning wurden drei Epochen, eine Lernrate von `5.0e-6`, eine Sequenzlänge von 3 072 Tokens, eine Mikro-Batchgröße von eins und 16 Schritte zur Gradientenakkumulation verwendet. Als Optimierer kam AdamW mit einem Kosinus-Lernratenplan und einer Aufwärmphase über fünf Prozent der Trainingsschritte zum Einsatz. Das Training verwendete `bfloat16`. Das Basismodell wurde nicht quantisiert.

Die LoRA-Konfiguration [8] nutzte einen Rang von 16, einen Skalierungsparameter α von 32 und einen Dropout von 0,05. Der Rang bestimmt die Größe der zusätzlich trainierten Matrizen und damit die Kapazität des Adapters. Der Parameter α skaliert den Beitrag der Adaptergewichte, während der Dropout während des Trainings zufällig Teile des Adapterpfades deaktiviert und damit einer Überanpassung entgegenwirkt. Die Adapter wurden in die Query-, Key-, Value- und Ausgabeprojektionen des Aufmerksamkeitsmechanismus sowie in die Projektionsschichten des Feedforward-Netzes eingefügt.

Für das DPO-Training wurden ebenfalls drei Epochen, eine Lernrate von `5.0e-6`, eine Mikro-Batchgröße von eins und 16 Schritte zur Gradientenakkumulation verwendet. Die Sequenzlänge betrug 8 192 Tokens, die maximale Länge einer Antwort 512 Tokens. Das Basismodell wurde in 4-Bit-Darstellung geladen und mit einem speicheroptimierten

8-Bit-AdamW-Optimierer trainiert. Gradienten-Checkpointing und Sample Packing reduzierten den Speicherbedarf beziehungsweise bündelten mehrere kurze Beispiele innerhalb einer Trainingssequenz. Rang, Skalierung und Dropout der LoRA-Adapter entsprachen der SFT-Konfiguration. Der DPO-Parameter $\beta = 0{,}05$ [10] gewichtet dabei die Abweichung vom ursprünglichen Basismodell.

I. Promptbeispiel der Aufgabentexterzeugung

Das folgende Beispiel zeigt einen gekürzten Ausschnitt eines echten Prompts aus den erzeugten Daten. Der Prompt gehört zur Datenbank `department_management` und zur SQL-Anfrage, die alle Abteilungen ausgibt, deren Erstellungsjahr vor 1900 liegt.

```
SYSTEM:
Du bist Dozent fuer Datenbanksysteme. Du formulierst aus einer
gepruefteN SQL-Query eindeutige deutschsprachige SQL-Uebungs-
oder Klausurfragen.

USER:
Du bist Dozent fuer Datenbanksysteme.

Deine Aufgabe ist es, aus einer gepruefteN SQL-Query mehrere
eindeutige deutschsprachige Aufgabenfragen fuer eine SQL-Uebungs-
oder Klausuraufgabe zu formulieren.

Die Fragen sollen von Studierenden durch Schreiben einer SQL-Query
beantwortet werden koennen.

Datenbank:
department_management

Datenbankschema:
Database: department_management
```

```
Schema:
- Table department (department) primary key: Department_ID
  - Department_ID: INT; label: department id; primary key
  - Name: TEXT; label: name
  - Creation: TEXT; label: creation
  - Ranking: INT; label: ranking
  - Budget_in_Billions: REAL; label: budget in billions
  - Num_Employees: REAL; label: num employees
- Table head (head) primary key: head_ID
  - head_ID: INT; label: head id; primary key
  - name: TEXT
  - born_state: TEXT; label: born state
  - age: REAL
- Table management (management) primary key: department_ID, head_ID
  - department_ID: INT; label: department id; primary key
  - head_ID: INT; label: head id; primary key;
    foreign key to head.head_ID
  - temporary_acting: TEXT; label: temporary acting

Foreign keys:
- management.Department_ID = department.Department_ID
- management.head_ID = head.head_ID

SQL-Query:
SELECT `Name`, `Creation`, `Budget_in_Billions`, `Num_Employees`
FROM `department`
WHERE `Creation` < '1900'

Strenge Regeln:
- Erwaehne alle SELECT-Ausgabespalten der SQL-Query.
- Erwaehne alle WHERE-Bedingungen exakt mit Operator und Wert.
```

```
- Fuehre keine neuen Bedingungen, Werte, Sortierungen,
  Gruppierungen oder Begrenzungen ein.
- Gib wirklich nur eine Frage aus.

Ausgabeformat fuer diesen einzelnen Aufruf:
[EXPLANATION-START]
Kurze Erklaerung der SQL-Query.
[EXPLANATION-END]

[QUESTION-START]
Eine einzelne Kandidatenfrage.
[QUESTION-END]
```

J. Beispiel zur Auswahl des besten Aufgabentextes

Das folgende Beispiel stammt aus der Datenbank `bike_1`. Die Musterlösungs-Anfrage gruppiert die Fahrten nach Abonnementtyp und berechnet für jede Gruppe fünf Kennzahlen:

```sql
SELECT `subscription_type`,
       COUNT(*) AS `total_trips`,
       AVG(`duration`) AS `average_duration`,
       SUM(`duration`) AS `total_duration`,
       MIN(`duration`) AS `shortest_trip`,
       MAX(`duration`) AS `longest_trip`
FROM `trip`
GROUP BY `subscription_type`
```

Zu dieser Anfrage wurden die folgenden fünf Aufgabentexte erzeugt. Für die Darstellung wurden lediglich die ASCII-Umschreibungen deutscher Umlaute orthografisch normalisiert.

A. Ermitteln Sie für jeden Abonnementtyp die Gesamtzahl der Fahrten, die durchschnittliche Fahrtdauer, die Gesamtfahrtdauer, die kürzeste und die längste

Fahrtdauer.

B. Geben Sie für jeden Abonnementtyp die Anzahl der Fahrten, die durchschnittliche Fahrtdauer, die Gesamtfahrtdauer sowie die kürzeste und längste Fahrtdauer aus.

C. Listen Sie für jeden Abonnementtyp die Anzahl der Fahrten, die durchschnittliche Fahrtdauer, die Gesamtfahrtdauer sowie die kürzeste und längste Fahrtdauer auf.

D. Ermitteln Sie für jeden Abonnementtyp die Gesamtzahl der Fahrten, die durchschnittliche Fahrtdauer, die Gesamtfahrtdauer sowie die kürzeste und längste Fahrtdauer.

E. Ermitteln Sie für jeden Abonnementtyp die Anzahl der Fahrten, die durchschnittliche Fahrtdauer, die Gesamtfahrtdauer sowie die kürzeste und längste Fahrtdauer.

Die semantische Auswahl liefert folgende durchschnittliche Kosinusähnlichkeiten:

Kandidat	Durchschnittliche Kosinusähnlichkeit
A	0,9071
B	0,9128
C	0,8732
D	0,9109
E	0,9314

Kandidat E weist den höchsten Durchschnittswert auf und wird daher als Aufgabentext ausgewählt:

> Ermitteln Sie für jeden Abonnementtyp die Anzahl der Fahrten, die durchschnittliche Fahrtdauer, die Gesamtfahrtdauer sowie die kürzeste und längste Fahrtdauer.

K. Beispiel zur Rekonstruktionsprüfung

Das folgende Beispiel zeigt eine erfolgreiche Rekonstruktionsprüfung für die Datenbank `bike_1`. Die ursprüngliche Musterlösungs-Anfrage lautet:

```
SELECT `subscription_type`,
       COUNT(*) AS `total_trips`,
       AVG(`duration`) AS `average_duration`,
       SUM(`duration`) AS `total_duration`,
       MIN(`duration`) AS `shortest_trip`,
       MAX(`duration`) AS `longest_trip`
FROM `trip`
GROUP BY `subscription_type`
```

Der ausgewählte Aufgabentext lautet:

> Ermitteln Sie für jeden Abonnementtyp die Anzahl der Fahrten, die durchschnittliche Fahrtdauer, die Gesamtfahrtdauer sowie die kürzeste und längste Fahrtdauer.

Aus diesem Aufgabentext erzeugt das Text-to-SQL-Modell folgende rekonstruierte SQL-Anfrage:

```
SELECT subscription_type,
       COUNT(*) AS trip_count,
       AVG(duration) AS avg_duration,
       SUM(duration) AS total_duration,
       MIN(duration) AS min_duration,
       MAX(duration) AS max_duration
FROM trip
GROUP BY subscription_type
```

Beide Anfragen liefern auf demselben SQLite-Datenbankzustand dieselben Ergebniswerte. Die Spaltenaliasnamen unterscheiden sich teilweise, da das Text-to-SQL-Modell eigene Aliasnamen erzeugt. Für beide Ausführungen wurde derselbe Ergebnis-Hash gespeichert. Der konkrete Hashwert wird hier nicht angegeben.

L. Beispiel für eine finale Übungsaufgabe

Datenbank: `bike_1`

Schema:

```
station(id, name, lat, long, dock_count, city, installation_date)

status(station_id -> station, bikes_available, docks_available, time)

trip(id, duration, start_date, start_station_name, start_station_id,
     end_date, end_station_name, end_station_id, bike_id,
     subscription_type, zip_code)

weather(date, max_temperature_f, mean_temperature_f, min_temperature_f,
        max_dew_point_f, mean_dew_point_f, min_dew_point_f,
        max_humidity, mean_humidity, min_humidity,
        max_sea_level_pressure_inches, mean_sea_level_pressure_inches,
        min_sea_level_pressure_inches, max_visibility_miles,
        mean_visibility_miles, min_visibility_miles,
        max_wind_Speed_mph, mean_wind_speed_mph, max_gust_speed_mph,
        precipitation_inches, cloud_cover, events, wind_dir_degrees,
        zip_code)
```

Aufgabe: Ermitteln Sie für jeden Abonnementtyp die Anzahl der Fahrten, die durchschnittliche Fahrtdauer, die Gesamtfahrtdauer sowie die kürzeste und längste Fahrtdauer.

Die zugehörige Musterlösungs-Anfrage wird getrennt vom Aufgabentext gespeichert:

```
SELECT `subscription_type`,
       COUNT(*) AS `total_trips`,
       AVG(`duration`) AS `average_duration`,
       SUM(`duration`) AS `total_duration`,
       MIN(`duration`) AS `shortest_trip`,
       MAX(`duration`) AS `longest_trip`
FROM `trip`
GROUP BY `subscription_type`
```

Literaturverzeichnis

[1] Sami Sarsa u. a. „Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models“. In: *Proceedings of the 2022 ACM*

Conference on International Computing Education Research. ICER '22. Association for Computing Machinery, 2022, S. 27–43. DOI: 10.1145/3501385.3543957. URL: https://doi.org/10.1145/3501385.3543957.

[2] Willem Aerts, George H. L. Fletcher und Daphne Miedema. „A Feasibility Study on Automated SQL Exercise Generation with ChatGPT-3.5". In: *Proceedings of the Third International Workshop on Data Systems Education: Bridging Education Practice with Education Research*. DataEd '24. Association for Computing Machinery, 2024, S. 13–19. DOI: 10.1145/3663649.3664368. URL: https://doi.org/10.1145/3663649.3664368.

[3] Tao Yu u. a. „Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task". In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2018, S. 3911–3921. DOI: 10.18653/v1/D18-1425. URL: https://aclanthology.org/D18-1425/.

[4] Ashish Vaswani u. a. „Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Bd. 30. 2017. URL: https://proceedings.neurips.cc/paper/7181-attention-is-all-you-need.

[5] Jason Wei u. a. „Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *Advances in Neural Information Processing Systems*. Bd. 35. 2022, S. 24824–24837. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.

[6] Tom B. Brown u. a. „Language Models Are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Bd. 33. 2020, S. 1877–1901. URL: https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

[7] Nils Reimers und Iryna Gurevych. „Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. Association for Computational Lin-

guistics, 2019, S. 3982–3992. DOI: 10.18653/v1/D19-1410. URL: https://aclanthology.org/D19-1410/.

[8] Edward J. Hu u. a. „LoRA: Low-Rank Adaptation of Large Language Models“. In: *International Conference on Learning Representations*. 2022. URL: https://openreview.net/forum?id=nZeVKeeFYf9.

[9] Tim Dettmers u. a. „QLoRA: Efficient Finetuning of Quantized LLMs“. In: *Advances in Neural Information Processing Systems*. Bd. 36. 2023, S. 10088–10115. DOI: 10.52202/075280-0441. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract.html.

[10] Rafael Rafailov u. a. „Direct Preference Optimization: Your Language Model Is Secretly a Reward Model“. In: *Advances in Neural Information Processing Systems*. Bd. 36. 2023, S. 53728–53741. DOI: 10.52202/075280-2338. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/a85b405ed65c6477a4fe8302b5e06ce7-Abstract-Conference.html.

[11] Naihao Deng, Yulong Chen und Yue Zhang. „Recent Advances in Text-to-SQL: A Survey of What We Have and What We Expect“. In: *Proceedings of the 29th International Conference on Computational Linguistics*. Gyeongju, Republic of Korea: International Committee on Computational Linguistics, 2022, S. 2166–2187. URL: https://aclanthology.org/2022.coling-1.190/.

[12] Kun Xu u. a. „SQL-to-Text Generation with Graph-to-Sequence Model“. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2018, S. 931–936. DOI: 10.18653/v1/D18-1112. URL: https://aclanthology.org/D18-1112/.

[13] Stefan Brass und Christian Goldberg. „Semantic Errors in SQL Queries: A Quite Complete List“. In: *Journal of Systems and Software* 79.5 (2006), S. 630–644. DOI: 10.1016/j.jss.2005.06.028. URL: https://dbs.informatik.uni-halle.de/sqllint/jss06.pdf.

[14] Stefan Brass und Christian Goldberg. *SQLLint – Detecting Semantic Errors in SQL Queries*. 2006. URL: https://dbs.informatik.uni-halle.de/sqllint/ (besucht am 08. 09. 2026).

[15] Prashanth Dintyala, Arpit Narechania und Joy Arulraj. „SQLCheck: Automated Detection and Diagnosis of SQL Anti-Patterns“. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. SIGMOD '20. Association for Computing Machinery, 2020, S. 2331–2345. DOI: 10.1145/3318464.3389754. URL: https://doi.org/10.1145/3318464.3389754.

[16] Ruiqi Zhong, Tao Yu und Dan Klein. „Semantic Evaluation for Text-to-SQL with Distilled Test Suites“. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2020, S. 396–411. DOI: 10.18653/v1/2020.emnlp-main.29. URL: https://aclanthology.org/2020.emnlp-main.29/.

[17] Heegyu Kim u. a. „FLEX: Expert-level False-Less EXecution Metric for Text-to-SQL Benchmark“. In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2025, S. 4448–4475. DOI: 10.18653/v1/2025.naacl-long.228. URL: https://aclanthology.org/2025.naacl-long.228/.

[18] Mohammadreza Pourreza u. a. „CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL“. In: *The Thirteenth International Conference on Learning Representations*. 2025. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/974ff7b5bf08dbf9400b5d599a39c77f-Abstract-Conference.html.

[19] Mohammadreza Pourreza und Davood Rafiei. „DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction“. In: *Advances in Neural Information Processing Systems*. Bd. 36. 2023. DOI: 10.52202/075280-1577.

[20] Daya Guo u. a. „Question Generation from SQL Queries Improves Neural Semantic Parsing“. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2018, S. 1597–1607. DOI: 10.18653/v1/D18-1188. URL: https://aclanthology.org/D18-1188/.

[21] Kun Wu u. a. „Data Augmentation with Hierarchical SQL-to-Question Generation for Cross-domain Text-to-SQL Parsing“. In: *Proceedings of the 2021 Con-*

ference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, 2021, S. 8974–8983. DOI: 10.18653/v1/2021.emnlp-main.707. URL: https://aclanthology.org/2021.emnlp-main.707/.

[22] Haoyang Li u. a. „OmniSQL: Synthesizing High-quality Text-to-SQL Data at Scale“. In: *Proceedings of the VLDB Endowment* 18.11 (2025), S. 4695–4709. DOI: 10.14778/3749646.3749723. URL: https://www.vldb.org/pvldb/vol18/p4695-li.pdf.

[23] Shaoming Duan u. a. „DSQG-Syn: Synthesizing High-quality Data for Text-to-SQL Parsing by Domain Specific Question Generation“. In: *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 2025, S. 2971–2989. DOI: 10.18653/v1/2025.findings-naacl.162. URL: https://aclanthology.org/2025.findings-naacl.162/.

[24] Paul Christ, Torsten Munkelt und Jörg M. Haake. „Generating SQL-Query-Items Using Knowledge Graphs“. In: *Proceedings of the 17th International Conference on Computer Supported Education – Volume 1: AIG*. INSTICC. SciTePress, 2025, S. 785–796. ISBN: 978-989-758-746-7. DOI: 10.5220/0013498300003932.

[25] Fangyu Lei u. a. „Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows“. In: *International Conference on Learning Representations*. 2025. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/46c10f6c8ea5aa6f267bcdabcb123f97-Abstract-Conference.html.

[26] Jinyang Li u. a. „Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs“. In: *Advances in Neural Information Processing Systems*. Bd. 36. 2023. DOI: 10.52202/075280-1835. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/83fc8fab1710363050bbd1d4b8cc0021-Abstract-Datasets_and_Benchmarks.html.

[27] Stefan Brass. *Datenbank-Programmierung: Folien*. Sommersemester 2026. 2026. URL: https://users.informatik.uni-halle.de/~brass/dbp26/folien.html (besucht am 20. 08. 2026).

[28] Seth Poulsen u. a. „Insights from Student Solutions to SQL Homework Problems“. In: *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education*. ITiCSE '20. Association for Computing Machinery, 2020, S. 404–410. DOI: 10.1145/3341525.3387391. URL: https://doi.org/10.1145/3341525.3387391.

[29] Morris A. Jette und Mark Grondona. „SLURM: Simple Linux Utility for Resource Management“. In: *Job Scheduling Strategies for Parallel Processing*. Bd. 2862. Lecture Notes in Computer Science. Springer, 2003, S. 44–60. DOI: 10.1007/10968987_3. URL: https://slurm.schedmd.com/slurm_design.pdf.

[30] Dirk Merkel. „Docker: Lightweight Linux Containers for Consistent Development and Deployment“. In: *Linux Journal* 2014.239 (2014), S. 2. URL: https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment.

[31] Edmon Begoli u. a. „Apache Calcite: A Foundational Framework for Optimized Query Processing over Heterogeneous Data Sources“. In: *Proceedings of the 2018 International Conference on Management of Data*. SIGMOD '18. Association for Computing Machinery, 2018, S. 221–230. DOI: 10.1145/3183713.3190662. URL: https://doi.org/10.1145/3183713.3190662.

[32] SQLite Project. *About SQLite*. 2026. URL: https://www.sqlite.org/about.html (besucht am 14. 09. 2026).

[33] Ollama. *Ollama*. 2023. URL: https://github.com/ollama/ollama (besucht am 14. 09. 2026).

[34] Toby Mao und SQLGlot Contributors. *SQLGlot: Python SQL Parser and Transpiler*. 2021. URL: https://github.com/tobymao/sqlglot (besucht am 03. 09. 2026).

[35] Hannes Jasper Maas. *Evaluation von Äquivalenzbeweisern für SQL-Anfragen*. Universitätsinternes Software-Repository, Martin-Luther-Universität Halle-Wittenberg. Stand des zitierten Repositoriums; nicht öffentlich zugänglich. 2026. URL: https://gitlab.informatik.uni-halle.de/akrau/bachelorarbeit (besucht am 08. 09. 2026).

[36] Axolotl maintainers and contributors. *Axolotl: Open Source LLM Post-Training*. Open-Source-Framework für das Fine-Tuning großer Sprachmodelle. 2023. URL: https://github.com/axolotl-ai-cloud/axolotl (besucht am 10. 09. 2026).

[37] PyTorch Contributors. *torchrun (Elastic Launch)*. 2026. URL: https://docs.pytorch.org/docs/stable/elastic/run.html (besucht am 15. 09. 2026).

[38] Yanli Zhao u. a. „PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel“. In: *Proceedings of the VLDB Endowment* 16.12 (2023), S. 3848–3860. DOI: 10.14778/3611540.3611569. URL: https://www.vldb.org/pvldb/vol16/p3848-huang.pdf.

[39] ggml-org. *LoRA-to-GGUF Conversion Script for PEFT Adapters*. 2024. URL: https://github.com/ggml-org/llama.cpp/blob/master/convert_lora_to_gguf.py (besucht am 10. 09. 2026).

[40] Qwen Team. *Qwen3-Coder-30B-A3B-Instruct*. Hugging-Face-Modellkarte. 2025. URL: https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct (besucht am 10. 09. 2026).

[41] Sentence Transformers. *sentence-transformers/all-mpnet-base-v2*. Hugging-Face-Modellkarte. 2021. URL: https://huggingface.co/sentence-transformers/all-mpnet-base-v2 (besucht am 25. 08. 2026).

[42] Zhewei Yao u. a. „Arctic-Text2SQL-R1: Simple Rewards, Strong Reasoning in Text-to-SQL“. In: *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 2026, S. 26966–26995. DOI: 10.18653/v1/2026.findings-acl.1345. URL: https://aclanthology.org/2026.findings-acl.1345/.

[43] OpenAI. *GPT-5.6 Sol*. Am 13.08.2026 für die modellgestützte Vorsichtung der erzeugten SQL-Übungsaufgaben verwendet. 2026. URL: https://developers.openai.com/api/docs/models/gpt-5.6-sol (besucht am 10. 09. 2026).

[44] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri u. a. „The Llama 3 Herd of Models“. In: *arXiv preprint arXiv:2407.21783* (2024). DOI: 10.48550/arXiv.2407.21783. URL: https://arxiv.org/abs/2407.21783.

[45] OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. URL: https://openai.com/index/gpt-oss-model-card/ (besucht am 27. 08. 2026).

[46] Meta AI. *Llama 4 Model Card*. 2025. URL: https://github.com/meta-llama/llama-models/blob/main/models/llama4/MODEL_CARD.md (besucht am 27. 08. 2026).

[47] Binyuan Hui u. a. „Qwen2.5-Coder Technical Report“. In: *arXiv preprint arXiv:2409.12186* (2024). DOI: 10.48550/arXiv.2409.12186. URL: https://arxiv.org/abs/2409.12186.

[48] An Yang u. a. „Qwen3 Technical Report“. In: *arXiv preprint arXiv:2505.09388* (2025). DOI: 10.48550/arXiv.2505.09388. URL: https://arxiv.org/abs/2505.09388.

[49] Paul Denny u. a. „Can We Trust AI-Generated Educational Content? Comparative Analysis of Human and AI-Generated Learning Resources“. In: *arXiv preprint arXiv:2306.10509* (2023). DOI: 10.48550/arXiv.2306.10509. URL: https://arxiv.org/abs/2306.10509.

[50] Adam Paszke u. a. „PyTorch: An Imperative Style, High-Performance Deep Learning Library“. In: *Advances in Neural Information Processing Systems*. Bd. 32. 2019. URL: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[51] NVIDIA Corporation. *CUDA C++ Programming Guide*. 2026. URL: https://docs.nvidia.com/cuda/cuda-c-programming-guide/ (besucht am 14. 09. 2026).

[52] Tri Dao u. a. „FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness“. In: *Advances in Neural Information Processing Systems*. Bd. 35. 2022, S. 16344–16359. URL: https://proceedings.neurips.cc/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig, ohne unzulässige fremde Hilfe und unter Beachtung der allgemeinen Grundsätze guter wissenschaftlicher Praxis verfasst und keine anderen als die in der Arbeit angegebenen Quellen und Hilfsmittel verwendet habe. Die aus fremden Quellen direkt oder indirekt übernommenen Texte, Gedankengänge, Konzepte, Abbildungen, Tabellen und sonstigen Inhalte habe ich eindeutig gekennzeichnet und mit vollständigen Verweisen auf die jeweiligen Quellen versehen.

Mir ist bewusst, dass die Nutzung maschinell generierter Inhalte keine Garantie für deren Qualität und Richtigkeit bietet. Ich versichere daher, dass ich generative KI-Systeme lediglich als Hilfsmittel verwendet habe und meine eigenen geistigen und gestalterischen Anteile an der vorliegenden Arbeit überwiegen. Die Verantwortung für alle übernommenen maschinell generierten Inhalte sowie für deren fachliche Prüfung und Überarbeitung liegt vollumfänglich bei mir. Alle verwendeten KI-gestützten Hilfsmittel sind im Folgenden aufgeführt. Ihre Verwendung wird hinsichtlich Zweck, Einsatztiefe und Umfang erläutert.

Verwendung generativer KI-Systeme

Hilfsmittel	**Verwendungszweck**	**Einsatztiefe**	**Umfang**
ChatGPT und OpenAI Codex (OpenAI)	Unterstützung bei Gliederung und Strukturierung, sprachlicher Überarbeitung und Formulierung sowie bei der Literaturrecherche. OpenAI Codex wurde darüber hinaus als Programmierhilfe bei der Entwicklung, Überarbeitung, Fehleranalyse und Dokumentation des Programm- und LaTeX-Codes eingesetzt	Die erzeugten Vorschläge wurden inhaltlich und fachlich geprüft, mit den zugrunde liegenden Quellen beziehungsweise Daten abgeglichen, angepasst und teilweise grundlegend überarbeitet. Literaturangaben wurden vor ihrer Verwendung anhand der Originalveröffentlichungen kontrolliert.	großer Umfang
ArcticSQL, Llama 8B, GPT-OSS 20B, Meta Llama 4 Scout, Qwen2.5- und Qwen3-Varianten sowie ChatGPT	Experimentelle Erzeugung, Reparatur, Rekonstruktion und Prüfung von SQL-Anfragen und Aufgabentexten im Rahmen der entwickelten Pipeline	Die Systeme wurden als Untersuchungswerkzeuge entsprechend der in den Kapiteln 4 und 5 beschriebenen Konfigurationen eingesetzt. Ihre Ausgaben wurden durch die dort dokumentierten automatischen Verfahren analysiert und evaluiert.	großer Umfang als Forschungsinstrument

Halle (Saale), den 17. September 2026 _____

Ort, Datum Paul Göttert